

MariaDB Komplettkurs

Agenda

1. Architektur MariaDB

- [Architecture Server](#)
- [Architecture Server \(Steps\)](#)
- [Storage Engines](#)

2. Installation

- [Installation \(Ubuntu\)](#)
- [Installation Centos/RockyLinux](#)
- [Start/Status/Stop/Enable von MariaDB](#)
- [Does mariadb listen to the outside world](#)

3. Configuration

- [Adjust configuration and restart](#)
- [Set global server system variable](#)

4. Administration / Troubleshooting

- [Create fresh datadir](#)
- [Debug not starting service](#)
- [Debugging not starting service on Windows](#)
- [Langlaufende Queries identifizieren \(die noch laufen\)](#)

5. Information Schema / Status / Processes

- [Show server/session status](#)
- [Kill long running process](#)
- [Kill \(kickout user\) and stop server](#)

6. Upgrade

- [Upgrade Documentation](#)
- [Linux - Upgrade between Major Versions and from MySQL <8](#)
- [MariaDB Upgrade 11.4 -> 11.8 \(debian\)](#)
- [MariaDB Upgrade 11.4 -> 11.8 \(RHEL\)](#)
- [MariaDB Upgrade 10.6 -> 10.11 \(Debian/Ubuntu\)](#)
- [MariaDB Upgrade 10.6 -> 10.11 \(RHEL\)](#)

7. Graphical Tools

- [Overview](#)

8. Database Objects

- [Data Types](#)
- [Create Database](#)
- [Show structure of table](#)
- [Show all tables within db](#)

9. InnoDB - Storage Engine

- [InnoDB - Storage Engine - Structure](#)
- [InnoDB Buffer Pool Size bestimmen und setzen inkl. free buffers](#)
- [Important InnoDB - configuration - options to optimized performance](#)
- [Calculate innodb logfile size](#)

10. Training Data

- [Setup training data "contributions"](#)
- [Setup sakila test db](#)

11. User Rights / Users

- [Create User/Grant/Revoke - Management of users](#)
- [Change password of user](#)
- [Automatisches Einloggen ohne Passwort](#)
- [Disable unix_socket authentication for user](#)
- [Debug and Setup External Connection](#)
- [Get Rights of user](#)

- [Secure with SSL server/client](#)
- [Auth with unix socket](#)
- [User- and Permission-concepts \(best-practice\)](#)
- [Setup external access](#)
- [Users zwingen sich neu anzumelden](#)
- [max_connections pro user festlegen](#)

12. User Authentication

- [ed25519 authentication](#)

13. Binlog, Backup and Restore (Point-In-Time aka PIT)

- [binlog aktivieren und auslesen](#)
- [binlog_format](#)
- [Backup with mysqldump - best practices](#)
- [backup user with mysqldump](#)
- [PIT - Point in time Recovery - Exercise](#)
- [Backup Single Database, Structure and only data](#)
- [Flashback](#)
- [mysqldump-vs-mariabackup](#)
- [mariabackup](#)
- [mariabackup mit windows](#)
- [incrementelles backup mit mariadb](#)
- [Delete binary logs](#)

14. Logging

- [General Log](#)
- [Binary Logs - Activating in Ubuntu](#)
- [Binary Logs - Basics](#)
- [Error logging \(Output of start / stop of mariadb\)](#)

15. Security

- [Table encryption](#)

16. Performance

- [Slow Query Log](#)
- [Percona-toolkit-Installation - Rocky/Redhat und Ubuntu](#)
- [pt-query-digest under Windows](#)
- [pt-query-digest exercise \(Hitliste von slow-query-log erstellen\)](#)
- [Umgang mit grossen Datenbeständen](#)

17. Optimal use of indexes

- [Describe and indexes](#)
- [Find out indexes](#)
- [Index and Functions](#)
- [Index and Likes](#)
- [Find out cardinality without index](#)

18. Joins

- [Overview Joins](#)

19. Monitoring

- [What to monitor?](#)
- [Percona Management and Monitoring](#)
- [Monitoring mit IBM](#)

20. Replication

- [Slave einrichten - gtid \(mit mariabackup\)](#)
- [Slave einrichten - old styke - masterpos](#)
- [Binary Logs auf master bis slave-log-master-position ändern](#)

21. Replication / Galera

- [FAQ](#)

22. Galera

- [How does Galera transfer data IST/SST](#)

- [Installation and Configuration \(Centos/Redhat 8\) with mariabackup as sst](#)
- [Installation and Configuration \(Ubuntu\)](#)
- [Installation and Configuration with mariabackup as sst \(Ubuntu\)](#)
- [1. Node started nicht nach Crash, z.B. Stromausfall](#)
- [Upgrade Minor/Major](#)
- [Determine rights size of gcache - e.g. for maintenance](#)
- [Monitoring Galera - What to monitor](#)

23. Tipps & Tricks

- [Set hostname on systemd-Systems](#)
- [Frisches Datenverzeichnis anlegen](#)
- [In den Root-Benutzer wechseln](#)
- [Service Debuggen](#)
- [online schema change without blocking](#)

24. Locking

- [Table Locks](#)
- [Implicit Locks](#)
- [Identify Deadlocks in innodb](#)

25. Optimal use of indexes

- [Index-Types](#)
- [Describe and indexes](#)
- [Find out indexes](#)
- [Index and Functions](#)
- [Index and Likes](#)
- [profiling-get-time-for-execution-of-query](#)
- [Find out cardinality without index](#)

26. Dokumentation (Releases)

- [Identify Long-Term Support Releases](#)

27. Dokumentation (Galera)

- [MariaDB Galera Cluster](#)
- [MySQL Galera Cluster](#)

28. Dokumentation (Sandbox mode)

- [mariadb-client Sandbox-mode and mariadb-dump \(in newer versions\)](#)

29. Dokumentation (functions)

- [Built-In Functions](#)

30. Dokumentation (Summary Tables)

- [Summary Tables](#)

31. Dokumentation / Library

- [Server System Variables](#)

32. Mehrere Instanzen in Linux starten

- [Multiple instances Linux with systemd](#)

33. Misc

- [Bis zu welcher Größe taugt mariadb](#)

34. Database Functions/Procedure/Triggers/Events

- [Events](#)
- [Event: Beispiel täglich Daten löschen](#)
- [Event: Beispiel mit Daten aggregieren](#)
- [Procedures](#)
- [Functions](#)
- [Triggers](#)

Backlog

1. Architecture of MariaDB

- [Query Cache Usage and Performance](#)
- [Optimizer-Basics](#)

2. Installation

- [Installation SLES15](#)
- [Does mariadb listen to the outside world](#)

3. Backup

- [Use xtrabackup for MariaDB 5.5](#)
- [Ready-made-back-scripts](#)
- [Simple-Backup-Script](#)

4. Information Schema / Status / Processes

- [Show server/session status](#)
- [Kill long running process](#)
- [Kill \(kickout user\) and stop server](#)

5. User Rights

- [Debug and Setup External Connection](#)
- [Get Rights of user](#)
- [Secure with SSL server/client](#)
- [Auth with unix socket](#)
- Authentication and Authorization
- [User- and Permission-concepts \(best-practice\)](#)
- [Setup external access](#)

6. SELinux

- [Welche Ports sind freigegeben? \(MariaDb startet damit\)](#)
- [Neues Datenverzeichnis SELinux bekanntmachen - semanage fcontext](#)
- [Probleme mit SELinux erkennen und debuggen](#)

7. Database - Objects

- [Triggers](#)
- [Functions](#)
- [Stored Procedure](#)
- [Events](#)

8. Upgrade

- [MariaDB Upgrade 10.3 \(Centos\) -> 10.4 \(Mariadb.org\)](#)
- [MariaDB Upgrade 10.4 -> 10.5 \(Centos\)](#)
- [MariaDB Upgrade 5.5 -> 10.5](#)

9. Performance

- [io-Last/CPU-Last](#)
- [Views and performance](#)
- [Partitions and Explain](#)
- [3 Phases of DataSize](#)
- [Slow Query Log](#)
- [profiling-get-time-for-execution-of-query](#)

10. Replication

- [Slave einrichten - Centos - old style \(master_pos\)](#)
- [MaxScale installieren](#)
- [Reference: MaxScale-Proxy mit Monitoring](#)
- [Walkthrough:Automatic Failover Master Slave](#)

11. Tools & Tricks

- [Percona-toolkit-Installation - Ubuntu](#)
- [pt-query-digest - analyze slow logs](#)
- [pt-online-schema-change howto](#)
- [Ubuntu-with-Vagrant](#)
- [mysql-client](#)
- [Schweizer Such-Taschenmesser grep -r](#)
- [Set timezone in Centos 7/8](#)
- [Ist die Netzwerkkarte eingerichtet - nmtui](#)

- [User anlegen und passwort vergeben \(Centos/Redhat\)](#)
- [Scripts for deploying galera-cluster to Ubuntu 20.04](#)

12. Extras

- [User Variables](#)
- [Installation sakila-db](#)

13. Diagnosis and measurement of performance

- [Best practices to narrow down performance problems](#)

14. Performance and optimization of SQL statements

- [Do not use "*" whenever possible](#)
- [Optimizer-hints \(and why you should not use them\)](#)

15. Replication

- [Replikation Read/Write](#)

16. Performance

- [Best Practices](#)
- [Example sys-schema and Reference](#)
- [Change schema online \(pt-online-schema-change\)](#)
- [Optimizer-Hints](#)

17. Documentation / Literature

- [MySQL - Performance Blog](#)
- [Killing connection](#)
- [MariaDB - One Script Installation](#)
- [MariaDB - Information Schema Tables](#)
- [MariaDB - slow query log](#)
- [MariaDB - sys - vor 10.6](#)
- [Differences Community / Enterprise Version - nearly the same](#)
- [Hardware Optimization](#)
- [Source-Code MariaDB](#)
- [Effective MySQL](#)
- [Last Training](#)

18. Questions and Answers

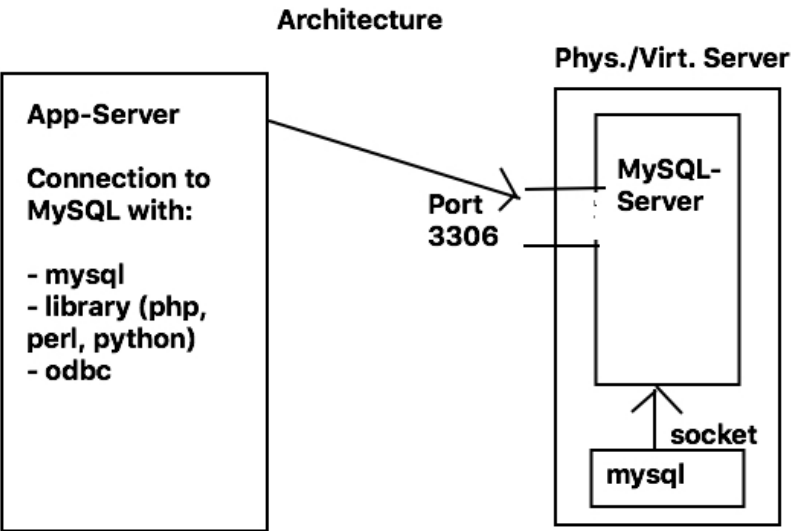
- [Questions and Answers](#)
- [Best filesystem for MariaDB](#)

19. MySQL Do-Nots

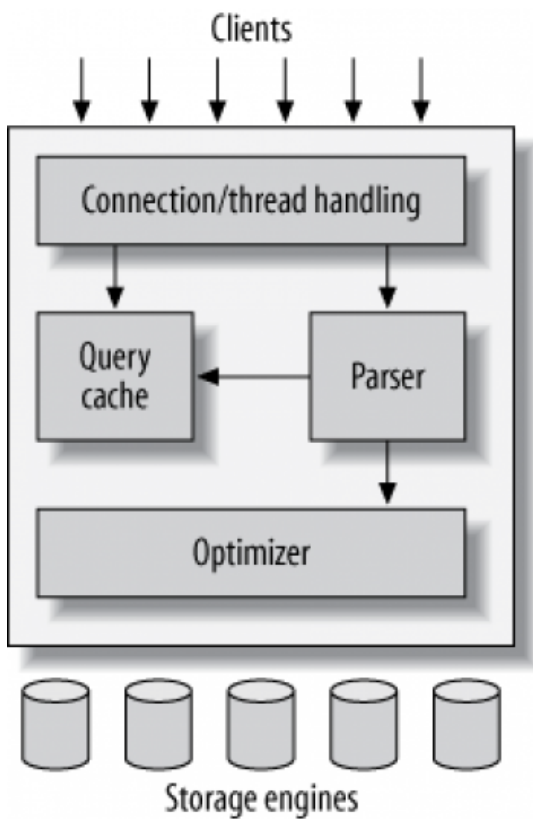
- [mysql-do-nots](#)

Architektur MariaDB

Architecture Server



Architecture Server (Steps)



Storage Engines

Why ?

Let's you choose:
How your data is stored

What ?

- Performance, features and other characteristics you want

Where ?

- Theoretically you can use a different engine for every table
- But: For performance optimization and future, it is better to concentrate on one

What do they do ?

- In charge for: Responsible for storing and retrieving all data stored in MariaDB
- Each storage engine has its:
 - Drawbacks and benefits
- Server communicates with them through the storage engine API
 - this interface hides differences
 - makes them largely transparent at query layer
 - api contains a couple of dozen low-level functions e.g. "begin a transaction", "fetch the row that has this primary key"

Storage Engine do not

- Storage Engines do not parse SQL
- Storage Engines do not communicate with each other

They simply

- They simply respond to requests from the server

Which are the most important one ?

- InnoDB (currently default engine)

- MyISAM/Aria
- Memory
- CSV
- Blackhole (/dev/null)
- Archive
- Partition
- (Federated/FederatedX)

Comparison MyISAM vs. InnoDB

On Detail: MyISAM - Storage Engine

Features

- table locks
- Locks are done table-wide
- no automatic data-recovery
- you can loose more data on crashes than with e.g. InnoDB
- no transactions
- only indices are save in memory through MySQL
- compact saving (data is saved really dense)
- table scans are quick

In Detail: InnoDB - Storage Engine

Features

- support hot backups (because of transactions)
- transactions are supported
- foreign keys are supported
- row-level locking
- multi-versioning
- indexes refer to the data through primary keys
- indexes can quickly get huge in size
 - if size of primary index is not small

Difference MyISAM / Aria

- Crash Recovery (only difference)

Installation

Installation (Ubuntu)

Setup repo and install

[!NOTE] 2025-11-17 MariaDB 11.8 is the latest MariaDB LTS Version. Previous version was 11.4

- <https://downloads.mariadb.org/mariadb/repositories/>

```
sudo apt-get install apt-transport-https curl
sudo mkdir -p /etc/apt/keyrings
sudo curl -o /etc/apt/keyrings/mariadb-keyring.pgp 'https://mariadb.org/mariadb_release_signing_key.pgp'
```

```
sudo nano /etc/apt/sources.list.d/mariadb.sources
```

```
## MariaDB 11.4 repository list - created 2025-03-11 10:13 UTC
## https://mariadb.org/download/
X-Repolib-Name: MariaDB
Types: deb
## deb.mariadb.org is a dynamic mirror if your preferred mirror goes offline. See https://mariadb.org/mirrorbits/
for details.
## URIs: https://deb.mariadb.org/11.4/ubuntu
URIs: https://mirror1.hs-esslingen.de/pub/Mirrors/mariadb/repo/11.4/ubuntu
Suites: noble
Components: main main/debug
Signed-By: /etc/apt/keyrings/mariadb-keyring.pgp
```

```
sudo apt update
sudo apt search mariadb
sudo apt install mariadb-server
```



```
systemctl status mariadb
```

Secure installation

```
mariadb-secure-installation
## OR: if not present before 10.4
mysql_secure_installation
```

Installation Centos/RockyLinux

Install from Distribution

```
dnf search mariadb
## find version
dnf info mariadb-server
dnf install -y mariadb-server mariadb
```

Install from MariaDB Foundation (Repo)

Find Repo Settings

- https://mariadb.org/download/?t=repo-config&d=Red+Hat+Enterprise+Linux+9&v=10.6&r_m=agdsn

Setup Repo MariaDB - Server 11.4

```
## Setup repo
sudo su -
nano /etc/yum.repos.d/MariaDB.repo
```

```
## MariaDB 11.4 RedHatEnterpriseLinux repository list - created 2025-03-11 09:58 UTC
## https://mariadb.org/download/
[mariadb]
name = MariaDB
## rpm.mariadb.org is a dynamic mirror if your preferred mirror goes offline. See https://mariadb.org/mirrorbits/
for details.
## baseurl = https://rpm.mariadb.org/11.4/rhel/$releasever/$basearch
baseurl = https://ftp.agdsn.de/pub/mirrors/mariadb/yum/11.4/rhel/$releasever/$basearch
## gpgkey = https://rpm.mariadb.org/RPM-GPG-KEY-MariaDB
gpgkey = https://ftp.agdsn.de/pub/mirrors/mariadb/yum/RPM-GPG-KEY-MariaDB
gpgcheck = 1
```

```
## Install
sudo dnf install -y MariaDB-server MariaDB-client
sudo systemctl start mariadb # always works - systemd - alias
sudo systemctl status mariadb # Findout real service - name
## like Windows-Autostart
sudo systemctl enable mariadb
sudo systemctl status mariadb
```

Secure installation

- Removes anonymous users (users without username)
- Remove test-db
- Eventually removes root-user connection from outside

```
mariadb-secure-installation
## OR: if not present before 10.4
mysql_secure_installation
```

Start/Status/Stop/Enable von MariaDB

start/stop/status

```
## als root - user
systemctl status mariadb
systemctl stop mariadb
systemctl start mariadb
```

```
##
systemctl restart mariadb
```

enable / disable

- autostart aktivieren (beim Booten des Systems automatisch starten)

```
## enable to be started after reboot
systemctl enable mariadb

## autostart deaktivieren
systemctl disable mariadb

## autostart config abfragen
systemctl is-enabled mariadb
```

how is service configured / systemd-wise

```
systemctl cat mariadb
```

Does mariadb listen to the outside world

How to check ?

```
lsof -i | grep mariadb
## localhost means it does NOT listen to the outside now
## mariabdd 5208          mysql  19u  IPv4  56942      0t0  TCP localhost:mysql (LISTEN)

netstat -tupel
## or
netstat -an
```

How to fix (Ubuntu -> Mariadb Foundation -> 11.4)

```
nano /etc/mysql/mariadb.conf.d/99-server.cnf
```

```
## In Section [mysqld]
## Change bind-address to -> bind-address = 0.0.0.0
[mariabdd]
bind-address = 0.0.0.0
```

```
systemctl restart mariadb
systemctl status mariadb
lsof -i
```

Can I bind to multiple addresses ?

```
## from 10.11 it is possible, before not, you have to use 0.0.0.0
Starting with MariaDB 10.11, a comma-separated list of addresses to bind to can be given.
```

- https://mariadb.com/kb/en/server-system-variables/#bind_address

Configuration

Adjust configuration and restart

```
## change config in /etc/mysql/50-server.cnf
## After that restart server - so that it takes your new config
systemctl restart mariadb
echo $? # Was call restart succesful -> 0
```

Set global server system variable

Find out current value

```
## show global variable
show global variables like '%automatic_sp%'
## or // variable_name needs to be in captitals
use information_schema
select * from global_variables where variable_name like '%AUTOMATIC_SP%';

## If you know the exact name
select @@global.automatic_sp_privileges;
select @@GLOBAL.automatic_sp_privileges;

## Find out session variable, if you know exact name
select @@automatic_sp_privileges;
```

Set global Variable

```
## will be set like so till next restart of mysql server
set global automatic_sp_privileges = 0
```

automatic_sp_privileges can only be set globally

```
## Refer to: server system variable doku

## Has same value in global an session scope
MariaDB [information_schema]> select @@automatic_sp_privileges; select @@global.automatic_sp_privileges;
+-----+
| @@automatic_sp_privileges |
+-----+
| 0 |
+-----+
1 row in set (0.000 sec)

+-----+
| @@global.automatic_sp_privileges |
+-----+
| 0 |
+-----+
1 row in set (0.000 sec)
```

Reference:

- https://mariadb.com/kb/en/server-system-variables/#automatic_sp_privileges

Administration / Troubleshooting

Create fresh datadir

Walkthrough (Centos/RHEL/Rocky)

```
## Schritt 1: Prepare
systemctl stop mariadb
cd /var/lib
## eventually delete old back dir
rm -fR /var/lib/mysql.bkup
##
mv mysql mysql.bkup

## Schritt 2: Fresh
mysql_install_db --user=mysql
chown -R mysql:mysql mysql
chmod -R g+rx,o+rx mysql
restorecon -rv /var/lib/mysql

## Schritt 3: Start
systemctl start mariadb
```

Walkthrough (Debian/Ubuntu)

```

## Schritt 1: Prepare
systemctl stop mariadb
cd /var/lib
## eventually delete old back dir
rm -fR /var/lib/mysql.bkup
##
mv mysql mysql.bkup

## Schritt 2: Fresh
mysql_install_db --user=mysql

## not sure, but safe !
chown -R mysql:mysql mysql
chmod -R g+rx,o+rx mysql

## Schritt 3: Start
systemctl start mariadb

```

Debug not starting service

Fehler erzeugen (Rocky)

```

cd /etc
nano my.cnf.d/server.cnf

```

```

## konfigurationsoption, die es nicht gibt
## in den bereich: [mysqld]
gummitulpe=23M

```

```

## Speichern
systemctl restart mariadb

```

Fehler erzeugen (Ubuntu)

```

cd /etc/mysql/mariadb.conf.d
nano 50-server.cnf

```

```

## konfigurationsoption, die es nicht gibt
## in den Bereich [mariabdb]
gummitulpe=23M

```

```

## Speichern
systemctl restart mariadb

```

Walkthrough

```

## Service is not restarting - error giving
systemctl restart mariadb.service

## Step 1 : status -> what do the logs tell (last 10 lines)
systemctl status mariadb.service

## no findings -> step 2:
journalctl -xeu mariadb.service

## no findings -> step 3:
## search specific log for service
## and eventually need to increase the log level
## e.g. with mariadb (find through internet research)
less /var/log/mysql/error.log
## or
less /var/log/mariadb/mariadb.log

## Nicht fündig -> Schritt 4
## Allgemeines Log

```

```
## Debian/Ubuntu
/var/log/syslog
## Redhat/Centos
/var/log/messages
```

Find errors in logs quickly

```
cd /var/log/mysql
## -i = case insensitive // egal ob gross- oder kleingeschrieben
cat error.log | grep -i error
```

Find configuration - option in config - files

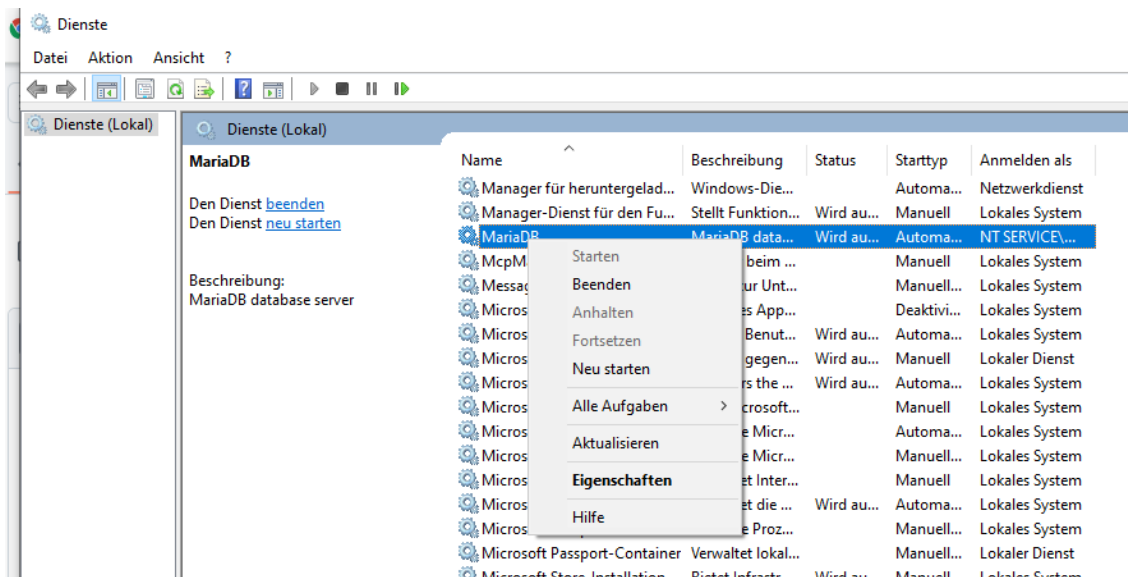
```
grep -r datadir /etc
```

log_warnings hochsetzen

- https://mariadb.com/docs/server/server-management/variables-and-modes/server-system-variables#log_warnings

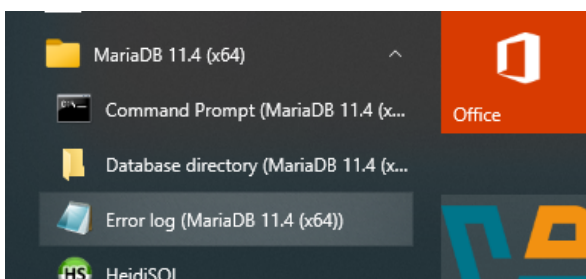
Debugging not starting service on Windows

Step 1: Restart des Dienstes

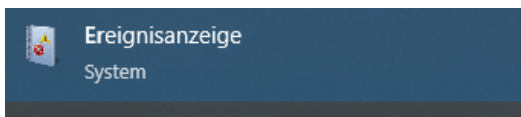


Step 2: Analyse des MariaDB-Logs

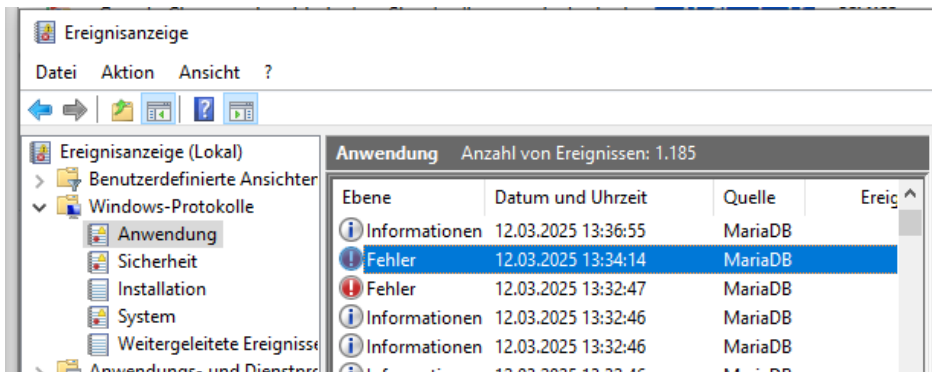
- (keine Einträge, weil nicht gestartet werden konnte)



Step 3: Lösung: Ereignisanzeige anschauen



- Ereignisanzeige(Local) -> Windowsprotokolle -> Anwendung



Langlaufende Queries identifizieren (die noch laufen)

```
SELECT ID FROM information_schema.processlist WHERE Command = 'Query' AND Time > 60;
```

Ref:

- <https://releem.com/blog/managing-long-running-queries-in-mysql#rec746614581>

Information Schema / Status / Processes

Show server/session status

Through mysql

```
## in mysql interface (client)
mysql
status;
```

With mysqladmin

```
mysqladmin status
## or if you want to know more
mysqladmin extended status
```

with mysql -> show status

```
## Status within session (status - counters)
mysql> show status;
## Status global (since last reboot/start of mariadb server)
mysql> show global status;
mysql> -- reset session status
mysql> flush status;
## Show session status
mysql> show session status;
```

Kill long running process

```
## Session 1
## sleep for 120 seconds
select sleep(120)

## Session 2
```

```

show processlist
## kill process you have identified for sleep(120)
MariaDB [(none)]> show processlist;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host      | db      | Command | Time | State      | Info              | Progress |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 36 | root | localhost | NULL    | Query   | 0    | starting   | show processlist | 0.000 |
| 37 | root | localhost | training | Query   | 4    | User sleep | select sleep(120) | 0.000 |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)

## take 37
kill 37

## Session 1: query terminates
ERROR 2013 (HY000): Lost connection to MySQL server during query

```

Kill (kickout user) and stop server

```

MariaDB [mysql]> show processlist;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host      | db      | Command | Time | State      | Info              | Progress |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 30 | root | localhost | mysql   | Sleep   | 10   |           | NULL              | 0.000 |
| 34 | root | localhost | mysql   | Query   | 0    | starting   | show processlist | 0.000 |
| 43 | training | localhost | training | Sleep   | 5    |           | NULL              | 0.000 |
+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

MariaDB [mysql]> quit
Bye
root@its-lu20s04:~# mysql -e 'kill 43' && systemctl stop mariadb
root@its-lu20s04:~#

```

Upgrade

Upgrade Documentation

- <https://mariadb.com/docs/server/server-management/install-and-upgrade-mariadb/upgrading>

Linux - Upgrade between Major Versions and from MySQL <8

Upgrading Between Major MariaDB Versions

MariaDB is designed to allow easy upgrades. You should be able to trivially upgrade from ANY earlier MariaDB version to the latest one (for example MariaDB 10.3.x to MariaDB 10.11.x), usually in a few seconds. This is also mainly true for any MySQL version < 8.0 to MariaDB 10.4 and up.

Referenz:

- <https://mariadb.com/docs/server/server-management/install-and-upgrade-mariadb/upgrading/platform-specific-upgrade-guides/upgrading-on-linux/upgrading-between-major-mariadb-versions>

MariaDB Upgrade 11.4 -> 11.8 (debian)

Step 1: Backup anlegen.

- Eventually not necessary for slave, because we can set it up anyways (with mariabackup from master)
- Best Practice, start with slave

Step 2: Change Version .sources or .list - file

```

## Change version in
## or where you have your repo definition
## Change 11.4 -> 11.8
/etc/apt/sources.list
## or
/etc/apt/sources.list.d/mariadb.sources

```

```
apt update
```

```
systemctl stop mariadb
```

```
apt list --installed | grep -i mariadb
```

```
apt remove -y mariadb-server mariadb-server-core mariadb-common libmariadb3
apt autoremove -y
## Sicherstellen, dass alles weg ist
apt list --installed | grep -i mariadb
```

```
sudo apt install -y mariadb-server # Achtung muss 11.8 sein
apt list --installed | grep -i mariadb # ist wirklich 11.8 installiert.
```

Step 3: Check config and start

```
cd /etc/mysql/mariadb.conf.d/
ls -la 50-server.cnf*
## e.g.
```

```
## Optional !
## normally not needed, server should already be running
systemctl start mariadb
systemctl enable mariadb
```

Step 4: Check if mysql_upgrade already was done ?

```
## Only necessary, if mysql_upgrade_info is not 10.11.x in /var/lib/mysql
mysql_upgrade # After that mysql_upgrade_info will be present in /var/lib/mysql with version-info
```

Reference

- <https://mariadb.com/kb/en/upgrading-from-mariadb-10-6-to-mariadb-10-11/>

MariaDB Upgrade 11.4 -> 11.8 (RHEL)

Walkthrough

```
## Step 0;
## Sicherung anlegen (mysqldump / mariabackup)
```

```
## Step 1:
## Change version in
## or where you have your repo definition
## Change 11.4 -> 11.8
cd /etc/yum.repos.d/
nano MariaDB.repo
```

```
## Change version in file from 11.4 -> 11.8
## Save + quit
```

```
## Step 2:
systemctl stop mariadb
```

```
## Step 3
dnf remove -y MariaDB-*
## verify nothing is present
dnf list installed | grep -i mariadb
```

```
## Step 4
dnf install -y MariaDB-server MariaDB-backup
dnf list --installed | grep -i mariadb # ist wirklich 11.8 installiert.
```

```
## Step 4.5
## Check if old config files were saved as .rpmsave after delete of package 11.4
```



```
cd /etc/my.cnf.d/
ls -la server.cnf
## Eventually consolidate everything in one file loaded as last entry, e.g.
## z_settings.cnf

## Step 5:
systemctl start mariadb
systemctl enable mariadb

## Only necessary, if mysql_upgrade_info is not 11.8 in /var/lib/mysql
mariadb-upgrade # After that mysql_upgrade_info will be present in /var/lib/mysql with version-info
```

Reference:

- [Upgrade mariadb 11.4 -> 11.8](#)

MariaDB Upgrade 10.6. -> 10.11 (Debian/Ubuntu)

Step 1: Backup anlegen.

- Eventually not necessary for slave, because we can set it up anyways (with mariabackup from master)
- Best Practice, start with slave

Step 2: Change Version .sources or .list - file

```
## Change version in
## or where you have your repo definition
## Change 10.6 -> 10.11
/etc/apt/sources.list
## or
/etc/apt/sources.list.d/mariadb.sources
```

```
apt update
```

```
systemctl stop mariadb
```

```
apt list --installed | grep -i mariadb
```

```
apt remove -y mariadb*10.6 libmariadb3
apt autoremove -y
```

```
sudo apt install -y mariadb-server # Achtung muss 10.11 sein
apt list --installed | grep -i mariadb # ist wirklich 10.11 installiert.
```

Step 3: Check config and start

```
cd /etc/mysql/mariadb.conf.d/
ls -la 50-server.cnf*
## e.g.
```

```
systemctl start mariadb
systemctl enable mariadb
```

Step 4: Check if mysql_upgrade already was done ?

```
## Only necessary, if mysql_upgrade_info is not 10.11.x in /var/lib/mysql
mysql_upgrade # After that mysql_upgrade_info will be present in /var/lib/mysql with version-info
```

Reference

- <https://mariadb.com/kb/en/upgrading-from-mariadb-10-6-to-mariadb-10-11/>

MariaDB Upgrade 10.6 -> 10.11 (RHEL)

Walkthrough

```
## Step 0;
## Sicherung anlegen (mysqldump / mariabackup)
```

```
## Step 1:
## Change version in
## or where you have your repo definition
## Change 10.6 -> 10.11
cd /etc/yum.repos.d/
nano MariaDB.repo
```

```
## Change version in file from 10.6 -> 10.11
## Save + quit
```

```
## Step 2:
systemctl stop mariadb
```

```
## Step 3
dnf remove -y MariaDB-*
## verify nothing is present
dnf list installed | grep -i mariadb
```

```
## Step 4
dnf install -y MariaDB-server MariaDB-backup
dnf list --installed | grep -i mariadb # ist wirklich 10.11 installiert.
```

```
## Step 4.5
## Check if old config files were saved as .rpmsave after delete of package 10.4
cd /etc/my.cnf.d/
ls -la server.cnf
## Eventually consolidate everything in one file loaded as last entry, e.g.
## z_settings.cnf
```

```
## Step 5:
systemctl start mariadb
systemctl enable mariadb
```

```
## Only necessary, if mysql_upgrade_info is not 10.11.x in /var/lib/mysql
mysql_upgrade # After that mysql_upgrade_info will be present in /var/lib/mysql with version-info
```

Reference:

- <https://mariadb.com/kb/en/upgrading-from-mariadb-10-6-to-mariadb-10-11/>

Graphical Tools

Overview

```
DataGrip jetbrains
HeidiSQL
HeidiSQL - über ssh-tunnel https://marcus-obst.de/wiki/Database%20-%20HeidiSQL%20SSH%20Tunnel%20Setup
```

Database Objects

Data Types

- <https://mariadb.com/kb/en/data-types/>

Create Database

```
create schema training
create database training
```

Show structure of table

show create table

```
use mysql;
show create table user;
-- better output for huge rows
show create table user \G
```

describe table

```
use mysql;
describe user;
```

Show all tables within db

show all tables in database

```
## connect with db training
mysql training
mysql> show tables;
|training|
```

describe

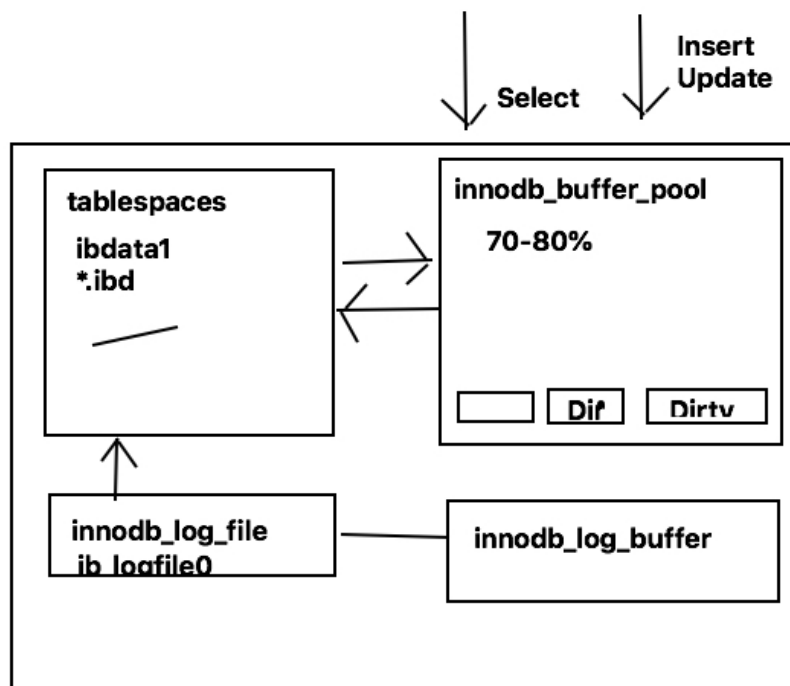
```
MariaDB [training]> describe mitarbeiter;
+-----+-----+-----+-----+-----+-----+
| Field | Type                | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | tinyint(3) unsigned | NO   | PRI | NULL    |       |
| name  | varchar(50)         | YES  |     | NULL    |       |
| vorname | varchar(30)        | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.001 sec)
```

show create

```
MariaDB [training]> show create table mitarbeiter;
+-----+-----+
+-----+-----+
| Table          | Create Table
+-----+-----+
+-----+-----+
| mitarbeiter | CREATE TABLE `mitarbeiter` (
  `id` tinyint(3) unsigned NOT NULL,
  `name` varchar(50) DEFAULT NULL,
  `vorname` varchar(30) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 |
+-----+-----+
+-----+-----+
1 row in set (0.000 sec)
```

InnoDB - Storage Engine

InnoDB - Storage Engine - Structure



InnoDB Buffer Pool Size bestimmen und setzen inkl. free buffers

Schritt 1: Herausfinden, wie unser innodb_buffer_pool eingestellt ist ?

```
mysql
```

```
## in mysql
select @@innodb_buffer_pool_size;
show variables like 'innodb%buffer%';
exit
```

Schritt 2: Arbeitsgröße und Innodb Buffer Pool Size berechnen

```
## wie gross ist unser Arbeitsspeicher auf dem server
top
## q
## oder
free
```

```
## berechnen einer guten Größe
## mysql -e 'select <speichergröße>/10 * 8'
mariadb -e 'select 3.8/10 * 8'
```

Schritt 3: innodb_buffer_pool_size in config setzten

```
## Variante 1: Centos / Redhat
cd /etc/my.cnf.d/
nano server.cnf
```

```
## Variante 2:
cd /etc/mysql/mariadb.conf.d
nano 50-service.cnf
```

```
## unter mysqld - sektion eintrage
innodb-buffer-pool-size=2500M # 3G
```

```
## server neu starten
systemctl restart mariadb
```

Schritt 4: Überprüfen und freien Buffer rausfinden

```
mysql

-- konfigurierte Größe
select @@innodb_buffer_pool_size;

-- freien Seiten ermitteln
show status like '%free%';
show engine innodb status \G

-- verwendete = freie seiten * 16 * 1024
```

Important InnoDB - configuration - options to optimized performance

How big is the innodb buffer currently (setup) ?

```
mysql>select @@innodb_buffer_pool_size;
mysql>show variables like '%buffer%';
```

Innodb buffer pool

- How much data fits into memory
- Free buffers = pages of 16 Kbytes
- Free buffer * 16Kbytes = free innodb buffer pool in KByte

```
## does not in windows -> pager grep
pager grep -i 'free buffers'
## does not work with workbench or heidisql because of formatting + \G only works in client
show engine innodb status \G
Free buffers          7905
1 row in set (0.00 sec)
```

Innodb buffer pool stats with status

```
## Also works in heidisql or workbench
show status like '%buffer%';
```

Overview innodb server variables / settings

- <https://dev.mysql.com/doc/refman/5.7/en/innodb-parameters.html>

Change innodb_buffer_pool

```
## /etc/mysql/mysql.conf.d/mysqld.cnf
## 70-80% of memory on dedicated mysql
[mysqld]
innodb-buffer-pool-size=6G

##
systemctl restart mysql

##
mysql
mysql>show variables like 'innodb%buffer%';
```

problems, when dynamically increasing buffer

- <https://www.percona.com/blog/2018/06/19/chunk-change-innodb-buffer-pool-resizing/>

innodb_log_buffer_size

```
1 commit should fit in this buffer
```

Question: In your application are your commits bigger or smaller

innodb_flush_method

Ideally O_DIRECT on Linux, but please test it, if it really works well.

innodb_flush_log_at_trx_commit

When is flushing done from innodb_log_buffer to log.

Default: 1 : After every commit

-> best performance 2. -> once per second

Good to use 2, if you are willing to loose 1 second of data on powerfail

innodb_flush_neighbors

on ssd disks set this to off, because there is no performance improvement

innodb_flush_neighbors=0

Default = 1

innodb_log_file_size

Should hold 60-120 min of data flow

Calculate like so:

<https://www.percona.com/blog/2008/11/21/how-to-calculate-a-good-innodb-log-file-size/>

skip-name-resolv.conf

work only with ip's - better for performance

/etc/my.cnf.d/mariadb-server.cnf

skip-name-resolve

- <https://nixcp.com/skip-name-resolve/>

Ref:

- <https://dev.mysql.com/doc/refman/5.7/en/innodb-buffer-pool-resize.html>

Privileges for show engine innodb status

```
show engine innodb status \G
```

ERROR 1227 (42000): Access denied; you need (at least one of) the PROCESS privilege(s) for this operation

Calculate innodb logfile size

Exercise (simple)

```
pager grep sequence;
show engine innodb status;
select sleep(60);
show engine innodb status;
```

```
mysql> select (3838334638 - 3836410803) / 1024 / 1024 as MB_per_min;
```

```
+-----+
| MB_per_min |
+-----+
| 1.83471203 |
+-----+
```

- <https://www.percona.com/blog/how-to-calculate-a-good-innodb-log-file-size/>

Exercise 2: with sakila

```
Open 2 sessions
Session 1: mysql to measure
Session 2: import sakila
```

Step 1: In Session 1 (measure)

```
mysql
```

```
pager grep sequence;
show engine innodb status;
select sleep(120);
show engine innodb status;
```

Step 2: In Session 2 (import sakila)

```
cd /usr/src/sakila-db
mysql < sakila-schema.sql; mysql < sakila-data.sql;
```

Step 3: In Session 1 (measure) - Calculate values

```
-- Calculate values
-- second-value-from-step1 - first-value-from-step1
-- select (second_value - first_value) / 1024 / 1024 as MB_per_2min

pager;
## eg. 312MB
select (3838334638 - 3836410803) / 1024 / 1024 as MB_per_2min;
exit
```

Step 4: Adjust config accordingly

```
## Ubuntu / Debian
cd /etc/mysql/mariadb.conf.d/
nano 50-server.cnf
```

```
## mysqld section
[mysqld]
## other settings
innodb-log-file-size=312M
```

```
systemctl restart mariadb
mysql
```

```
select @@innodb_log_file_size
## oder
show variables like '%log_file%';
exit
```

Training Data

Setup training data "contributions"

Walkthrough

- Complete process takes about 10 minutes

```
cd /usr/src
apt update; apt install -y git
git clone https://github.com/jmetzger/dedupe-examples.git
cd dedupe-examples
cd mysql_example
## Eventually you need to enter (in mysql_example/mysql.cnf)
## Only necessary if you cannot connect to db by entering "mysql"
```

```
## password=<your_root_pw>
./setup.sh
```

Setup sakila test db

```
cd /usr/src
wget https://downloads.mysql.com/docs/sakila-db.tar.gz
tar xvf sakila-db.tar.gz

cd sakila-db
mariadb < sakila-schema.sql
mariadb < sakila-data.sql
```

User Rights / Users

Create User/Grant/Revoke - Management of users

Create user

```
create user training@localhost identified by 'deinpasswort';
```

Exercise 1: create user

In Session 1: (mysql - user - root)

```
## Als root: 1. Nutzer training anlegen, der sich von lokal anmelden kann
create user training@localhost identified by 'deinpasswort';
## Wir zeigen uns die Rechte an:
SHOW GRANTS FOR training@localhost;
```

In Session 2:

```
## anmelden mit nutzer training über mysql-client
## Passwort eingeben
mysql -utesting -p
```

```
## 4. Anschauen, welchen Rechte wir als dieser Nutzer haben
show grants;
show databases;
select * from information_schema.schemata \G
use sakila;
```

Exercise 1a: privileges anpassen / alle Rechte

In Session 1: mysql -> root

```
GRANT ALL ON *.* TO training@localhost;
show grants for training@localhost;
```

In Session 2: mysql -> training

```
## das geht noch nicht
create schema planung;
exit;
```

```
mysql -utesting -p
```

```
## jetzt geht es
create schema planung;
```

Exercise 1b: privileges anpassen / nur SELECT

In Session 1: mysql -> root


```
REVOKE ALL ON *.* FROM training@localhost;
show grants for training@localhost;

GRANT SELECT ON *.* TO training@localhost;
show grants for training@localhost;
```

In Session 2: mysql -> training

```
use sakila;
## should not work but does work
update actor set first_name = 'johanna' where actor_id = 1;
exit;
```

```
mysql -utrainng -p
```

```
## jetzt geht es nicht mehr
use sakila;
update actor set first_name = 'johanna' where actor_id = 1;
## aber das geht
select * from actor where actor_id = 1;
```

Exercise 1c: Drop user (=delete user)

```
## as user root
drop user training@localhost
```

Exercise 2: create external user with privileges

Schritt 1 (auf Remote-Server):

```
Variante 1:
## Auf dem Remote-System, auf dem der Server läuft (m[1-6].t3isp.de)
## Als root: 1. Nutzer ext anlegen der von überall aus zugreifen darf '%'
```

```
Variante 2:
create user ext@'192.168.56.%' identified by 'password';
```

```
## Achtung. Bei Redhat/Rocky - Firewall freischalten
firewall-cmd --list-all
firewall-cmd --info-service=mysql
firewall-cmd --add-service=mysql
firewall-cmd --runtime-to-permanent
```

Schritt 2 (auf lokalen Server):

```
## von entfernten System aus, auf dem ein mysql-client existiert (bei uns server1)
## Verbindung aufbauen
mysql -uext -p -h <ip-des-remote-servers>
```

```
-- hier erfahren unsere ip - adresse
status;
show databases;
show grants;
exit;
```

Schritt 3: Remote-Server Set db rights for a user

```
grant all on sakila.* to ext@'192.168.56.%;'
```

Schritt 4: Local System

```
## on local system test connection
mysql -uext -p -h<ip des remoteserver>
```

```
show grants;
show databases;
exit;
```

Refs:

- <https://mariadb.com/kb/en/grant/#the-grant-option-privilege>
- <https://mariadb.com/kb/en/revoke/>

Change password of user

```
## you must be root or privileged to changed passwords
alter user training@localhost identified by 'password';
```

Automatisches Einloggen ohne Passwort

```
cat /home/kurs/.my.cnf
[mysql]
user=training
password=password
```

```
## einloggen als training
mysql
```

Disable unix_socket authentication for user

```
## before
show grants for root@localhost;
GRANT ALL PRIVILEGES ON *.* TO `root`@`localhost` IDENTIFIED VIA mysql_native_password USING
'*2470C0C06DEE42FD1618BB99005ADCA2EC9D1E19' OR unix_socket
```

```
##after
alter user root@localhost identified by 'meinpasswort';
```

Debug and Setup External Connection

Prerequisites

```
client1: 192.168.56.104
server1: 192.168.56.103
```

Step 1: Be sure server is communicating to the outside

```
lsof -i
## should be
*:mysql
```

Step 2: Test connection from client

```
mysqladmin ping -h 192.168.56.103
## on succesful connection also without authentication
echo $?
0 # 0 was success also without proper authentication

## Bad news, if
echo $?
1
## Could not connect at all
```

Step 2a: No connection possible ? check Firewall

```
## Server 1
systemctl status firewalld
firewall-cmd --state
firewall-cmd --list-all # do we see mysql as a service
```

```
## no ?
firewall-cmd --get-services
firewall-cmd --add-service=mysql # only in runtime
firewall-cmd --runtime-to-permanent # config - works after reboot

### Recheck with Step 2
```

Step 3: Setup user without grants - Server1

```
## Server 1
mysql> create user ext@192.168.56.104 identified by 'topsecretpassword';
## Doing this twice triggers an weird error
```

Step 3a: test connection from client - Client 1

```
mysql -uext -p -h 192.168.56.103
## on success
mysql>show grants
## should only be usage
mysql>show schemas
```

Step 3b: Add privileges (testing giving all) - Server1

```
## *.* = all databases and all tables
mysql> GRANT ALL ON *.* TO ext@192.168.56.104
```

Step 3c: See, if we have grants - Client 1

```
mysql>show grants
## grants will be shown but do not work yet
## we need to reconnect
mysql>quit
mysql -uext -p -h 192.168.56.103
mysql> -- now it works
```

Get Rights of user

Root can show rights of a specific user

```
## shows the right of the logged in user (you as a user)
show grants;

## show grants for a specific user
## no need for ' (quotes) if there are not special chars withing
## e.g.
show grants for training@localhost;
## if there are special chars, use quotes
show grants for 'mariadb.sys'@localhost;

## if you want to see rights of a user that has rights from everywhere
show grants for training@'%';
```

If you cannot remember the exact user (user@host) look it up

```
## within mysql client
use mysql
select * from user \G
```

Secure with SSL server/client

Variant 1: Setup 1-way ssl encryption

Create CA and Server-Key

```
## On Server - create ca and certificates
sudo mkdir -p /etc/my.cnf.d/ssl
sudo cd /etc/my.cnf.d/ssl

## create ca.
sudo openssl genrsa 4096 > ca-key.pem

## create ca-certificate
## Common Name: MariaDB CA
sudo openssl req -new -x509 -nodes -days 365000 -key ca-key.pem -out ca-cert.pem

## create server-cert
## Common Name: server1.training.local
## Password: --- leave empty ---
sudo openssl req -newkey rsa:2048 -days 365000 -nodes -keyout server-key.pem -out server-req.pem

## Next process the rsa - key
sudo openssl rsa -in server-key.pem -out server-key.pem

## Now sign the key
sudo openssl x509 -req -in server-req.pem -days 365000 -CA ca-cert.pem -CAkey ca-key.pem -set_serial 01 -out
server-cert.pem
```

Verify certificates

```
openssl verify -CAfile ca-cert.pem server-cert.pem
```

Configure Server

```
## create file
## /etc/my.cnf.d/z_ssl.cnf
[mysqld]
ssl-ca=/etc/my.cnf.d/ssl/ca-cert.pem
ssl-cert=/etc/my.cnf.d/ssl/server-cert.pem
ssl-key=/etc/my.cnf.d/ssl/server-key.pem
### Set up TLS version here. For example TLS version 1.2 and 1.3 ##
## Starts from mariadb 10.4.6 not possible before. !!!!
tls_version = TLSv1.2,TLSv1.3

## Set ownership
chown -vR mysql:mysql /etc/my.cnf.d/ssl/
```

Restart and check for errors

```
systemctl restart mariadb
journalctl -u mariadb
```

Test connection on client

```
## only if we use option --ssl we will connect with ssl
mysql --ssl -uxyz -p -h <ip-of-server>
mysql>status
SSL:                                Cipher in use is TLS_AES_256_GCM_SHA384
```

Force to use ssl

```
## on server
## now client can only connect, when using ssl
mysql> grant USAGE on *.* to remote@10.10.9.144 require ssl;
```

Variant 2: 1-way ssl-encryption but checking server certificate

Prerequisites

```
server1: 192.168.56.103
client1: 192.168.56.104
```

Copy ca-cert to client

```
## on server1
cd /etc/my.cnf.d/ssl
scp ca-cert.pem kurs@192.168.56.104:/tmp

## on clien1
cd /etc/my.cnf.d
mkdir ssl
cd ssl
mv /tmp/ca-cert.pem .
```

Configure client1 - client -config

```
sudo vi /etc/my.cnf.d/mysql-clients.cnf

Append/edit in [mysql] section:

### MySQL Client Configuration ##
ssl-ca=/etc/my.cnf.d/ssl/ca-cert.pem

### Force TLS version for client too
##tls_version = TLSv1.2,TLSv1.3
#### This option is disabled by default ###
#### ssl-verify-server-cert ###

## only works if you have no self-signed certificate
ssl-verify-server-cert
ssl

## domain-name in hosts setzen
## because in dns
vi /etc/hosts
192.168.56.103 server1.training.local

## now you to connect with hostname
## otherwise no check against certificate can be done
mysql -uext -p -h server1.training.local

## if it does not work, you get
ERROR 2026 (HY000): SSL connection error: Validation of SSL server certificate failed
```

Variant 3: 2-way - Security (Encryption) - validated on server and client

Client - Create certificate on server

- we are using the same ca as on the server

```
## on server1
cd /etc/my.cnf.d/ssl
## Bitte Common-Name: MariaDB Client
openssl req -newkey rsa:2048 -days 365 -nodes -keyout client-key.pem -out client-req.pem

## process RSA - Key
## Eventually also works without - what does it do ?
## openssl rsa -in client-key.pem -out client-key.pem

## sign certificate with CA
openssl x509 -req -in client-req.pem -days 365 -CA ca-cert.pem -CAkey ca-key.pem -set_serial 01 -out client-cert.pem
```

Client - Zertifikate validieren

```
openssl verify -CAfile ca-cert.pem client-cert.pem
```

Zertifikate für Client zusammenpacken

```
mkdir cl-certs; cp -a client* cl-certs; cp -a ca-cert.pem cl-certs ; tar cvfz cl-certs.tar.gz cl-certs
```

Zertifikate auf Client transferieren

```
scp cl-certs.tar.gz kurs@192.168.56.104:/tmp
```

Zertifikate einrichten

```
## on client1
## cleanup old config
rm /etc/my.cnf.d/ssl/ca-cert.pem

mv /tmp/cl-certs.tar.gz /etc/my.cnf.d/ssl
cd /etc/my.cnf.d; tar xzvf cl-certs.tar.gz

vi mysql-clients.cnf
[mysql]
ssl-ca=/etc/my.cnf.d/cl-certs/ca-cert.pem
ssl-cert=/etc/my.cnf.d/cl-certs/client-cert.pem
ssl-key=/etc/my.cnf.d/cl-certs/client-key.pem
```

Test the certificate

```
## on server1 verify: X509 for user
select user,ssl_type from mysql.user where user='ext'

## connect from client1
## Sollte die Verbindung nicht klappen stimmt auf dem
## Client etwas mit der Einrichtung nicht
mysql -uext -p -h192.168.56.103
mysql> status
```

Ref

- <https://mariadb.com/kb/en/securing-connections-for-client-and-server/>
- <https://www.cyberciti.biz/faq/how-to-setup-mariadb-ssl-and-secure-connections-from-clients/>

Auth with unix_socket

```
mysql>create user training@localhost identified via unix_socket
useradd training
passwd training

## testing
su - training
## mysql
## should not work without password
## Be sure, that use has access to socket
cd /var/lib/mysql
ls -la mysql.socket
```

User- and Permission-concepts (best-practice)

```
## user should have as little permissions as possible
## so many as needed ;o)
MariaDB [mysql]> create database eventplanner;
Query OK, 1 row affected (0.000 sec)

MariaDB [mysql]> create user eventplanner@localhost identified by 'eventplanner';
Query OK, 0 rows affected (0.001 sec)

MariaDB [mysql]> grant all on eventplanner.* to eventplanner@localhost;
Query OK, 0 rows affected (0.003 sec)
```

Setup external access

Testing

```
## Where .104 is the server you want to connect to
## Variante 1
mysqladmin ping -h 192.168.56.104
echo $?
-> 0 // it is possible to reach mysql - server

## Variante 2
mysqladmin ping -h 192.168.56.104
echo $?
-> 1 // i cannot reach mysql-server -> port might close / firewall ?

## or use telnet
telnet 192.168.56.104 3306
```

Checks on MariaDB (Theory)

- Is MariaDB - Server running ?
- Is 3306 port open (exposed to the outside)
- Is firewall open for port 3306
- Is there a valid user, who connect)

Checks on MariaDB (Practical)

```
## Step 1: Running
systemctl status mariadb

## Step 2: Port open ?
lsof -i # does it listen to all interfaces. -> *
        # or an external interface

## Step 3: Firewall open -> see next block
## Step 4: User who can connect ?
```

Checks on Firewall.

```
## Is firewall running and enabled
systemctl status firewalld
firewall-cmd --state

## Is interface setup for usage of firewalld
firewall-cmd --get-active-zones

## Is service "mysql" in zones
firewall-cmd --list-all-zones | less # is it within public - zone -> mysql

## To enable it, if not set
firewall-cmd --add-service=mysql --zone=public --permanent # writes to filesystem config
firewall-cmd --reload # rereads settings from filesystem
```

Setup valid user

```
## on server you want to connect to
mysql> create user extern@'192.168.56.%' identified by 'mysecretpass'
mysql> grant all on sakila.* to extern@'192.168.56.%'
```

```
## alternative with subnet mask
CREATE USER 'maria'@'247.150.130.0/255.255.255.0';
```

Now test from external with mysql

```
mysql -uextern -p -h 192.168.56.104
mysql> show databases;
```

Users zwingen sich neu anzumelden

```

## Session 1
## sleep for 120 seconds
select sleep(120)

## Session 2
show processlist
## kill process you have identified for sleep(120)
MariaDB [(none)]> show processlist;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host      | db      | Command | Time | State      | Info                | Progress |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 36 | root | localhost | NULL    | Query   | 0    | starting   | show processlist    | 0.000 |
| 37 | root | localhost | training | Query   | 4    | User sleep | select sleep(120)    | 0.000 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)

## take 37
kill 37

## Session 1: query terminates
ERROR 2013 (HY000): Lost connection to MySQL server during query

```

max_connections pro user festlegen

```
ALTER USER ext@'192.168.56.%' WITH MAX_USER_CONNECTIONS 10
```

Reference

- <https://mariadb.com/docs/server/reference/sql-statements/account-management-sql-statements/alter-user>

User Authentication

ed25519 authentication

Walkthrough / Exercise

Prerequisites

- Open 2 sessions (same server - Makes things a bit clearer)

Step 1: Session 1:

```

INSTALL SONAME 'auth_ed25519.so';
-- that one is being loaded now on every startup
use mysql;
select * from plugin;

-- Create user
CREATE USER alice@localhost IDENTIFIED VIA ed25519 USING PASSWORD('secret');

```

Step 2: Session 2:

```
## connecting through localhost
mysql -ualice -p
```

```
show grants;
```

Ref:

- <https://mariadb.com/kb/en/authentication-plugin-ed25519/>

Binlog, Backup and Restore (Point-In-Time aka PIT)

binlog aktivieren und auslesen

Binlog - Wann ?

- PIT (Point-in-Time) - Recovery
- Master/Slave - Replication
- Galera Cluster (MariaDB Cluster)

Binlog aktivieren (Centos)

```
## /etc/my.cnf.d/server.cnf
[mysqld]
log-bin

## Server neu starten
systemctl restart mariadb
```

Alte Logs automatisch löschen

- https://mariadb.com/kb/en/replication-and-binary-log-system-variables/#expire_logs_days

Rowbasiertes Logging aktivieren

```
## /etc/my.cnf.d/server.cnf
[mysqld]
log-bin
## Server neu starten
systemctl restart mariadb
```

binlog auslesen

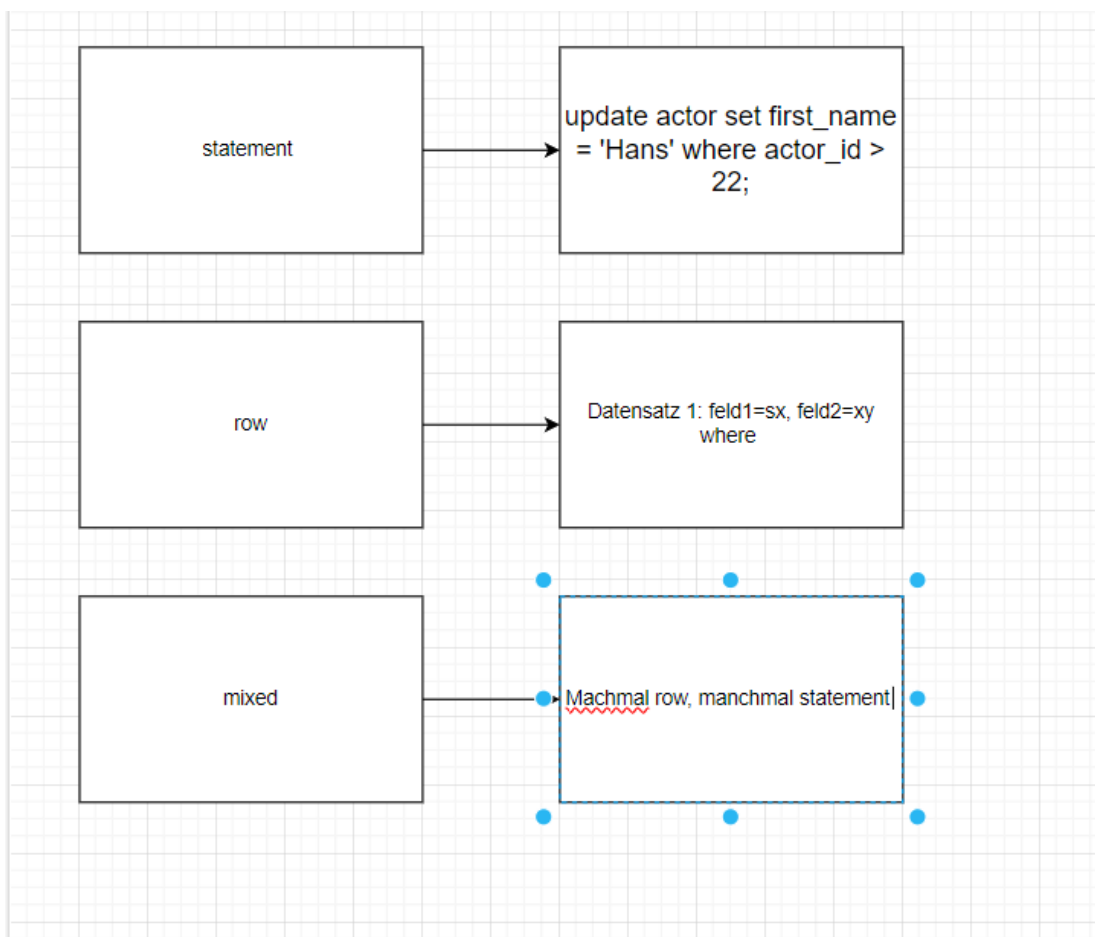
```
cd /var/lib/mysql
## Zeigt auch mit Kommentar die SQL-Statements an die bei ROW-basierten binlog ausgeführt werden
mysqlbinlog -vv mysql-bin.000001
```

Wie finde ich raus, welches binlog aktiv ist ?

```
## mysql -client starten
mysql> show master status;
```

binlog_format

Wirkweise



Format herausfinden, welches eingestellt ist

```
select @@binlog_format;
```

Backup with mysqldump - best practices

Dumping (best option) without active binary log

```
mariadb-dump --all-databases --single-transaction > /usr/src/all-databases.sql
## if you want to include procedures use --routines
## with event - scheduled tasks
mariadb-dump --all-databases --single-transaction --routines --events > /usr/src/all-databases.sql
```

Windows-Version

```
mariadb-dump -uroot -p --all-databases --single-transaction --routines --events >
C:\Users\Administrator\Desktop\all-databases.sql
```

Useful options for PIT

```
## -quick not needed, because included in -opt which is enabled by default

## on local systems using socket, there are no huge benefits concerning --compress
## when you dump over the network use it for sure
mariadb-dump --all-databases --single-transaction --routines --events --master-data=2 --flush-logs >
/usr/src/all-databases.sql;
```

With PIT_Recovery you can use --delete-master-logs

- All logs before flushing will be deleted

```
mariadb-dump --all-databases --single-transaction --gtid --master-data=2 --routines --events --flush-logs --delete-master-logs > /usr/src/all-databases.sql;
```

```
## from mariadb 11.4
mariadb-dump --all-databases --single-transaction --gtid --master-data=2 --routines --events --flush-logs --delete-master-logs > /usr/src/all-databases.sql;
```

Flush binary logs from mysql

```
mariadb -e "PURGE BINARY LOGS BEFORE '2013-04-22 09:55:22'";
```

Version with zipping

```
mariadb --all-databases --single-transaction --gtid --master-data=2 --routines --events --flush-logs --compress | gzip > /usr/src/all-databases.sql.gz
```

Performance Test mysqldump (1.7 Million rows in contributions)

```
date; mariadb-dump --all-databases --single-transaction --gtid --master-data=2 --routines --events --flush-logs --compress > /usr/src/all-databases.sql; date
Mi 20. Jan 09:40:44 CET 2021
Mi 20. Jan 09:41:55 CET 2021
```

Seperated sql-structure files and data-txt files including master-data for a specific database

```
# backups needs to be writeable for mysql
mkdir /backups
chmod 777 /backups
chown mysql:mysql /backups
mariadb-dump --tab=/backups contributions
mariadb-dump --tab=/backups --master-data=2 contributions
mariadb-dump --tab=/backups --master-data=2 contributions > /backups/master-data.tx
```

Create new database base on sakila database

```
cd /usr/src
mariadb-dump sakila > sakila-all.sql
echo "create database mynewdb" | mariadb
mariadb mynewdb < sakila-all.sql
```

backup user with mysqldump

General

- What are the minimum rights, needed (single-transaction not lock-tables (default))

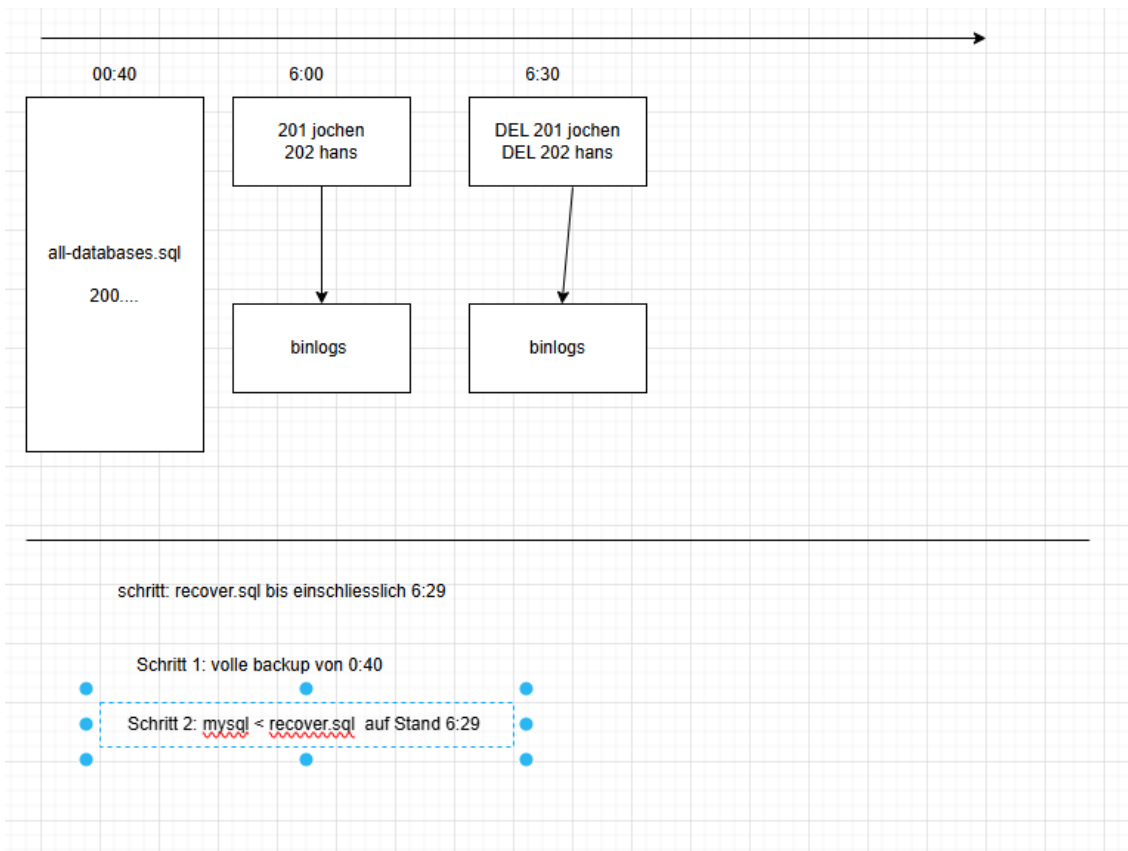
Walktrough

```
CREATE USER backup@localhost identified by 'yoursupersecretpassword';
GRANT SELECT, SHOW VIEW, EVENT, TRIGGER ON sakila.* TO backup@localhost;
```

```
## how to use it
mysqldump -ubackup -p --single-transaction --routines --events sakila > /usr/src/sakila-db.sql
```

PIT - Point in time Recovery - Exercise

Planning



Problem coming up

```
## Step 1 : Create full backup (assuming 24:00 o'clock)
mariadb-dump --all-databases --single-transaction --master-data=2 --routines --events --flush-logs --delete-master-logs > /usr/src/all-databases.sql;

## Step 1.5: look into data
mariadb
mysql>use sakila;
mysql>select * from actor;

## Step 2: Working on data
mysql>use sakila;
mysql>insert into actor (first_name,last_name) values ('john','The Rock');
mysql>insert into actor (first_name,last_name) values ('johanne','Johannson');
mysql> exit;

## Optional: Step 3: Looking into binary to see this data
cd /var/lib/mysql
## last binlog
mariadb-binlog -vv mariadb-bin.000005

## Step 4: Some how a guy deletes data
mariadb
mysql>use sakila; delete from actor where actor_id > 200;
## now only 200 datasets
mysql>use sakila; select * from actor;
mysql>exit;
```

Fixing the problem

```
## find out the last binlog
## Simple take the last binlog

cd /var/lib/mysql
## Find the position where the problem occurred
## Look into
## mariadb-binlog -vv mysqld-bin.000005
## and create a recover.sql - file (before apply full backup)
mariadb-binlog -vv --stop-position=857 mysqld-bin.000005 > /usr/src/recover.sql
## in case of multiple binlog like so:
## mariadb-binlog -vv --stop-position=857 mysqld-bin.000005 mysqld-bin.000006 > /usr/src/recover.sql
```

```
## Step 1: Apply full backup
cd /usr/src/
mariadb < all-databases.sql
```

```
-- Step 2: Testing in client
-- im mysql-client durch eingeben des Befehls 'mysql'
-- not more than 200
use sakila; select * from actor;
```

```
## Step 3: now apply recover.sql
## auf der Kommandozeile
mariadb < recover.sql
```

```
-- Step 4: now check
-- im mysql client
-- now it should have all actors before deletion
use sakila; select * from actor;
```

Backup Single Database, Structure and only data

Dump database (data and structure) of sakila and reuse for new database sakilaneu

- Why ? Developers need a test database

```
mysqldump --events --routines sakila > /usr/src/all-sakila.sql
## Datenbank erstellen
mysql -e "create schema sakilaneu;"
mysql sakilaneu < /usr/src/all-sakila.sql
```

Only dump structure of database sakila

```
mysqldump --events --routines --no-data sakila > /usr/src/structure-sakila.sql
```

Only data / no create of database sakila and table actor

```
mysqldump --no-create-info sakila actor > /usr/src/sakila-actor-data.sql
```

Flashback

- Redoes insert/update/delete entries from binlog (binlog_format = 'ROW')

Referenz:

- <https://mariadb.com/kb/en/flashback/>

mysqldump-vs-mariabackup

Feature	mariabackup	mysqldump
Geschwindigkeit beim Zurücksicherung	gute Performance	langsam bei grossen Datenmengen - aus logischen SQL-Statements müssen physische Daten werden
Einzelne Datenbanken, einzelne Tabellen sichern	möglich aber nicht straightforward (besser nicht)	ja, problemlos
Einzelne Tabellen und Datenbanken zurücksichern	ich kann nur das zurücksichern, was ich gesichert habe, als ganze.	ja, kann ich machen, gute Struktur der Dumps vorausgesetzt
Wann, was?	Bei größeren Datenmenge mehrere 100 GB und wen zurückspielen mit mysqldump langsam wäre	so lange, bis das Rückspielen nicht zu lange dauert für Aufsetzen neuer Server in neuer Version
Wann nehme ich es nicht ?	Update (neuen Server aufsetzen in neuer Version und Backup einspielen) (mariabackup darf nur für die gleiche Version und das gleiche Betriebssystem verwendet werden (Linux oder Windows))	wenn Zurückspielen zu langsam wäre
Wie könnte ich trotzdem mariabackup für Migration zu neuem Debian / Ubuntu verwenden	In-Place Upgrade: 1. Neuen Server mit neuer Linux-Version aufsetzen 2. Gleiche MariaDb-Version verwenden. 3. Daten aus mariabackup einspieln 4. Upgrade auf neuen Version (die alte Version deinstallieren, Daten belassen, danach neue Version installieren und starten.	

mariabackup

Installation

dnf (using mariadb from mariadb.org - repo)

```
dnf install MariaDB-backup
```

Installation von Distri (Centos/Rocky/RHEL)

```
## Rocky 8
dnf install mariadb-backup
```

Installation deb (Ubuntu/Debian)

```
apt update
apt search mariadb-backup
apt install -y mariadb-backup
```

Walkthrough (Ubuntu/Debian)

Schritt 1: Grundkonfiguration (vor MariaDB 11.4)

```
## user eintrag in /root/.my.cnf
[mariabackup]
user=root
## pass is not needed here, because we have the user root with unix_socket - auth
## or generic
## /etc/mysql/mariadb.conf.d/mariabackup.cnf
[mariabackup]
user=root
```

Schritt 2: Backup erstellen

```
mkdir /backups
## target-dir needs to be empty or not present
mariabackup --target-dir=/backups/2023091901 --backup
```

Schritt 3: Prepare durchführen

```
## apply ib_logfile0 to tablespaces
## after that ib_logfile0 -> 0 bytes
mariabackup --target-dir=/backups/2023091901 --prepare
```

Schritt 4: recover

```
systemctl stop mariadb
mv /var/lib/mysql /var/lib/mysql.bkup
mariabackup --target-dir=/backups/2023091901 --copy-back
chown -R mysql:mysql /var/lib/mysql
## only relevant if socket is created in data - folder
##
## chmod -R 755 /var/lib/mysql # otherwise socket for unprivileged user does not work
systemctl start mariadb
systemctl status mariadb
```

Walkthrough (Redhat/Centos/Rocky Linux)

Schritt 1: Grundkonfiguration (vor 11.4)

```
## user eintrag in /root/.my.cnf
[mariabackup]
user=root
## pass is not needed here, because we have the user root with unix_socket - auth
## or generic
## /etc/my.cnf.d/mariabackup.cnf
[mariabackup]
user=root
```

Schritt 2: Backup erstellen

```
mkdir /backups
## target-dir needs to be empty or not present
mariabackup --target-dir=/backups/2023091901 --backup
```

Schritt 3: Prepare durchführen

```
## apply ib_logfile0 to tablespaces
## after that ib_logfile0 -> 0 bytes
mariabackup --target-dir=/backups/2023091901 --prepare
```

Schritt 4: Recover

```
systemctl stop mariadb
mv /var/lib/mysql /var/lib/mysql.bkup
mariabackup --target-dir=/backups/2023091901 --copy-back
chown -R mysql:mysql /var/lib/mysql
chmod -R 755 /var/lib/mysql # otherwise socket for unprivileged user does not work
## Does not work !!! Because of selinux // does not start
## ls -laZ /var/lib
systemctl start mariadb

### important for selinux if it does not start
restorecon -vr /var/lib/mysql
systemctl start mariadb

### Cleanup if everything works
rm -fR /var/lib/mysql/mysql.bkup
```

Ref.

<https://mariadb.com/kb/en/full-backup-and-restore-with-mariabackup/>

mariabackup mit windows

Prerequisites

- On Windows Mariabackup is already installed by default installer

Walkthrough (Windows)

Schritt 2: Backup erstellen

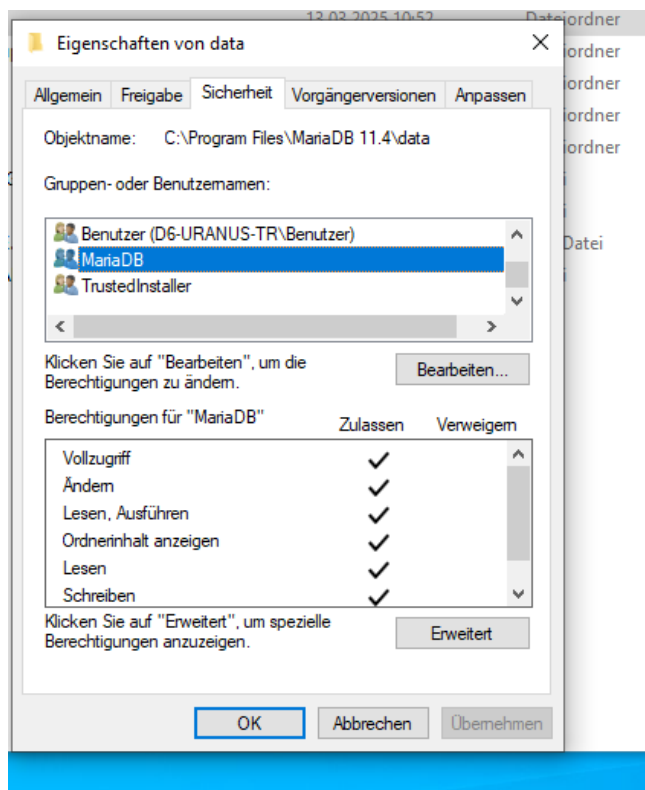
```
## Auf dem Desktop bspw. backups - Ordner anlegen
mariabackup -uroot -p<password-hier-rein> --target-dir=C:\Users\Administrator\Desktop\backups\2025031301 --backup
```

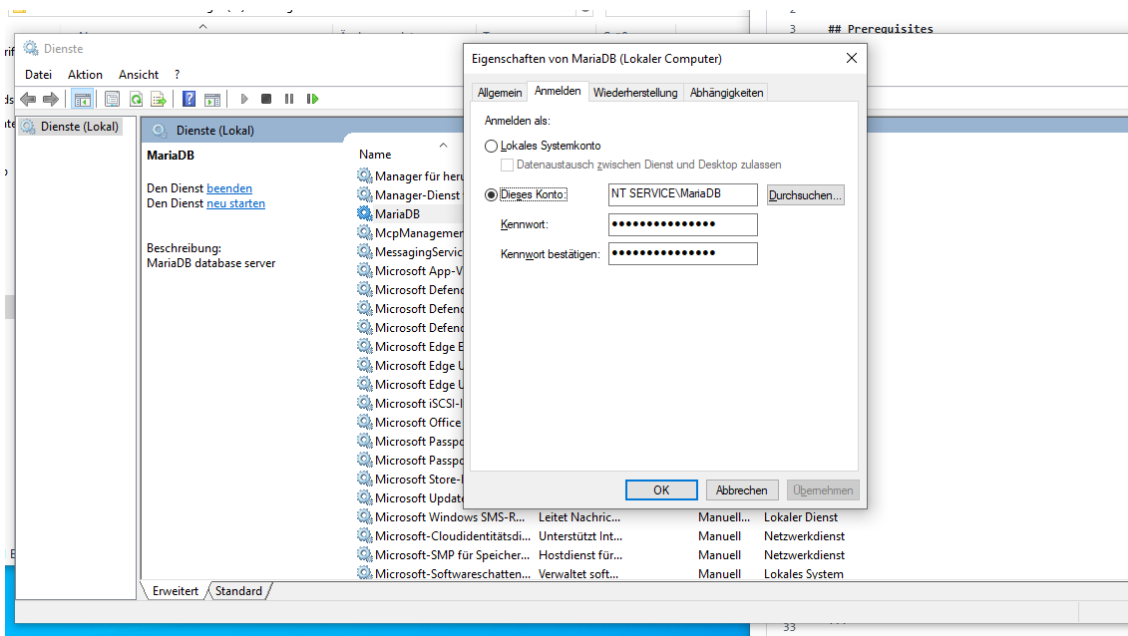
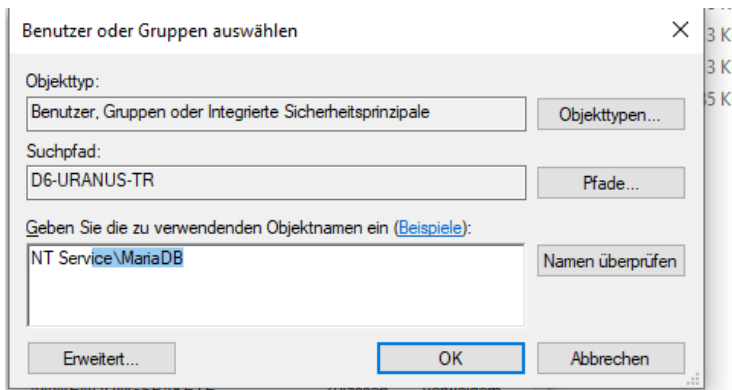
Schritt 3: Prepare durchführen

```
## apply ib_logfile0 to tablespaces
## after that ib_logfile0 -> 0 bytes
mariabackup -uroot -p<password-hier-rein> --target-dir=C:\Users\Administrator\Desktop\backups\2025031301 --prepare
```

Schritt 4: Recover

```
## 1. mariadb-dienst stoppen
## 2. Umbenennen des Ordner data -> data.bkup
mariabackup -uroot -p<password-hier-rein> --target-dir=C:\Users\Administrator\Desktop\backups\2025031301 --copy-back
## 3. my.ini aus data.bkup nach data kopieren
## RECHTE: 4. Benutzer NT Service\MariaDB mit Vollzugriff (!!) unter Sicherheit hinterlegen für data - Verzeichnis
(gleicher Benutzer wie beim Dienst -> MariaDB --> kann auch von dort kopiert werden)
```





4. Dienst starten

mariabackup ohne Passwort - Eingabe verwenden

Variante 1: Umgebungsvariable verwenden

1. **Umgebungsvariable setzen:** Du kannst die Umgebungsvariable `MYSQL_PWD` verwenden, um das Passwort für die MariaDB-Verbindung anzugeben.
 - Öffne die Eingabeaufforderung (CMD) und setze die Umgebungsvariable temporär:

```
set MYSQL_PWD=deinpasswort
```

2. **mariadb-backup ausführen:** Nachdem du die Umgebungsvariable gesetzt hast, kannst du `mariadb-backup` ausführen, ohne das Passwort anzugeben:

```
mariadb-backup --user=deinbenutzername --host=deinhost --backup-dir=C:\Pfad\zum\Sicherungsordner
```

Variante 2: Verwendung der `--defaults-extra-file` Option

Falls du die globale Konfigurationsdatei nicht bearbeiten möchtest, kannst du eine benutzerdefinierte Konfigurationsdatei erstellen und die Option `--defaults-extra-file` verwenden.

1. **Erstellen einer benutzerdefinierten Konfigurationsdatei (z.B. backup.cnf):** Erstelle eine neue `.cnf` oder `.ini` Datei (z.B. `C:\Pfad\zur\backup.cnf`) mit folgendem Inhalt:

```
[client]
user=deinbenutzername
password=deinpassword
```

2. **mariadb-backup mit --defaults-extra-file ausführen:** Verwende die Option `--defaults-extra-file`, um auf deine benutzerdefinierte Konfigurationsdatei zu verweisen:

```
mariadb-backup --defaults-extra-file=C:\Pfad\zur\backup.cnf --backup-dir=C:\Pfad\zum\Sicherungsordner
```

Ref.

<https://mariadb.com/kb/en/full-backup-and-restore-with-mariabackup/>

incrementelles backup mit mariadb

- <https://mariadb.com/kb/en/incremental-backup-and-restore-with-mariabackup/>

Delete binary logs

What ways do we have to delete binary logs (purge)

- Global Variable or in config: `expire_logs_days`
- Global Variable or in config:
 - `binlog_expire_logs_seconds`
 - https://mariadb.com/kb/en/replication-and-binary-log-system-variables/#binlog_expire_logs_seconds
- SQL-Kommando: `PURGE BINARY LOGS TO`
- Automatically with `mysql -> param: --delete-master-logs`

Best practice

- Do it when performing backup, either with `--delete-master-logs` (PIT-Recovery), or in backup script with `PURGE BINARY LOGS`

Logging

General Log

Exercise Version 1: Enable in config

```
## set in configuration
## /etc/my.cnf.d/server.cnf
## under mysqld
general-log
```

```
systemctl restart mariadb
mysql
```

```
-- in mysql
select @@general_log;
show processlist;
use sakila;
select * from actor;
exit
```

```
## depending on your server-name
cd /var/lib/mysql
cat server1.log
```

Exercise Version 2: Enable/Disable general_log during runtime

Step 1:

```
## if general_log is activated disable like so
mysql
set global general_log = 1
```

Step 2: fill with data

```
-- in mysql
select @@general_log;
show processlist;
use sakila;
select * from actor;
exit
```

Step 3: See what's in general_log

```
## depending on your server-name
cd /var/lib/mysql
## servename + log
cat server1.log
```

Binary Logs - Activating in Ubuntu

How to activate ?

```
nano /etc/mysql/mariadb.conf.d/zz_config.cnf
```

```
[mysqld]
log-bin
```

```
systemctl restart mariadb
cd /var/lib/mysql
## is binary log there ? mariadb-
ls -la *mysqld-bin*
mariadb
```

```
-- is log-bin activated
show variables like '%log%bin';
select @@log_bin;
exit
```

Exercise

```
cd /var/lib/mysql
## Das letzte nehmen, wenn mehrere da sind
mariadb-binlog -vv mariadb-bin.000001
mariadb
```

```
create schema kurs;
exit;
```

```
## Das letzte nehmen, wenn mehrere da sind
mysqlbinlog -vv mariadb-bin.000001
```

Search in binlog with Unix-Tools

```
cd /var/lib/mysql
mysqlbinlog mysql-bin.000001 | grep -B 10 -A 10 kurs
```

Binary Logs - Basics

Binlog - Wann ?

- PIT (Point-in-Time) - Recovery
- Master/Slave - Replication
- Galera Cluster (MariaDB Cluster)

Binlog aktivieren (Centos)

```
## /etc/my.cnf.d/server.cnf
[mysqld]
log-bin

## Server neu starten
systemctl restart mariadb
```

Alte Logs automatisch löschen

- https://mariadb.com/kb/en/replication-and-binary-log-system-variables/#expire_logs_days

Rowbasiertes Logging aktivieren

```
## /etc/my.cnf.d/server.cnf
[mysqld]
log-bin
## Server neu starten
systemctl restart mariadb
```

binlog auslesen

```
cd /var/lib/mysql
## Zeigt auch mit Kommentar die SQL-Statements an die bei ROW-basierten binlog ausgeführt werden
mysqlbinlog -vv mysqld-bin.000001
```

Wie finde ich raus, welches binlog aktiv ist ?

```
## mysql -client starten
mysql> show master status;
```

Error logging (Output of start / stop of mariadb)

Was ist das ?

- Zeigt alle Ausgaben beim Starten und Stoppen des MariaDB - Dienstes
- Nicht nur Fehler, sondern alles

Error Log aktivieren in Datei -> Walkthrough (using log in filesystem)

```
## Ubuntu / Debian
nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

```
[mysqld]
log-error=/var/log/mysql/mariadb-error.log
```

```
systemctl restart mariadb
```

Problem: Keine Ausgabe über journal

- journalctl -u mariadb -> keine Ausgabe bzw. keine Einträge

Lösung

```
## Prüfen ob systemd-journal - dienst läuft bzw. einfach noch mal neu starten
systemctl restart systemd-journald.services
```

Security

Table encryption

Step 1: Set up keys

```
mkdir -p /etc/mysql/encryption;
echo "1;"$(openssl rand -hex 32) > /etc/mysql/encryption/keyfile;

openssl rand -hex 128 > /etc/mysql/encryption/keyfile.key;
openssl enc -aes-256-cbc -md sha1 -pass file:/etc/mysql/encryption/keyfile.key -in /etc/mysql/encryption/keyfile -
```

```
out /etc/mysql/encryption/keyfile.enc;

rm -f /etc/mysql/encryption/keyfile;

chown -R mysql:mysql /etc/mysql;
chmod -R 500 /etc/mysql;
```

Step 2: Verify data before encryption

```
cd /var/lib/mysql/mysql
strings gtid_slave_pos.ibd
```

Step 3: Setup configuration

```
## debian / ubuntu
nano /etc/mysql/mariadb.conf.d/z_encryption.cnf
```

```
## centos / rocky / redhat
nano /etc/my.cnf.d/z_encryption.cnf
```

```
[mysqld]
plugin_load_add = file_key_management
file_key_management_filename = /etc/mysql/encryption/keyfile.enc
file_key_management_filekey = FILE:/etc/mysql/encryption/keyfile.key
file_key_management_encryption_algorithm = AES_CTR

innodb_encrypt_tables = FORCE
innodb_encrypt_log = ON
innodb_encrypt_temporary_tables = ON

encrypt_tmp_disk_tables = ON
encrypt_tmp_files = ON
encrypt_binlog = ON
aria_encrypt_tables = ON

innodb_encryption_threads = 4
innodb_encryption_rotation_iops = 2000
```

Step 4: Restart server

```
systemctl restart mariadb
```

Step 5: Verify encryption

```
cd /var/lib/mysql/mysql
strings gtid_slave_pos;
mysql;
```

```
use information_schema;
select * from innodb_tablespace_encryption;
SELECT CASE WHEN INSTR(NAME, '/') = 0
            THEN '01-SYSTEM TABLESPACES'
            ELSE CONCAT('02-', SUBSTR(NAME, 1, INSTR(NAME, '/')-1)) END
        AS "Schema Name",
        SUM(CASE WHEN ENCRYPTION_SCHEME > 0 THEN 1 ELSE 0 END) "Tables Encrypted",
        SUM(CASE WHEN ENCRYPTION_SCHEME = 0 THEN 1 ELSE 0 END) "Tables Not Encrypted"
FROM information_schema.INNODB_TABLESPACES_ENCRYPTION
GROUP BY CASE WHEN INSTR(NAME, '/') = 0
            THEN '01-SYSTEM TABLESPACES'
            ELSE CONCAT('02-', SUBSTR(NAME, 1, INSTR(NAME, '/')-1)) END
ORDER BY 1;
```

Step 6: disable encryption runtime

```
SET GLOBAL innodb_encrypt_tables = OFF;
```

```
## Create a user that is not allowed to do so .... no set global
create user noroot@'localhost' identified by 'password';
grant all on *.* to noroot@'localhost';
revoke super on *.* from noroot@'localhost';
```

working with mysqlbinlog and encryption

```
mysqlbinlog -vv --read-from-remote-server --socket /run/mysqld/mysqld.sock mysqld-bin.000003 | less
```

Ref:

- <https://mariadb.com/de/resources/blog/mariadb-encryption-tde-using-mariadbs-file-key-management-encryption-plugin/>

Performance

Slow Query Log

Walkthrough (MariaDB from 10.11)

```
## Step 1
## /etc/my.cnf.d/server.cnf
## or: debian /etc/mysql/mariadb.conf.d/50-server.cnf
[mysqld]
slow-query-log
```

```
systemctl restart mariadb
```

```
mariadb
```

```
-- kleinst mögliche Zeit 0.000001 Sekunden
SET GLOBAL log_slow_query_time=0.000001;
-- in der session setzen, damit das sofort funktioniert
SET log_slow_query_time=0.000001;
-- Achtung, steht nach nächstem Neustart wieder auf 10 Sekunden (Default)
```

```
-- prüfen
select @@GLOBAL.log_slow_query_time;
select @@log_slow_query_time;
```

Walkthrough (before 10.11)

```
## Step 1
## /etc/my.cnf.d/mariadb-server.cnf
## or: debian /etc/mysql/mariadb.conf.d/50-server.cnf
[mysqld]
slow-query-log
```

```
systemctl restart mariadb
```

```
## ODER
## Alternative: Step 1a
mysql>SET GLOBAL slow_query_log = 1
mysql>SET slow_query_log = 1
```

```
## Step 2:
## Zeit festlegen, ab wann eine query langsam ist
## global für den Server (greift erst bei der nächsten Session)
mysql>SET GLOBAL long_query_time = 0.000001;
## und für die aktuelle Session
mysql>SET long_query_time = 0.000001
```

```
## Step 3
## run some time / data
## and look into your slow-query-log
/var/lib/mysql/hostname-slow.log
```

Exercise (mariadb 10.6 from mariadb.org)

```
## Step 1
## /etc/my.cnf.d/server.cnf
[mysqld]
slow-query-log
```

```
## Step 2: restart server
systemctl restart mariadb
mysql
```

```
-- Step 3: set long_query_time (global and in session)
select @@slow_query_log;

-- set and show global
set global long_query_time = 0.000001;
select @@global.long_query_time;
show global variables like '%long%';

-- (Optional) set and show session (for this session)
set long_query_time = 0.000001;
select @@long_query_time;
show variables like '%long%';
```

```
## Step 4: Import data
cd /usr/src/sakila-db
mysql < sakila-schema.sql
mysql < sakila-data.sql
```

```
## Step 5: what did we log
cd /var/lib/mysql
ls -la server1-slow.log
less server1-slow.log
```

Geschwätzigkeit (Verbosity) erhöhen

Before 10.6.16, 10.11.6

```
SET GLOBAL log_slow_verbosity='query_plan,explain'
```

Best option from 10.6.16 and 10.11.6

```
-- Enable all possible options
SET GLOBAL log_slow_verbosity='full'
```

Queries die keine Indizes verwenden

```
SET GLOBAL log_queries_not_using_indexes=ON;
```

Reference

- <https://mariadb.com/kb/en/slow-query-log-overview/>

Percona-toolkit-Installation - Rocky/Redhat und Ubuntu

Walkthrough (Centos / Redhat)

```
## Howto
## https://www.percona.com/doc/percona-toolkit/LATEST/installation.html

## Step 1: repo installieren mit rpm -paket
```

```
dnf install -y https://repo.percona.com/yum/percona-release-latest.noarch.rpm
percona-release enable tools release
dnf install -y percona-toolkit
```

pt-<tab><tab>

Debian / Ubuntu

```
curl -O https://repo.percona.com/apt/percona-release_latest.generic_all.deb
sudo apt install gnupg2 lsb-release ./percona-release_latest.generic_all.deb
apt update
apt install percona-toolkit
```

pt-query-digest under Windows

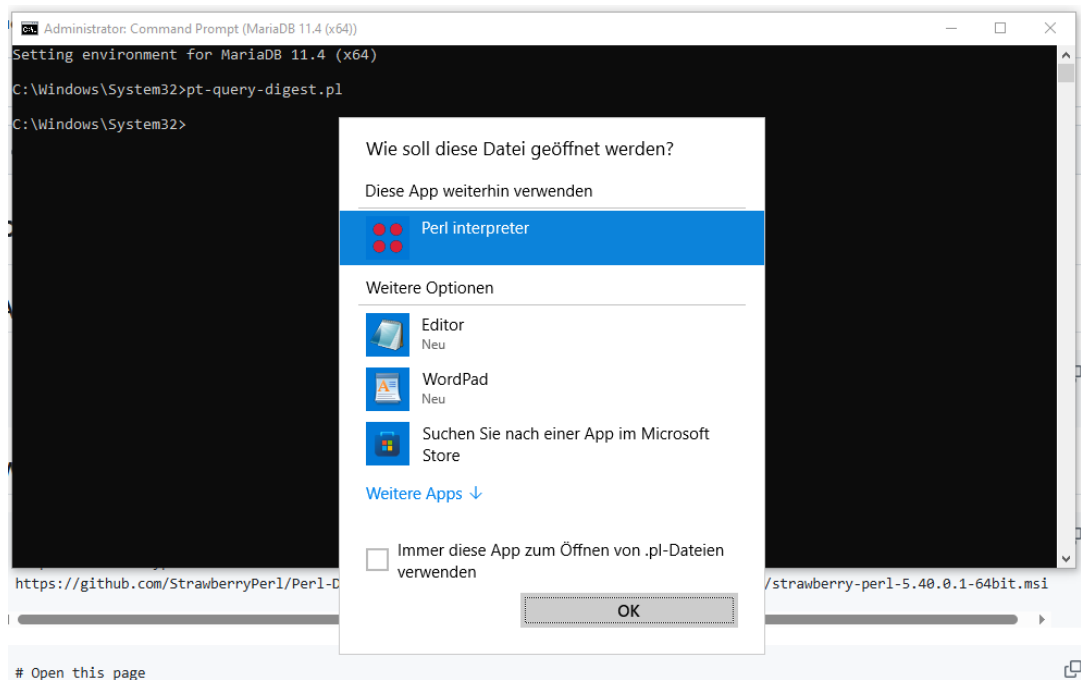
Attention about download

url is wrong in Reference document, us:
<https://www.percona.com/get/pt-query-digest>

Walkthrough

```
## Download and install msi-package - newest version
https://strawberryperl.com/
https://github.com/StrawberryPerl/Perl-Dist-Strawberry/releases/download/SP_54001_64bit_UCRT/strawberry-perl-5.40.0.1-64bit.msi
```

```
## Open this page
https://www.percona.com/get/pt-query-digest
## Save as -> txt filetype -> pt-query-digest.pl in the mariadb/bin folder
```



Open this page

```
## Please confirm with o.k.
```

```
Installed strawberry perl on my windows computer
Saved percona.com/get/pt-query-digest to a .pl file on my laptop
In the run part I typed cmd
In the new command window, I typed pt-query-digest.pl slow.log > digest.txt
```


Reference

- <http://www.jonathanlevin.co.uk/2012/01/query-digest-on-windows.html>

pt-query-digest exercise (Hitliste von slow-query-log erstellen)

```
dnf install -y https://repo.percona.com/yum/percona-release-latest.noarch.rpm && dnf install -y percona-toolkit
cd /var/lib/mysql
pt-query-digest server1-slow.log > /usr/src/report.txt
```

Umgang mit grossen Datenbeständen

Mariabackup vs. mysqldump

- Bei großen Daten ist mariabackup zu empfehlen, da das rücksichern wesentlich schneller ist

Änderung von Struktur

- Änderung können sehr teuer sein. (Originaltabelle wird gesperrt)
- Alternative Lösung: pt-online-schema-change
 - <https://docs.percona.com/percona-toolkit/pt-online-schema-change.html>

Optimal use of indexes

Describe and indexes

Walkthrough

Step 1:

```
## Database and Table with primary key
create database descindex;
use descindex;
create table people (id int unsigned auto_increment, first_name varchar(25), last_name varchar(25), primary key
(id), passcode mediumint unsigned);
## add an index
## This will always !! translate into an alter statement.
create index idx_last_name_first_name on people (last_name,first_name)
##
create unique index idx_passcode on people (passcode)

desc people;
+-----+-----+-----+-----+-----+-----+
| Field      | Type                | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(10) unsigned    | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(25)         | YES  |     | NULL    |                |
| last_name  | varchar(25)         | YES  |     | NULL    |                |
| passcode   | mediumint(8) unsigned | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Step 2:

```
## Add simple combined index on first_name, last_name
create index idx_first_name_last_name on people (first_name, last_name);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
desc people;

-- show the column where the combined index starts (MUL = Multi)

+-----+-----+-----+-----+-----+-----+
| Field      | Type                | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(10) unsigned    | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(25)         | YES  | MUL | NULL    |                |
| last_name  | varchar(25)         | YES  |     | NULL    |                |
| passcode   | mediumint(8) unsigned | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
```

```
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Step 3:

```
## Add a unique index on passcode
create index idx_passcode on people (passcode)
mysql> desc people;

-- Line with UNI shows this indexes.
+-----+-----+-----+-----+-----+-----+
| Field      | Type                | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(10) unsigned    | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(25)         | YES  | MUL | NULL    |                |
| last_name  | varchar(25)         | YES  |     | NULL    |                |
| passcode   | mediumint(8) unsigned | YES  | UNI | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Step 4:

```
## Get to know all your indexes on a table
show indexes for people
mysql> show index from people;
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name          | Seq_in_index | Column_name | Collation | Cardinality | Sub_part |
| Packed | Null | Index_type | Comment | Index_comment |
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
| people | 0 | PRIMARY |          |          |          | 1 | id | A | 0 | NULL |
| NULL |   | BTREE |          |          |          |   |   |   |   |   |
| people | 0 | idx_passcode |          |          |          | 1 | passcode | A | 0 | NULL |
| NULL | YES | BTREE |          |          |          |   |   |   |   |   |
| people | 1 | idx_first_name_last_name |          |          |          | 1 | first_name | A | 0 | NULL |
| NULL | YES | BTREE |          |          |          |   |   |   |   |   |
| people | 1 | idx_first_name_last_name |          |          |          | 2 | last_name | A | 0 | NULL |
| NULL | YES | BTREE |          |          |          |   |   |   |   |   |
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Find out indexes

Show index from table

```
create database showindex;
use showindex;
CREATE TABLE `people` (
  `id` int(10) unsigned NOT NULL AUTO_INCREMENT,
  `first_name` varchar(25) DEFAULT NULL,
  `last_name` varchar(25) DEFAULT NULL,
  `passcode` mediumint(8) unsigned DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `idx_passcode` (`passcode`),
  KEY `idx_first_name_last_name` (`first_name`,`last_name`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1
show index from people
```

Show create table

```
show create table people
```

show index from

```
show index from contributions
```

Index and Functions

No function can be used on an index:

```
explain select * from actor where substring(last_name,1,1) = 'A';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | actor | NULL | ALL | NULL | NULL | NULL | NULL | 200 | 100.00 | Using where |
```

Workaround with generated columns

```
## 1. Create Virtual Column with upper
MariaDB [sakila]> alter table actor add last_name_upper varchar(45) AS (upper(last_name)) VIRTUAL;
Query OK, 0 rows affected (0.006 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
MariaDB [sakila]> create index idx_upper on actor (last_name_upper);
Query OK, 0 rows affected (0.008 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
MariaDB [sakila]> explain select * from actor where last_name_upper like 'A%';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | possible_keys | key | key_len | ref | rows | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | actor | range | idx_upper | idx_upper | 183 | NULL | 7 | Using where |
1 row in set (0.001 sec)
```

Now we try to search the very same

```
explain select * from actor where last_name_upper like 'A%';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | actor | NULL | range | idx_last_name_upper | idx_last_name_upper | 183 | NULL | 7 | 100.00 | Using where |
1 row in set, 1 warning (0.00 sec)
```

Reference

- <https://mariadb.com/kb/en/generated-columns/>
- <https://mariadb.com/kb/en/slow-query-log-overview/>

Index and Likes

1. like 'Will%' - Index works

```
explain select last_name from donors where last_name like 'Will%';
```

2. like '%iams' - Index does not work

```
-- because like starts with a wildcard
explain select last_name from donors where last_name like '%iams';
```

3. How to fix 3, if you are using this often ?

```
## Walkthrough
## Step 1: modify table
alter table donors add last_name_reversed varchar(70) GENERATED ALWAYS AS (reverse(last_name));
create index idx_last_name_reversed on donors (last_name_reversed);

## besser - Variante 2 - untested
alter table donors add last_name_reversed varchar(70) GENERATED ALWAYS AS (reverse(last_name)), add index
idx_last_name_reversed on donors (last_name_reversed);

## Step 2: update table - this take a while
update donors set last_name_reversed = reversed(last_name)
## Step 3: work with it
select last_name,last_name_reversed from donor where last_name_reversed like reverse('%iams');
```

```
## Version 2 with pt-online-schema-change
```

Find out cardinality without index

Find out cardinality without creating index

```
select count(distinct donor_id) from contributions;
```

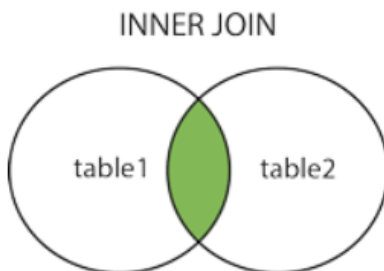
```
select count(distinct(vendor_city)) from contributions;
+-----+
| count(distinct(vendor_city)) |
+-----+
|                1772         |
+-----+
1 row in set (4.97 sec)
```

Joins

Overview Joins

- * combines rows from two or more tables
- * based on a related column between them.

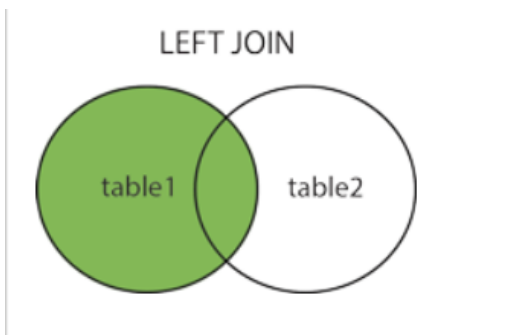
MariaDB (Inner) Join



MariaDB (Inner) Join (explained)

- * Inner Join and Join are the same
- * Returns records that have matching values in both tables
- * Inner Join, Cross Join and Join
 - * are the same in MySQL

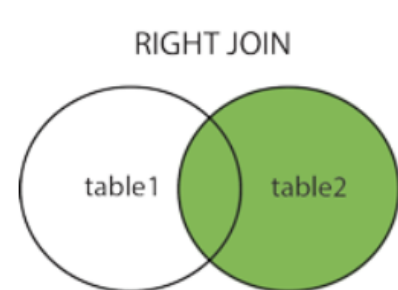
MariaDB Left Join



MySQL Left (outer) Join (explained) ====

- Return all records from the left table
- *AND* the matched records from the right table
- The result is NULL on the right side
 - if there are no matched columns on the right
- Left Join and Left Outer Join are the same

MySQL Right Join



MySQL Right Join (explained)

- Return all records from the right table
 - *AND* the matched records from the left table
- Right Join and Right Outer Join are the same

MySQL Straight Join

- MySQL (inner) Join and Straight Join are the same
- **Difference:**
 - The left column is always read first
- **Downside:**
 - Bad optimization through mysql (query optimizer)
- **Recommendation:**
 - Avoid straight join if possible
 - use join instead

Type of Joins

- [inner] join
 - **inner join** and **join** are the same
- left [outer] join
- right [outer] join
- full [outer] join
- straight join < equals > join
- cross join = join (in mysql)
- natural join <= equals => join (but syntax is different)

In Detail: [INNER] JOIN

- Return rows when there
 - is a match in both tables
- Example

```
SELECT actor.first_name, actor.last_name, film.title
FROM film_actor
INNER JOIN actor ON film_actor.actor_id = actor.actor_id
INNER JOIN film ON film_actor.film_id = film.film_id;
```

In Detail: Joining without JOIN - Keyword

- Explanation: Will have the same query execution plan as [INNER] JOIN

```
SELECT actor.first_name, actor.last_name, film.title
FROM film_actor,actor,film
where film_actor.actor_id = actor.actor_id
and film_actor.film_id = film.film_id;
```

In Detail: Left Join

- Return all rows from the left side
 - even if there is not result on the right side
- Example

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    a.actor_id,
    a.first_name,
    a.last_name
FROM customer c
LEFT JOIN actor a
ON c.last_name = a.last_name
ORDER BY c.last_name;
```

In Detail: Right Join

- Return all rows from the right side
 - even if there are no results on the left side
- Example

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    a.actor_id,
    a.first_name,
    a.last_name
FROM customer c
RIGHT JOIN actor a
ON c.last_name = a.last_name
ORDER BY a.last_name;
```

In Detail: Having

- Simple: WHERE for GroupBy (because where does not work here)
- Example

```
SELECT last_name, COUNT(*)
FROM sakila.actor
GROUP BY last_name
HAVING count(last_name) > 2
```

Internal (type of joins) - NLJ

- NLJ - (Nested Loop Join)

```
for each row in t1 matching range {
  for each row in t2 matching reference key {
    for each row in t3 {
      if row satisfies join conditions, send to client
    }
  }
}
```

Internal (type of joins) - BNL

- BNL - (Block Nested Loop)
 - in explain: -> using join buffer
 - columns of interest to a join are stored in join buffer
 - --> not whole rows.
 - join_buffer_size system variable
 - -> determines the size of each join buffer used to process a query.
- <https://dev.mysql.com/doc/refman/5.7/en/nested-loop-joins.html>

BNL - Who can I see, if it is used ?

- Can be seen in explain

```
1 | PRIMARY | SE | ALL | PRIMARY | NULL | NULL | NULL | 5 | Using where; Using join buffer (Block Nested Loop)
```

```
* explain select * from t1, t2 where t1.col < 10 and t2.col < 'bar';
```

Monitoring

What to monitor?

What to monitor (stand-alone an cluster and replication as a basis)

System

- Last auf dem System (top)
- Festplatte (z.B. 85% voll ?) df /var/lib/mysql
- Swap (Wenn geswappt wird ist Hopfen und Malz verloren)

Erreichbarkeit

- Server per ping erreichen (mysqladmin ping -h ziel-ip)
- Einlogbar ? (mysqladmin ping -h ziel-ip -u control_user)

Platte aka IO-Subsystem (iostats)

- <http://schulung.t3isp.de/documents/pdfs/mysql/mysql-performance.pdf>

--	--	--
Read/Write requests	IOPS (Input/Output operations per second)	--
Average IO wait	Time that queue operations have to wait for disk access	--
Average Read/Write time	Time it takes to finish disk access operations (latency)	--
Read/Write bandwidth	Data transfer from and towards your disk	--

General mysql metrics

```
mysql -E -e "select variable_value from information_schema.session_status where variable_name = 'uptime'";

# max connections
MariaDB [(none)]> show status like 'max_used_connections';
+-----+
| Variable_name | Value |
+-----+
| Max_used_connections | 1 |
+-----+
```

```
1 row in set (0.001 sec)

MariaDB [(none)]> show variables like 'max_connections';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| max_connections | 151 |
+-----+-----+
1 row in set (0.001 sec)

mysqladmin status
## you will find uptime here in seconds
```

Metric	Comments	Suggested Alert
Uptime	Seconds since the server was started. We can use this to detect respawns.	When uptime is < 180. (seconds)
Threads_connected	Number of clients currently connected. If none or too high, something is wrong.	None
Max_used_connections	Max number of connections at a time since server started. (max_used_connections / max_connections) indicates if you could run out soon of connection slots.	When connections usage is > 85%.
Aborted_connects	Number of failed connection attempts. When growing over a period of time either some credentials are wrong or we are being attacked. show status like 'Aborted_connects'	When aborted connects/min > 3.

InnoDB

Metric	Coments	Suggested Alert
Innodb_row_lock_waits	Number of times InnoDB had to wait before locking a row.	None
Innodb_buffer_pool_wait_free	Number of times InnoDB had to wait for memory pages to be flushed. If too high, innodb_buffer_pool_size is too small for current write load.	None

Query tracking

Metric	Comments	Suggested Alert
Slow_queries	Number of queries that took more than long_query_time seconds to execute. Slow queries generate excessive disk reads, memory and CPU usage. Check slow_query_log to find them. show status like '%slow%';	None
Select_full_join	Number of full joins needed to answer queries. If too high, improve your indexing or database schema.	None
Created_tmp_disk_tables	Number of temporary tables (typically for joins) stored on slow spinning disks, instead of faster RAM.	None
(Full table scans) Handler_read%	Number of times the system reads the first row of a table index. (if 0 a table scan is done - because no key was read). Sequential reads might indicate a faulty index. None	

Track Errors

```
journalctl -u mariadb | grep -i Error
```

Replication

```
show slaves status \G
## These are important values for slaves
Slave_IO_Running: Yes
Slave_SQL_Running: Yes
Seconds_Behind_Master: 0
```

Galera

- see monitoring galera

External Tools

Monitoring with pmm (Percona Management Monitoring)

<https://pmmdemo.percona.com>

[Documentation](#)

Percona Management and Monitoring

- <https://docs.percona.com/percona-monitoring-and-management/setting-up/client/index.html#add-services>
- <https://pmmdemo.percona.com/>

Monitoring mit IBM

- <https://www.ibm.com/support/pages/tivoli-composite-application-manager-applications-721-fp2-monitoring-agent-mysql-server-721-fp1-721-tiv-itmmysql-fp0001>

Replication

Slave einrichten - gtid (mit mariabackup)

Step 1: Master: set server-id 1 and log-bin

```
## Variante Centos/Rocky
cd /etc/my.cnf.d
nano z_settings.cnf
```

```
Variante Debian/Ubuntu
cd /etc/mysql/mariadb.conf.d
nano z_settings.cnf
```

```
[mysqld]
server-id = 1
log-bin
```

```
systemctl restart mariadb

### you should add data, otherwise no gtid will get created if you enable the binlog only from now on
mysql -e "create schema foo;"
```

Step 2a: Installation on ubuntu/debian (master)

```
## Debian / Ubuntu
apt update
apt install mariadb-backup
```

```
dnf install -y MariaDB-server MariaDB-client MariaDB-backup
```

```
## check if available
mariabackup --version
```

Step 2b: Installation on centos/rocky/rhel (master)

```
## When using mariadb from repo rhel/rocky
dnf install -y mariadb-backup

## When using mariadb from foundation mariadb.org
## check if available
mariabackup --version
```

Step 3: Setup mariabackup (before 11.4)

```
## prepare for mariabackup if you use it with root and with unix_socket
/root/.my.cnf
[mariabackup]
user=root
```

Step 4: mariabackup on master

```
mkdir -p /backups
## target-dir needs to be empty or not present
mariabackup --target-dir=/backups/2023092001 --backup
## apply ib_logfile0 to tablespaces
## after that ib_logfile0 -> 0 bytes
mariabackup --target-dir=/backups/2023092001 --prepare
```

Step 4.5: Setup slave-server

```
## Debian / Ubuntu
## start server
apt update
apt install mariadb-server
```

```
## Redhat / Rocky
dnf install -y MariaDB-server MariaDB-client MariaDB-backup
```

```
## Ubuntu
cd /etc/mysql/mariadb.conf.d/
```

```
## Redhat / Rocky
cd /etc/my.cnf.d/
```

```
## use the same config-settings as on master
## Why ? Get the same performance
nano z_settings.cnf
```

```
## Example

[mysqld]

server-id=2 # slave please do not use server-id=1 -> must be unique
log-bin

bind-address=0.0.0.0

innodb-buffer-pool-size = 2500MB
innodb-log-file-size=320M
```

```
systemctl restart mariadb
```

Step 5: Transfer to new slave (from master)

```
## root@master:
rsync -e ssh -avP /backups/2023092001 kurs@192.168.56.104:/home/kurs
```

Step 6: Setup replication user on master

```
## as root@master
##mysql>
CREATE USER repl@'192.168.56.%' IDENTIFIED BY 'password';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'192.168.56.%';
```

Step 7 (Optional): Test repl user (connect) from slave

```
## as root@slave
## you be able to connect to the master
mysql --skip-ssl -urepl -p -h192.168.56.102
## test if grants are o.k.
show grants;
```

Step 8: Set server-id on slave -> 1 + same config as server 1 + log-slave-update

```
cd /etc/my.cnf.d
nano z_settings.cnf
```

```
[mysqld]
server-id          = 2
## activate master bin log, if this slave might be a master later
log-bin
log-slave-updates
```

```
systemctl restart mariadb
```

Step 9: Restore Data on slave

```
## Debian / Ubuntu
## Install mariaback if not present
apt install -y mariadb-backup
```

```
## Rocky / Redhat
dnf install -y MariaDB-backup
```

```
systemctl stop mariadb
mv /var/lib/mysql /var/lib/mysql.bkup
mariabackup --target-dir=/home/kurs/2023092001 --copy-back
chown -R mysql:mysql /var/lib/mysql
chmod -R 755 /var/lib/mysql
## only redhat
## restorecon -vr /var/lib/mysql
systemctl start mariadb
```

Step 10: master.txt for change command

```
## root@slave
## $ cat xtrabackup_binlog_info
cd /home/kurs/2023092001
cat xtrabackup_binlog_info
## mariadb-bin.000096 568 0-1-2
```

```
sudo su -
nano /root/master.txt
```

```
SET GLOBAL gtid_slave_pos = "0-1-2";
## /root/master.txt
## get information from master-databases.sql dump
CHANGE MASTER TO
  MASTER_HOST="192.168.56.102",
  MASTER_PORT=3306,
  MASTER_USER="repl",
  MASTER_PASSWORD="password",
  MASTER_USE_GTID=slave_pos;
```

```
mysql < /root/master.txt
mysql
```

```
-- in mysql
start slave
show slave status \G
-- # Looking for
-- Slave_IO_Running: Yes
-- Slave_SQL_Running: Yes
```

Walkthrough

- <https://mariadb.com/kb/en/setting-up-a-replication-slave-with-mariabackup/>

Slave einrichten - old styke - masterpos

Binary Logs auf master bis slave-log-master-position ändern

```
## Schritt 1: slave abfragen
mysql -u ext -p -h 192.168.56.103 --pager="grep -E 'Master_Log_File:' -e "show slave status \G"
## Beispiel Ausgabe
mariadb-bin.000003

## Schritt 2: logs bis dahin löschen auf master
mysql -e "purge logs to 'mariadb-bin.000003'"
```

Replication / Galera

FAQ

Wie gehe ich vor, wenn der Slave steht ?

- aufgrund eines SQL_Thread - Fehlers
- d.h. SQL Befehl konnte aufgrund eines Fehlers auf dem Slave nicht ausgeführt werden
- typisches Beispiel: Duplicate Key
- ** Neu aufsetzen **
- Alles andere wäre brandgefährliches Gefrickel.

Sobald der Slave nicht mehr ordnungsgemäß arbeitet (d.h. der sql_thread steht wg. z.B. Fehlermeldung (Duplicate key), MUSS (best practice) der Slave neu aufgesetzt werden

Schnellster Weg ist immer mariabackup (non-blocking) wenn ihr ausschliesslich innodb-tabellen einsetzt.

Galera

How does Galera transfer data IST/SST

How it works ?

1. On one node in the cluster, a state change occurs on the database.
2. In the database, the wsrep hooks translate the changes to the write-set.
3. dlopen() makes the wsrep provider functions available to the wsrep hooks.
4. The Galera Replication plugin handles write-set certification and replication to the cluster

SST / IST

State Snaphost Transfer (SST)

- Full Transfer is done
- Methods: mariabackup, mysqldump, rsync
- Some are blocking (mysqldump, rsync), some are not (mariabackup)
 - Means: it can not be written to the donor node in the meantime
- Please do not use !!! xtrabackup or xtrabackup_v2 for mariadb because of encryption

Incremental State Transfer (IST)

- A node only receives the missing write sets.
- Is only done, when
 - The missing write-sets are in the gcache of the donor
 - Otherwise SST is done

Installation and Configuration (Centos/Redhat 8) with mariabackup as sst

Preparation: Install mariadb

```
## On all nodes install mariadb and mariadb-server
dnf install -y mariadb mariadb-server mariadb-server-galera
```

Setting up 1st - node

```
nano /etc/my.cnf.d/z_galera.cnf
```

```
[mysqld]
binlog_format=ROW
default-storage-engine=innodb
innodb_autoinc_lock_mode=2
bind-address=0.0.0.0

## for better performance
innodb_flush_log_at_trx_commit=2

## Galera Provider Configuration
wsrep_on=ON
wsrep_provider=/usr/lib64/galera/libgalera_smm.so

## Galera Cluster Configuration
wsrep_cluster_name="test_cluster-<your shortcut e.g. r1>"
wsrep_cluster_address="gcomm://10.135.0.3,10.135.0.4,10.135.0.5"
wsrep_node_address=10.135.0.3

## Galera Synchronization Configuration
wsrep_sst_method=mariabackup
wsrep_sst_auth = mariabackup:mypassword
```

```
systemctl stop mariadb
galera_new_cluster
mysql
```

```
CREATE USER 'mariabackup'@'localhost' IDENTIFIED BY 'mypassword';
GRANT RELOAD, PROCESS, LOCK TABLES, REPLICATION CLIENT ON *.* TO 'mariabackup'@'localhost';
quit
```

```
mysql
```

```
show status like 'wsrep%';
quit
```

Setup Node 2:

```
dnf install -y mariadb-server mariadb-backup mariadb-server-galera
```

```
## Default datei löschen oder umbenennen
rm /etc/my.cnf.d/galera.cnf
nano /etc/my.cnf.d/z_galera.cnf
```

```
[mysqld]
binlog_format=ROW
default-storage-engine=innodb
innodb_autoinc_lock_mode=2
bind-address=0.0.0.0

## for better performance // Attention: You might loose data on power outage
innodb_flush_log_at_trx_commit=2

## Galera Provider Configuration
wsrep_on=ON
wsrep_provider=/usr/lib64/galera/libgalera_smm.so

## Galera Cluster Configuration
wsrep_cluster_name="test_cluster-<your shortcut e.g. r1>"
wsrep_cluster_address="gcomm://10.135.0.3,10.135.0.4,10.135.0.5"
wsrep_node_address=10.135.0.4
```

```
## Galera Synchronization Configuration
wsrep_sst_method=mariabackup
wsrep_sst_auth = mariabackup:mypassword
```

```
systemctl restart mariadb
```

Setup Node 3:

```
apt update
apt install -y mariadb-server mariadb-backup
```

```
nano /etc/mysql/mariadb.conf.d/z_settings.cnf
```

```
[mysqld]
binlog_format=ROW
default-storage-engine=innodb
innodb_autoinc_lock_mode=2
bind-address=0.0.0.0

## for better performance // Attention: You might loose data on power outage
innodb_flush_log_at_trx_commit=2

## Galera Provider Configuration
wsrep_on=ON
wsrep_provider=/usr/lib/galera/libgalera_smm.so

## Galera Cluster Configuration
wsrep_cluster_name="test_cluster-<your shortcut e.g. r1>"
wsrep_cluster_address="gcomm://10.135.0.3,10.135.0.4,10.135.0.5"
wsrep_node_address=10.135.0.5

## Galera Synchronization Configuration
wsrep_sst_method=mariabackup
wsrep_sst_auth = mariabackup:mypassword
```

```
systemctl restart mariadb
```

```
## Schritt 1: Create config

/etc/my.cnf.d/z_galera.cnf

[mysqld]
binlog_format=ROW
default-storage-engine=innodb
innodb_autoinc_lock_mode=2
bind-address=0.0.0.0
## Set to 1 sec instead of per transaction
## for better performance // Attention: You might loose data on power
innodb_flush_log_at_trx_commit=0
## Galera Provider Configuration
wsrep_on=ON
## centos7 (x86_64)
wsrep_provider=/usr/lib64/galera-4/libgalera_smm.so
## Galera Cluster Configuration
wsrep_cluster_name="test_cluster"
wsrep_cluster_address="gcomm://192.168.56.103,192.168.56.104,192.168.56.105"
wsrep_node_address=192.168.56.103
## Galera Synchronization Configuration
wsrep_sst_method=rsync
```

Stop the server and bootstrap cluster

```
## setup first node in cluster
systemctl stop mariadb
galera_new_cluster # statt systemctl start mariadb
```

Check if cluster is running

```
mysql> show status like 'wsrep%'\G
***** 38. row *****
Variable_name: wsrep_local_state_comment
Value: Synced
***** 56. row *****
Variable_name: wsrep_cluster_size
Value: 1
***** 57. row *****
Variable_name: wsrep_cluster_state_uuid
Value: 562e5455-a40f-11eb-b8c9-1f32a94e106e
***** 58. row *****
Variable_name: wsrep_cluster_status
Value: Primary
***** 59. row *****
Variable_name: wsrep_connected
Value: ON
```

Setup firwealld for galera

```
firewall-cmd --add-port=3306/tcp --permanent
firewall-cmd --add-port=4567/tcp --permanent
firewall-cmd --add-port=4568/tcp --permanent
firewall-cmd --add-port=4444/tcp --permanent
firewall-cmd --reload

firewall-cmd --add-port=3306/tcp --permanent; firewall-cmd --add-port=4567/tcp --permanent; firewall-cmd --add-port=4568/tcp --permanent; firewall-cmd --add-port=4444/tcp --permanent; firewall-cmd --reload
```

Installation and Configuration (Ubuntu)

Installation and Configuration with mariabackup as sst (Ubuntu)

Setup Node 1:

```
apt update
apt install -y mariadb-server mariadb-backup
```

```
nano /etc/mysql/mariadb.conf.d/z_settings.cnf
```

```
[mysqld]
binlog_format=ROW
default-storage-engine=innodb
innodb_autoinc_lock_mode=2
bind-address=0.0.0.0

## for better performance // Attention: You might loose data on power outage
innodb_flush_log_at_trx_commit=2

## Galera Provider Configuration
wsrep_on=ON
wsrep_provider=/usr/lib/galera/libgalera_smm.so

## Galera Cluster Configuration
wsrep_cluster_name="test_cluster-<your shortcut e.g. r1>"
wsrep_cluster_address="gcomm://10.135.0.3,10.135.0.4,10.135.0.5"
wsrep_node_address=10.135.0.3

## Galera Synchronization Configuration
wsrep_sst_method=mariabackup
wsrep_sst_auth = mariabackup:mypassword
```

```
mysql
```

```
CREATE USER 'mariabackup'@'localhost' IDENTIFIED BY 'mypassword';
GRANT RELOAD, PROCESS, LOCK TABLES, REPLICATION CLIENT ON *.* TO 'mariabackup'@'localhost';
quit
```

```
systemctl stop mariadb
galera_new_cluster
mysql
```

```
show status like 'wsrep%';
quit
```

Setup Node 2:

```
apt update
apt install -y mariadb-server mariadb-backup
```

```
nano /etc/mysql/mariadb.conf.d/z_settings.cnf
```

```
[mysqld]
binlog_format=ROW
default-storage-engine=innodb
innodb_autoinc_lock_mode=2
bind-address=0.0.0.0

## for better performance // Attention: You might loose data on power outage
innodb_flush_log_at_trx_commit=2

## Galera Provider Configuration
wsrep_on=ON
wsrep_provider=/usr/lib/galera/libgalera_smm.so

## Galera Cluster Configuration
wsrep_cluster_name="test_cluster-<your shortcut e.g. r1>"
wsrep_cluster_address="gcomm://10.135.0.3,10.135.0.4,10.135.0.5"
wsrep_node_address=10.135.0.4

## Galera Synchronization Configuration
wsrep_sst_method=mariabackup
wsrep_sst_auth = mariabackup:mypassword
```

```
systemctl restart mariadb
```

Setup Node 3:

```
apt update
apt install -y mariadb-server mariadb-backup
```

```
nano /etc/mysql/mariadb.conf.d/z_settings.cnf
```

```
[mysqld]
binlog_format=ROW
default-storage-engine=innodb
innodb_autoinc_lock_mode=2
bind-address=0.0.0.0

## for better performance // Attention: You might loose data on power outage
innodb_flush_log_at_trx_commit=2

## Galera Provider Configuration
wsrep_on=ON
wsrep_provider=/usr/lib/galera/libgalera_smm.so

## Galera Cluster Configuration
wsrep_cluster_name="test_cluster-<your shortcut e.g. r1>"
wsrep_cluster_address="gcomm://10.135.0.3,10.135.0.4,10.135.0.5"
```



```
wsrep_node_address=10.135.0.5

## Galera Synchronization Configuration
wsrep_sst_method=mariabackup
wsrep_sst_auth = mariabackup:mypassword
```

```
systemctl restart mariadb
```

1. Node started nicht nach Crash, z.B. Stromausfall

Warum startet nicht ?

```
## node ist in einem nicht-geordneten Zustand.
## und hat Angst ;o), dass die anderen Nodes u.U. weiter sind
## Ziel sollte sein, die letzte Node als 1. zu starten mit -> galera_new_cluster
```

Wie beheben ?

```
## Nach Informationen im Status gucken
systemctl status mariadb

## Nach Informationen in den Logs schauen
journalctl -u mariadb
## Speziell kann ich rausfiltern
journalctl -u mariadb | grep -i error

## In der Regel steht safe_to_bootstrap auf 0
# Fixend
/var/lib/grastate.dat
safe_to_bootstrap = 1 # setzen

## Immer nur ausführen, wenn es nur eine Node 1 !! git
galera-new-cluster
```

Upgrade Minor/Major

Minor z.B. 10.3.1 -> 10.3.2

- Always do a deinstallation of old version first, before installing new version
- <https://mariadb.com/kb/en/upgrading-between-minor-versions-with-galera-cluster/>

Major 10.3 -> 10.4

- <https://mariadb.com/kb/en/upgrading-from-mariadb-103-to-mariadb-104-with-galera-cluster/>

Determine rights size of gcache - e.g. for maintenance

Exercise

```
## SSH-Session 1 to node 1:
sudo su -
cd /usr/src
wget https://downloads.mysql.com/docs/sakila-db.tar.gz
tar xzvf sakila-db.tar.gz
cd sakila-db
```

```
-- # SSH-Session 2 on node 1:
-- mysql>
set @start := (select sum(VARIABLE_VALUE/1024/1024) from information_schema.global_status where VARIABLE_NAME like 'WSREP%bytes'); do sleep(60); set @end := (select sum(VARIABLE_VALUE/1024/1024) from information_schema.global_status where VARIABLE_NAME like 'WSREP%bytes'); set @gcache := (select SUBSTRING_INDEX(SUBSTRING_INDEX(@@GLOBAL.wsrep_provider_options,'gcache.size = ',-1), 'M', 1)); select round((@end - @start),2) as `MB/min`, round((@end - @start),2) * 60 as `MB/hour`, @gcache as `gcache Size(MB)`, round(@gcache/round((@end - @start),2),2) as `Time to full(minutes)`;
```

```
## SSH-Session 1 on node 1:
mysql < sakila-schema.sql; mysql < sakila-data.sql
```

```
## See the result in session 2
```

How long can we be down ?

```
set @start := (select sum(VARIABLE_VALUE/1024/1024) from information_schema.global_status where VARIABLE_NAME like 'WSREP%bytes'); do sleep(60); set @end := (select sum(VARIABLE_VALUE/1024/1024) from information_schema.global_status where VARIABLE_NAME like 'WSREP%bytes'); set @gcache := (select SUBSTRING_INDEX(SUBSTRING_INDEX(@@GLOBAL.wsrep_provider_options,'gcache.size = ',-1), 'M', 1)); select round((@end - @start),2) as `MB/min`, round((@end - @start),2) * 60 as `MB/hour`, @gcache as `gcache Size(MB)`, round(@gcache/round((@end - @start),2),2) as `Time to full(minutes)`;
```

Explanation ?

Yes, we can calculate it upside down ;o) How much downtime can i afford ?

Reference: https://fromdual.com/gcache_size_in_galera_cluster

```
##my.cnf
[mysqld]
wsrep_provider_options="gcache.size=256M"
```

Hold time = GCache size / Replication Rate.

Where:

Replication Rate = Amount of replicated data / time. (ime in seconds)

Amount of replicated data = (wsrep_replicated_bytes + wsrep_received_bytes) after the maintenance window - (wsrep_replicated_bytes + wsrep_received_bytes) before the maintenance window.

The amount of replicated data for the customer's case = 7200MB.

Now, we can find out how much GCache (default 128M) can handle for the customer's case:

Hold time = 128MB / (7200MB / 4h) = 128MB / 0.5 MB = 256s.

Then, we can calculate the right GCache size value to handle the maintenance window by the following formula:

GCache = Maintenance window * Replication Rate = 14400s * 0.5 MB. GCache = 7200MB.

In other words, the right GCache size should be equivalent to (or not less than) the amount of replicated data.

Reference:

- <https://severalnines.com/blog/how-avoid-sst-when-adding-new-node-mysql-galera-cluster/>

Monitoring Galera - What to monitor

What to monitor ?

- Single nodes
- wsrep_local_state_comment -> synced # should be synced (but problem if it will not be on synced for a longer time)
- wsrep_cluster_status # Primary // Non-Primary is BAD !!
- Does the node run
- Necessary ports available

set threads in galera according to cert_deps

Sequentially in Parallel

Last, you might monitor wsrep_cert_deps_distance. It will tell you the average distance between the lowest and highest sequence number, values a node can potentially apply in parallel.

Basically, this is the optimal value to set wsrep_slave_threads or wsrep_applier_threads, since it's pointless to assign more slave threads than the number of transactions that can be applied in parallel.

What to monitor

- <https://galeracluster.com/library/training/tutorials/galera-monitoring.html>

pmmdemo (Percona Management und Monitoring Tool)

- [https://pmmdemo.percona.com/graph/d/pmm-home/home-dashboard?orgId=1&refresh=1m&var-interval=\\$__auto_interval_interval&var-environment=All&var-node_name=pxc-80-2&var-service_name=All&var-cluster=All&var-replication_set=All&var-database=All&var-node_type=All&var-service_type=All&var-username=All&var-schema=All](https://pmmdemo.percona.com/graph/d/pmm-home/home-dashboard?orgId=1&refresh=1m&var-interval=$__auto_interval_interval&var-environment=All&var-node_name=pxc-80-2&var-service_name=All&var-cluster=All&var-replication_set=All&var-database=All&var-node_type=All&var-service_type=All&var-username=All&var-schema=All)
- <https://www.percona.com/doc/percona-monitoring-and-management/deploy/server/docker.html>

Tips & Tricks

Set hostname on systemd-Systems

```
## you have to be root
hostnamectl set-hostname mariadb1.training.local
## so that you will see it in your current prompt
su -
hostnamectl
```

Frisches Datenverzeichnis anlegen

Walkthrough (Centos/RHEL/Rocky)

```
## Schritt 1: Prepare
systemctl stop mariadb
cd /var/lib
## eventually delete old back dir
rm -fR /var/lib/mysql.bkup
##
mv mysql mysql.bkup

## Schritt 2: Fresh
mysql_install_db --user=mysql
chown -R mysql:mysql mysql
chmod -R g+rx,o+rx mysql
restorecon -rv /var/lib/mysql

## Schritt 3: Start
systemctl start mariadb
```

Walkthrough (Debian/Ubuntu)

```
## Schritt 1: Prepare
systemctl stop mariadb
cd /var/lib
## eventually delete old back dir
rm -fR /var/lib/mysql.bkup
##
mv mysql mysql.bkup

## Schritt 2: Fresh
mysql_install_db --user=mysql

## not sure, but safe !
chown -R mysql:mysql mysql
chmod -R g+rx,o+rx mysql

## Schritt 3: Start
systemctl start mariadb
```

In den Root-Benutzer wechseln

```
## einloggen als normaler Benutzer z.B. benutzer: kurs (wenn ich unter kurs eingeloggt bin)
sudo su -
## Eingeben des Passworts des Benutzers

### oder sudo -i (interaktiv)
sudo -i
## Eingeben des Passworts des normalen Benutzers
```

Service Debuggen

Walkthrough

```
## Dienst startet nicht / nach Ausführen von systemctl restart wird Fehlermeldung ausgegeben
systemctl restart mariadb.service
```

```
## Schritt 1 : status -> was sagen die logs (letzte 10 Zeilen)
systemctl status mariadb.service

## Nicht fündig-> Schritt 2:
journalctl -xeu mariadb.service

## Nicht fündig -> Schritt 3:
## Spezifisches Log von Dienst suchen
## und evtl. LogLevel von Dienst hochsetzen
## z.B. bei mariadb (durch Internetrecherche herausfinden)
less /var/log/mysql/error.log

## Nicht fündig -> Schritt 5
## Allgemeines Log
## Debian/Ubuntu
/var/log/syslog
## Redhat/Centos & SLES (OpenSuSE)
/var/log/messages
```

Wie verfahren bei SystemV

Wie bei walkthrough aber ab Schritt 4

Find error in logs quickly

```
cd /var/log/mysql
## -i = case insensitive // egal ob gross- oder kleingeschrieben
cat error.log | grep -i error
```

Schweizer Taschenmesser der Suche

```
## Fehler ist gummitulpe - option - falsch in Konfigurationsdatei, aber wo ?
grep -r gummitulpe /etc
## mit zeilennummer
grep -nr gummitulpe /etc
## mit zeilennummer und egal ob gross oder kleingeschrieben
grep -inr GUMMITULPE /etc
```

online schema change without blocking

```
pt-online-schema-change --execute --alter-foreign-keys-method 'auto' --alter "ADD COLUMN c1 INT" D=sakila,t=actor
```

Locking

Table Locks

Example

```
mysql> LOCK TABLES people write,people_data write;
Query OK, 0 rows affected (0.00 sec)

mysql> UNLOCK TABLES
-> ;
Query OK, 0 rows affected (0.00 sec)

## LOCK TABLES .... WRITE
-- We cannot read + write in other session

## LOCK TABLES ..... READ
-- We cannot write, but read in other session
```

Exercise

Vorbereitung:

- 2 Session die als root mit mysql - client am Server eingeloggt sind

Step 1: Session 1

```
use sakila;
LOCK TABLES actor write;
```

Step 2: Session 2

```
use sakila;
update actor set last_name = 'CHAMPION' where actor_id = 200;
----
```

Step 3: Session 1

```
UNLOCK TABLES
-- now update in Step2 should work
```

Implicit Locks

How do they work in general

- Implicit locks are done by InnoDB itself
- We can only partly influence them.

Who wants what ?

```
<who?, what?, how?, granted?>
```

Explanation (a bit clumsy)

- IS and IX (intended share and intended write lock)
- IS and IX can be triggered on SQL
- IX -> SUFFIX -> FOR UPDATE (this triggers a IX lock)
- IX and IS are the first step (on table layer)
- After that IX -> tries to get an write lock on row-level -> X
- Works unless there is another X
- IX and IS is not retrieved on TABLE spaced operations (construction --- alter)

Lock Type compatibility matrix

	X	IX	S	IS
X	Conflict	Conflict	Conflict	Conflict
IX	Conflict	Compatible	Conflict	Compatible
S	Conflict	Conflict	Compatible	Compatible
IS	Conflict	Compatible	Compatible	Compatible

The best explanation across the internet ;o)

- http://stackoverflow.com/questions/25903764/why-is-an-ix-lock-compatible-with-another-ix-lock-in-innodb|IX_and_IS-locks

Many people, both visitors and curators, enter the museum.
The visitors want to view paintings, so they wear a badge labeled "IS".
The curators may replace paintings, so they wear a badge labeled "IX".
There can be many people in the museum at the same time, with both types of badges.
They don't block each other.

During their visit, the serious art fans will get as close to the painting as they can,
and study it for lengthy periods.

They're happy to let other art fans stand next to them before the same painting.
They therefore are doing SELECT ... LOCK IN SHARE MODE and they have "S" lock,
because they at least don't want the painting to be replaced while they're studying it.

The curators can replace a painting, but they are courteous to the serious art fans,
and they'll wait until these viewers are done and move on.
So they are trying to do SELECT ... FOR UPDATE (or else simply UPDATE or DELETE).

They will acquire "X" locks at this time, by hanging a little sign up saying "exhibit being redesigned."
The serious art fans want to see the art presented in a proper manner, with nice lighting and some descriptive plaque.
They'll wait for the redesign to be done before they approach (they get a lock wait if they try).

Identify Deadlocks in innodb

Prerequisite

```
2 sessions (connected to same server):  
Session 1  
Session 2  
  
sakila database is installed
```

Session 1:

```
## Start transaction and lock row by updating it  
mysql>use sakila;  
mysql>begin;  
mysql>update actor set last_name='Johnsson' where actor_id = 200;  
  
## Attention: not commit yet please, leave transaction open
```

Session 2:

```
## Start transactio and try to update same row  
mysql>use sakila;  
mysql>begin;  
mysql>update actor set last_name='John' where actor_id = 200;  
  
## Now update cannot be done, because of lock from session one
```

Session 1: / or new Session 3

```
## find out who blocks session 2  
mysql>use information_schema;  
## find out trx_id of session 2  
mysql>select * from innodb_trx;  
## assuming we have trx_id 1468;  
## now we find out what is blocking this transaction  
mysql>select * from innodb_locks_waits;  
MariaDB [information_schema]> select * from innodb_lock_waits;  
+-----+-----+-----+-----+  
| requesting_trx_id | requested_lock_id | blocking_trx_id | blocking_lock_id |  
+-----+-----+-----+-----+  
| 1469              | 1469:66:3:201     | 1468             | 1468:66:3:201     |  
+-----+-----+-----+-----+  
1 row in set (0.001 sec)  
  
## either additional infos  
select * from innodb_trx where trx_id = 1468;  
  
## get thread_id -> e.g. 50  
  
## or directly kill this transaction  
show processlist;  
kill 50;
```

With this command you can also see pending locks

```
show engine innodb status \G
```

There is more (activate: do not only show last deadlock)

```
## by setting this, deadlocks are written to error log
set global innodb_print_all_deadlocks = ON
```

Refs:

- <https://fromdual.com/mariadb-deadlocks>

Refs (3 important tables)

- https://mariadb.com/kb/en/information-schema-innodb_lock_waits-table/ (most important one)
- https://mariadb.com/kb/en/information-schema-innodb_locks-table/
- https://mariadb.com/kb/en/information-schema-innodb_trx-table/

Optimal use of indexes

Index-Types

- Spatial (only for spatial - geo - date)
- unique
- none-unique
- primary
- fulltext

Describe and indexes

Walkthrough

Step 1:

```
## Database and Table with primary key
create database descindex;
use descindex;
create table people (id int unsigned auto_increment, first_name varchar(25), last_name varchar(25), primary key
(id), passcode mediumint unsigned);
## add an index
## This will always !! translate into an alter statement.
create index idx_last_name_first_name on people (last_name,first_name)
##
create unique index idx_passcode on people (passcode)

desc people;
```

Field	Type	Null	Key	Default	Extra
id	int(10) unsigned	NO	PRI	NULL	auto_increment
first_name	varchar(25)	YES		NULL	
last_name	varchar(25)	YES		NULL	
passcode	mediumint(8) unsigned	YES		NULL	

```
4 rows in set (0.01 sec)
```

Step 2:

```
## Add simple combined index on first_name, last_name
create index idx_first_name_last_name on people (first_name, last_name);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
desc people;
```

```
-- show the column where the combined index starts (MUL = Multi)
```

Field	Type	Null	Key	Default	Extra
id	int(10) unsigned	NO	PRI	NULL	auto_increment
first_name	varchar(25)	YES	MUL	NULL	
last_name	varchar(25)	YES		NULL	
passcode	mediumint(8) unsigned	YES		NULL	

```
4 rows in set (0.01 sec)
```

Step 3:

```
## Add a unique index on passcode
create index idx_passcode on people (passcode)
mysql> desc people;

-- Line with UNI shows this indexes.
+-----+-----+-----+-----+-----+-----+
| Field      | Type                | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int(10) unsigned    | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(25)         | YES  | MUL | NULL    |                |
| last_name  | varchar(25)         | YES  |     | NULL    |                |
| passcode   | mediumint(8) unsigned | YES  | UNI | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Step 4:

```
## Get to know all your indexes on a table
show indexes for people
mysql> show index from people;
+-----+-----+-----+-----+-----+-----+-----+-----+
-+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name          | Seq_in_index | Column_name | Collation | Cardinality | Sub_part |
| Packed | Null | Index_type | Comment | Index_comment |
+-----+-----+-----+-----+-----+-----+-----+-----+
-+-----+-----+-----+-----+-----+
| people |      0 | PRIMARY          |      1 | id          | A         |          0 | NULL    |
| NULL   |      | BTREE           |      |             |           |           |         |
| people |      0 | idx_passcode      |      1 | passcode    | A         |          0 | NULL    |
| NULL   | YES  | BTREE           |      |             |           |           |         |
| people |      1 | idx_first_name_last_name |      1 | first_name  | A         |          0 | NULL    |
| NULL   | YES  | BTREE           |      |             |           |           |         |
| people |      1 | idx_first_name_last_name |      2 | last_name   | A         |          0 | NULL    |
| NULL   | YES  | BTREE           |      |             |           |           |         |
+-----+-----+-----+-----+-----+-----+-----+-----+
-+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Find out indexes

Show index from table

```
create database showindex;
use showindex;
CREATE TABLE `people` (
  `id` int(10) unsigned NOT NULL AUTO_INCREMENT,
  `first_name` varchar(25) DEFAULT NULL,
  `last_name` varchar(25) DEFAULT NULL,
  `passcode` mediumint(8) unsigned DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `idx_passcode` (`passcode`),
  KEY `idx_first_name_last_name` (`first_name`,`last_name`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1
show index from people
```

Show create table

```
show create table peple
```

show index from

```
show index from contributions
```


Index and Functions

No function can be used on an index:

```
explain select * from actor where substring(last_name,1,1) = 'A';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | actor | NULL | ALL | NULL | NULL | NULL | NULL | 200 | 100.00 | Using where |
```

Workaround with generated columns

```
## 1. Create Virtual Column with upper
MariaDB [sakila]> alter table actor add last_name_upper varchar(45) AS (upper(last_name)) VIRTUAL;
Query OK, 0 rows affected (0.006 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [sakila]> create index idx_upper on actor (last_name_upper);
Query OK, 0 rows affected (0.008 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [sakila]> explain select * from actor where last_name_upper like 'A%';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | possible_keys | key | key_len | ref | rows | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | actor | range | idx_upper | idx_upper | 183 | NULL | 7 | Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

Now we try to search the very same

```
explain select * from actor where last_name_upper like 'A%';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | actor | NULL | range | idx_last_name_upper | idx_last_name_upper | 183 | NULL | 7 | 100.00 | Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

Reference

- <https://mariadb.com/kb/en/generated-columns/>
- <https://mariadb.com/kb/en/slow-query-log-overview/>

Index and Likes

1. like 'Will%' - Index works

explain select last_name from donors where last_name like 'Will%';

2. like '%iams' - Index does not work

```
-- because like starts with a wildcard
explain select last_name from donors where last_name like '%iams';
```

3. How to fix 3, if you are using this often ?

```
## Walkthrough
## Step 1: modify table
alter table donors add last_name_reversed varchar(70) GENERATED ALWAYS AS (reverse(last_name));
create index idx_last_name_reversed on donors (last_name_reversed);

## besser - Variante 2 - untested
alter table donors add last_name_reversed varchar(70) GENERATED ALWAYS AS (reverse(last_name)), add index
idx_last_name_reversed on donors (last_name_reversed);

## Step 2: update table - this take a while
update donors set last_name_reversed = reversed(last_name)
## Step 3: work with it
select last_name,last_name_reversed from donor where last_name_reversed like reverse('%iams');

## Version 2 with pt-online-schema-change
```

profiling-get-time-for-execution-of.query

- Get better values, how long queries take

Example

```
set profiling = 1
## Step 2 - Execute query
select last_name as gross from donors where last_name like lower('WILLI%')

## Step 3 - Show profiles
show profiles;
+-----+-----+-----+
| Query_ID | Duration | Query |
+-----+-----+-----+
| 1 | 0.01993525 | select last_name as gross from donors where last_name like lower('WILLI%') |
4 rows in set, 1 warning (0.00 sec)

## Step 4 - Show profile for a specific query
mysql> show profile for query 1;
+-----+-----+
| Status | Duration |
+-----+-----+
| starting | 0.000062 |
| checking permissions | 0.000006 |
| Opening tables | 0.000021 |
| init | 0.000017 |
| System lock | 0.000007 |
| optimizing | 0.000007 |
| statistics | 0.000083 |
| preparing | 0.000012 |
| executing | 0.000004 |
| Sending data | 0.022251 |
| end | 0.000005 |
| query end | 0.000008 |
| closing tables | 0.000007 |
| freeing items | 0.001792 |
| cleaning up | 0.000016 |
+-----+-----+
15 rows in set, 1 warning (0.00 sec)
```

Find out cardinality without index

Find out cardinality without creating index

```
select count(distinct donor_id) from contributions;

select count(distinct(vendor_city)) from contributions;
+-----+
```

```
| count(distinct (vendor_city)) |  
+-----+  
| 1772 |  
+-----+  
1 row in set (4.97 sec)
```

Dokumentation (Releases)

Identify Long-Term Support Releases

- <https://mariadb.com/kb/en/mariadb-server-release-dates/>

Dokumentation (Galera)

MariaDB Galera Cluster

- <http://schulung.t3isp.de/documents/pdfs/mariadb/mariadb-galera-cluster.pdf>

MySQL Galera Cluster

- <https://galeracluster.com/downloads/>

Dokumentation (Sandbox mode)

mariadb-client Sandbox-mode and mariadb-dump (in newer versions)

- <https://mariadb.org/mariadb-dump-file-compatibility-change/>

Dokumentation (functions)

Built-In Functions

- <https://mariadb.com/kb/en/built-in-functions/>

Dokumentation (Summary Tables)

Summary Tables

- <https://mysql.rjweb.org/doc.php/summarytables>

Dokumentation / Library

Server System Variables

- <https://mariadb.com/docs/server/server-management/variables-and-modes/server-system-variables#list-of-server-system-variables>

Mehrere Instanzen in Linux starten

Multiple instances Linux with systemd

Übung: Mehrere MariaDB-Instanzen unter Rocky Linux (mit SELinux)

Ziel:

- 2. Instanz `mariadb@node1.service`
- eigenes Datadir
- eigene Config
- SELinux-konform
- systemd-Template nutzen

Rocky Linux verwendet SELinux (default **Enforcing**) → wenn du das nicht berücksichtigst, startet die zweite Instanz NICHT.

0. Voraussetzungen

```
sudo systemctl status mariadb  
systemctl list-unit-files | grep mariadb@
```

Template sollte aussehen wie:

```
mariadb@.service  enabled
```

1. Einmalig für alle Instanzen (template)

```
systemctl edit mariadb@.service
```

```
[Service]
ProtectHome=false
Environment='MYSQLD_MULTI_INSTANCE=--defaults-file=/etc/my%I.cnf \
              --socket=/var/run/mysql/mysql-%I.sock \
              --datadir=/var/lib/mysql-%I \
              --skip-networking'
```

```
## dann speichern
```

2. Eigenes Datadir anlegen (SELinux-konform!)

MariaDB verwendet Standard-Label:

- Datenverzeichnisse → `mysqld_db_t`
- Laufzeitdateien (Sockets) → `mysqld_var_run_t`

Wir erstellen:

```
sudo mkdir -p /var/lib/mysql-node1
sudo chown -R mysql:mysql /var/lib/mysql-node1
```

Jetzt SELinux richtig labeln:

```
sudo semanage fcontext -a -t mysqld_db_t "/var/lib/mysql-node1(/.*)?"
sudo restorecon -Rv /var/lib/mysql-node1
```

Wichtig: Ohne dieses Label startet die Instanz NICHT.

3. Runtime-Pfad für Socket vorbereiten (SELinux)

Rocky nutzt `idR` `/var/run/mysql`. Wir erstellen einen separaten Socket-Namen für `node1`:

```
sudo mkdir /var/run/mysql
chown mysql:mysql /var/run/mysql
sudo touch /var/run/mysql/mysql-node1.sock
sudo chown mysql:mysql /var/run/mysql/mysql-node1.sock
sudo semanage fcontext -a -t mysqld_var_run_t "/var/run/mysql/mysql-node1.sock"
sudo restorecon -v /var/run/mysql/mysql-node1.sock
```

(Das echte Socket wird später vom Daemon erstellt → aber das Label muss vorher existieren.)

4. Konfigurationsdatei für die neue Instanz anlegen

Auf RHEL/Rocky liegen instanz-spezifische Dateien standardmäßig unter:

Wir erstellen:

```
sudo tee /etc/mynode1.cnf >/dev/null <<'EOF'
[mariadb]
## Eigenes Datadir
datadir=/var/lib/mysql-node1

## Eigener Socket
socket=/var/run/mysql/mysql-node1.sock

## Eigener Port
port=3307

## Eigener PID-File
pid-file=/var/run/mysql/mysql-node1.pid
EOF
```

```
sudo semanage fcontext -a -t mysqld_etc_t "/etc/mynode1.cnf"
sudo restorecon -v /etc/mynode1.cnf
```

5. Instanz starten

```
sudo systemctl start mariadb@node1.service
sudo systemctl status mariadb@node1.service
```

Wenn SELinux etwas blockiert hätte, würdest du es sehen mit:

```
sudo journalctl -u mariadb@node1.service -xe
sudo grep AVC /var/log/audit/audit.log
```

6. Mit der neuen Instanz verbinden

```
sudo mysql --socket=/var/run/mysqld/mysqld-node1.sock -u root
```

Testbefehle:

```
SHOW VARIABLES LIKE 'port';
CREATE DATABASE multi_instance_test;
SHOW DATABASES;
EXIT;
```

7. SELinux Troubleshooting (falls nötig)

Sollte trotzdem ein AVC auftauchen, hilf dir selbst:

A) Zeige die letzten SELinux-Fehler

```
sudo ausearch -m avc -ts recent
```

B) Automatische Policy generieren (Notlösung!)

```
sudo grep mariabdbd /var/log/audit/audit.log | audit2allow -M mariadb_multi
sudo semodule -i mariadb_multi.pp
```

8. Instanz stoppen / Boot-Start aktivieren

```
sudo systemctl stop mariadb@node1.service
sudo systemctl start mariadb@node1.service
sudo systemctl enable mariadb@node1.service
```

Reference:

- <https://mariadb.com/docs/server/server-management/starting-and-stopping-mariadb/systemd#interacting-with-multiple-mariadb-server-processes>

Misc

Bis zu welcher Größe taugt mariadb

Lastprofil (InnoDB)

- 20% Schreiben und 80% Lesen

Datengrenze (Empfehlung) bei CPU gebundener Last

- Häufig verwendete Daten müssen in den innodb_buffer_pool passen
- Beispiel. Häufige Nutzdaten sind z.B. 200GB, die gesamten 2 TB

Tablespaces - Begrenzung aufgrund der Page - Größe:

Speziell:

bei 16 KByte Pages - Max 64 TB pro Tablespace.
oder maximal 1017 columns

Maximale Row-Länge bei Verwendung von InnoDB

- 50% der Page-Größe -> 16 Kbytes -> 8 Kbytes

Bei grossen Datenmengen

- innodb_log_file_size

Exercise

mysql

```
create schema if not exists training;
use training;

SET GLOBAL innodb_default_row_format='dynamic';

SET SESSION innodb_strict_mode=ON;

CREATE OR REPLACE TABLE tab (
  col1 varchar(40) NOT NULL,
  col2 varchar(40) NOT NULL,
  col3 varchar(40) NOT NULL,
  col4 varchar(40) NOT NULL,
  col5 varchar(40) NOT NULL,
  col6 varchar(40) NOT NULL,
  col7 varchar(40) NOT NULL,
  col8 varchar(40) NOT NULL,
  col9 varchar(40) NOT NULL,
  col10 varchar(40) NOT NULL,
  col11 varchar(40) NOT NULL,
  col12 varchar(40) NOT NULL,
  col13 varchar(40) NOT NULL,
  col14 varchar(40) NOT NULL,
  col15 varchar(40) NOT NULL,
  col16 varchar(40) NOT NULL,
  col17 varchar(40) NOT NULL,
  col18 varchar(40) NOT NULL,
  col19 varchar(40) NOT NULL,
  col20 varchar(40) NOT NULL,
  col21 varchar(40) NOT NULL,
  col22 varchar(40) NOT NULL,
  col23 varchar(40) NOT NULL,
  col24 varchar(40) NOT NULL,
  col25 varchar(40) NOT NULL,
  col26 varchar(40) NOT NULL,
  col27 varchar(40) NOT NULL,
  col28 varchar(40) NOT NULL,
  col29 varchar(40) NOT NULL,
  col30 varchar(40) NOT NULL,
  col31 varchar(40) NOT NULL,
  col32 varchar(40) NOT NULL,
  col33 varchar(40) NOT NULL,
  col34 varchar(40) NOT NULL,
  col35 varchar(40) NOT NULL,
  col36 varchar(40) NOT NULL,
  col37 varchar(40) NOT NULL,
  col38 varchar(40) NOT NULL,
  col39 varchar(40) NOT NULL,
  col40 varchar(40) NOT NULL,
  col41 varchar(40) NOT NULL,
  col42 varchar(40) NOT NULL,
  col43 varchar(40) NOT NULL,
  col44 varchar(40) NOT NULL,
  col45 varchar(40) NOT NULL,
```

col146 varchar(40) NOT NULL,
col147 varchar(40) NOT NULL,
col148 varchar(40) NOT NULL,
col149 varchar(40) NOT NULL,
col150 varchar(40) NOT NULL,
col151 varchar(40) NOT NULL,
col152 varchar(40) NOT NULL,
col153 varchar(40) NOT NULL,
col154 varchar(40) NOT NULL,
col155 varchar(40) NOT NULL,
col156 varchar(40) NOT NULL,
col157 varchar(40) NOT NULL,
col158 varchar(40) NOT NULL,
col159 varchar(40) NOT NULL,
col160 varchar(40) NOT NULL,
col161 varchar(40) NOT NULL,
col162 varchar(40) NOT NULL,
col163 varchar(40) NOT NULL,
col164 varchar(40) NOT NULL,
col165 varchar(40) NOT NULL,
col166 varchar(40) NOT NULL,
col167 varchar(40) NOT NULL,
col168 varchar(40) NOT NULL,
col169 varchar(40) NOT NULL,
col170 varchar(40) NOT NULL,
col171 varchar(40) NOT NULL,
col172 varchar(40) NOT NULL,
col173 varchar(40) NOT NULL,
col174 varchar(40) NOT NULL,
col175 varchar(40) NOT NULL,
col176 varchar(40) NOT NULL,
col177 varchar(40) NOT NULL,
col178 varchar(40) NOT NULL,
col179 varchar(40) NOT NULL,
col180 varchar(40) NOT NULL,
col181 varchar(40) NOT NULL,
col182 varchar(40) NOT NULL,
col183 varchar(40) NOT NULL,
col184 varchar(40) NOT NULL,
col185 varchar(40) NOT NULL,
col186 varchar(40) NOT NULL,
col187 varchar(40) NOT NULL,
col188 varchar(40) NOT NULL,
col189 varchar(40) NOT NULL,
col190 varchar(40) NOT NULL,
col191 varchar(40) NOT NULL,
col192 varchar(40) NOT NULL,
col193 varchar(40) NOT NULL,
col194 varchar(40) NOT NULL,
col195 varchar(40) NOT NULL,
col196 varchar(40) NOT NULL,
col197 varchar(40) NOT NULL,
col198 varchar(40) NOT NULL,
col199 varchar(40) NOT NULL,
col1100 varchar(40) NOT NULL,
col1101 varchar(40) NOT NULL,
col1102 varchar(40) NOT NULL,
col1103 varchar(40) NOT NULL,
col1104 varchar(40) NOT NULL,
col1105 varchar(40) NOT NULL,
col1106 varchar(40) NOT NULL,
col1107 varchar(40) NOT NULL,
col1108 varchar(40) NOT NULL,
col1109 varchar(40) NOT NULL,
col1110 varchar(40) NOT NULL,
col1111 varchar(40) NOT NULL,
col1112 varchar(40) NOT NULL,
col1113 varchar(40) NOT NULL,

col114 varchar(40) NOT NULL,
col115 varchar(40) NOT NULL,
col116 varchar(40) NOT NULL,
col117 varchar(40) NOT NULL,
col118 varchar(40) NOT NULL,
col119 varchar(40) NOT NULL,
col120 varchar(40) NOT NULL,
col121 varchar(40) NOT NULL,
col122 varchar(40) NOT NULL,
col123 varchar(40) NOT NULL,
col124 varchar(40) NOT NULL,
col125 varchar(40) NOT NULL,
col126 varchar(40) NOT NULL,
col127 varchar(40) NOT NULL,
col128 varchar(40) NOT NULL,
col129 varchar(40) NOT NULL,
col130 varchar(40) NOT NULL,
col131 varchar(40) NOT NULL,
col132 varchar(40) NOT NULL,
col133 varchar(40) NOT NULL,
col134 varchar(40) NOT NULL,
col135 varchar(40) NOT NULL,
col136 varchar(40) NOT NULL,
col137 varchar(40) NOT NULL,
col138 varchar(40) NOT NULL,
col139 varchar(40) NOT NULL,
col140 varchar(40) NOT NULL,
col141 varchar(40) NOT NULL,
col142 varchar(40) NOT NULL,
col143 varchar(40) NOT NULL,
col144 varchar(40) NOT NULL,
col145 varchar(40) NOT NULL,
col146 varchar(40) NOT NULL,
col147 varchar(40) NOT NULL,
col148 varchar(40) NOT NULL,
col149 varchar(40) NOT NULL,
col150 varchar(40) NOT NULL,
col151 varchar(40) NOT NULL,
col152 varchar(40) NOT NULL,
col153 varchar(40) NOT NULL,
col154 varchar(40) NOT NULL,
col155 varchar(40) NOT NULL,
col156 varchar(40) NOT NULL,
col157 varchar(40) NOT NULL,
col158 varchar(40) NOT NULL,
col159 varchar(40) NOT NULL,
col160 varchar(40) NOT NULL,
col161 varchar(40) NOT NULL,
col162 varchar(40) NOT NULL,
col163 varchar(40) NOT NULL,
col164 varchar(40) NOT NULL,
col165 varchar(40) NOT NULL,
col166 varchar(40) NOT NULL,
col167 varchar(40) NOT NULL,
col168 varchar(40) NOT NULL,
col169 varchar(40) NOT NULL,
col170 varchar(40) NOT NULL,
col171 varchar(40) NOT NULL,
col172 varchar(40) NOT NULL,
col173 varchar(40) NOT NULL,
col174 varchar(40) NOT NULL,
col175 varchar(40) NOT NULL,
col176 varchar(40) NOT NULL,
col177 varchar(40) NOT NULL,
col178 varchar(40) NOT NULL,
col179 varchar(40) NOT NULL,
col180 varchar(40) NOT NULL,
col181 varchar(40) NOT NULL,


```

col182 varchar(40) NOT NULL,
col183 varchar(40) NOT NULL,
col184 varchar(40) NOT NULL,
col185 varchar(40) NOT NULL,
col186 varchar(40) NOT NULL,
col187 varchar(40) NOT NULL,
col188 varchar(40) NOT NULL,
col189 varchar(40) NOT NULL,
col190 varchar(40) NOT NULL,
col191 varchar(40) NOT NULL,
col192 varchar(40) NOT NULL,
col193 varchar(40) NOT NULL,
col194 varchar(40) NOT NULL,
col195 varchar(40) NOT NULL,
col196 varchar(40) NOT NULL,
col197 varchar(40) NOT NULL,
col198 varchar(40) NOT NULL,
PRIMARY KEY (col1)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

ERROR 1118 (42000): Row size too large (> 8126). Changing some columns to TEXT or BLOB may help. In current row format, BLOB prefix of 0 bytes is stored inline.

- Reference: <https://mariadb.com/kb/en/innodb-limitations/#:~:text=Limitations%20on%20Size,-With%20the%20exception&text=MariaDB%20imposes%20a%20row%2Dsize,file%20size%20limit%20of%202GB.>

Referenz:

- <https://mariadb.com/kb/en/innodb-limitations/>

Ausweg

- RocksDB (Sharding), TokuDB, ColumnStore, Partition

Database Functions/Procedure/Triggers/Events

Events

Preparation

```

-- scheduler is not there
SHOW PROCESSLIST;

-- Prüfen ob scheduler läuft
show variables like '%event%';

```

Setup in config

```

## Redhat:
##/etc/my.cnf.d/server.cnf

## Ubuntu
## /etc/mysql/mariadb.conf.d/50-server.cnf

```

```

[mysqld]
event_scheduler

```

```

systemctl restart mariad
mariadb

```

```

-- scheduler appears
SHOW PROCESSLIST;

-- Events anzeigen
show events;

```

preparation

```

create schema if not exists schulung;
USE schulung;
CREATE TABLE messages (
    id INT PRIMARY KEY AUTO_INCREMENT,
    message VARCHAR(255) NOT NULL,
    created_at DATETIME NOT NULL
);

```

One time event

```

USE schulung;
CREATE EVENT IF NOT EXISTS test_event_01
ON SCHEDULE AT CURRENT_TIMESTAMP
DO
    INSERT INTO messages(message,created_at)
    VALUES('Test MariaDB Event 1',NOW());

SELECT * FROM messages;

```

Show all events from a specific database

```

SHOW EVENTS FROM schulung;

```

Show all events in active database

```

USE schulung;
SHOW EVENTS;

```

One time event but preserved (runs 1 minute later, after created = interval)

```

-- To keep the event after it is expired, you use the  ON COMPLETION PRESERVE clause.

CREATE EVENT test_event_02
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1 MINUTE
ON COMPLETION PRESERVE
DO
    INSERT INTO messages(message,created_at)
    VALUES('Test MariaDB Event 2',NOW());

```

Same version, but with begin end block

```

DELIMITER /
CREATE EVENT test_event_03
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1 MINUTE
ON COMPLETION PRESERVE
DO
    BEGIN
        INSERT INTO messages(message,created_at)
        VALUES('Test MariaDB Event 3',NOW());
    END /

-- Delimiter wieder auf ';' setzen
DELIMITER ;

SELECT * FROM messages;

```

Recurring Example

```

CREATE EVENT test_event_04
ON SCHEDULE EVERY 1 MINUTE
STARTS CURRENT_TIMESTAMP
ENDS CURRENT_TIMESTAMP + INTERVAL 1 HOUR
DO
    INSERT INTO messages(message,created_at)
    VALUES('Test MariaDB recurring Event',NOW());

SELECT * FROM messages;

// after 1 minute
SELECT * FROM messages;

```

Drop an event

```
DROP EVENT IF EXIST test_event_03;
```

Set event-scheduler in config / my.cnf / my.ini

```

[mysqld]
event-scheduler

## after that restawrt
systemctl restart mariadb

```

Fix timezone problem Linux (when time is displayed wrong)

```

## 09:32 UTC should be 11:32 CEST
## also root ausführen
timedatectl list-timezones | grep 'Europe/Berlin';
timedatectl set-timezone Europe/Berlin
timedatectl
date
systemctl restart mariadb
mysql
mysql>select now();
mysql>--- time should ok now

```

Exercise

Step 1: Events einschalten

```

-- scheduler is not there
SHOW PROCESSLIST;

-- Prüfen ob scheduler läuft
show variables like '%event%';
set GLOBAL event_scheduler = on;

-- scheduler appears
SHOW PROCESSLIST;

-- Events anzeigen
show events;

```

Step 2: create messages for test

```

create schema if not exists schulung;
USE schulung;
CREATE TABLE messages (
    id INT PRIMARY KEY AUTO_INCREMENT,
    message VARCHAR(255) NOT NULL,
    created_at DATETIME NOT NULL
);

```

Step 3: create recurring event

```
use schulung;
CREATE EVENT test_event_03
ON SCHEDULE EVERY 1 MINUTE
STARTS CURRENT_TIMESTAMP
ENDS CURRENT_TIMESTAMP + INTERVAL 1 HOUR
DO
    INSERT INTO messages(message,created_at)
    VALUES('Test MariaDB recurring Event',NOW());

show events;
```

```
use schulung;
SELECT * FROM messages;
```

```
use schulung;
-- after 1 minute
SELECT * FROM messages;
```

Event: Beispiel täglich Daten löschen

Voraussetzung:

- Event das regelmäßig in messages schreibt.

[Vorbereitung Tabelle](#) [Erstellung Event zum Schreiben](#)

Übung - zum Daten löschen (Event)

```
## Beispiele
## Alles älter als 1 Jahr löschen
delete FROM messages where createt_at < (now() - interval 2 year)

## alles älter als 10 Minuten
## Er rechnet aber 00:00 Uhr
delete FROM messages where created_at < (now() - interval 10 minute)
```

```
use schulung;
DELIMITER /
CREATE EVENT eraser_10min
ON SCHEDULE
    EVERY 10 MINUTE
    STARTS (TIMESTAMP(CURRENT_DATE))
DO
    BEGIN
        delete FROM messages where created_at < (now() - interval 10 minute);
    END /

DELIMITER ;
```

```
## mach 10 Minuten, z.B. 16:20
## Hat er etwas gelöscht
select * from messages;
```

Event: Beispiel mit Daten aggregieren

Schritt 1:

```
use sakila; select first_name from actor;
```

Schritt 2: Gruppieren nach Vorname

```
select first_name from actor group by first_name;
```

Schritt 3: auch noch die Anzahl ausgeben

```
select first_name,count(*) as namenzahl from actor group by first_name;
```

Schritt 4: nach buchstaben

```
select LEFT(first_name,1),count(*) as namenzahl from actor group by LEFT(first_name,1);
```

Schritt 5: Tabellenstruktur anlegen

```
CREATE TABLE actor_stats (    first_letter CHAR(1),    actor_count INT,    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

Schritt 6: Testweise Daten einfügen

```
INSERT INTO actor_stats (first_letter, actor_count)
SELECT LEFT(first_name,1),count(*) as namenzahl
FROM actor
GROUP BY LEFT(first_name,1);
```

```
select * from actor_stats;
truncate actor_stats;
select * from actor_stats;
```

Schritt 7: Procedure

```
DELIMITER $$
CREATE PROCEDURE aggregate_actors_by_letter()
BEGIN
    TRUNCATE actor_stats;
    INSERT INTO actor_stats (first_letter, actor_count)
    SELECT LEFT(first_name,1),count(*) as namenzahl
    FROM actor
    GROUP BY LEFT(first_name,1);

END$$
DELIMITER ;
```

Schritt 8: Testweise aufrufen

```
CALL aggregate_actors_by_letter;
```

Schritt 9: event

```
DELIMITER /
CREATE EVENT actor_stats
ON SCHEDULE EVERY 1 MINUTE
DO
    BEGIN
        CALL aggregate_actors_by_letter();
    END /

-- Delimiter wieder auf ';' setzen
DELIMITER ;

show events;
TRUNCATE actor_stats;
```

-- nach 1 minute SELECT * FROM actor_stats;

Procedures

```
use sakila;
DELIMITER //
```

```
CREATE PROCEDURE simpleproc (OUT param1 INT)
BEGIN
    SELECT COUNT(*) INTO param1 FROM actor;
END;
//
```

```
DELIMITER ;
```

```
CALL simpleproc(@a);
```

```
SELECT @a;
```

```
+-----+
| @a    |
+-----+
| 200   |
+-----+
```

Functions

```
CREATE FUNCTION hello (s CHAR(20))
RETURNS CHAR(50) DETERMINISTIC
RETURN CONCAT('Hello, ',s,'!');
```

```
SELECT hello('world');
```

```
+-----+
| hello('world') |
+-----+
| Hello, world!  |
+-----+
```

Triggers

Step 1: Create the structure

```
use sakila;
create table countries (
    country_id int auto_increment,
    name varchar(50) not null,
    primary key(country_id)
);
```

```
INSERT INTO countries (name) values ('Germany'), ('Austria');
```

```
create table country_stats(
    country_id int,
    year int,
    population int,
    primary key (country_id, year),
    foreign key(country_id)
    references countries(country_id)
);
```

```
INSERT INTO country_stats (country_id, year, population) values (1,2020,100000);
```

```
create table population_logs(
    log_id int auto_increment,
    country_id int not null,
    year int not null,
    old_population int not null,
    new_population int not null,
    updated_at timestamp default current_timestamp,
```

```
    primary key (log_id)
);
```

Create the trigger (Optional)

```
create trigger before_country_stats_update
before update on country_stats
for each row
insert into population_logs (
    country_id,
    year,
    old_population,
    new_population
)
values (
    old.country_id,
    old.year,
    old.population,
    new.population
);
```

Step 2: Create trigger (the same) but with BEGIN/END - Block

```
delimiter //
create trigger before_country_stats_update
before update on country_stats
for each row

BEGIN
SET @anfang = 1;
insert into population_logs (
    country_id,
    year,
    old_population,
    new_population
)
values (
    old.country_id,
    old.year,
    old.population,
    new.population
);
END //

delimiter ;
```

Step 3: Run a test

```
update
    country_stats
set
    population = 1352617399
where
    country_id = 1 and
    year = 2020;

-- what's the new result

select * from population_logs;
```

Continue although we have an error (Optional)

```
delimiter //
create or replace trigger before_country_stats_update
```

```

before update on country_stats
for each row

BEGIN
DECLARE CONTINUE HANDLER FOR 1146
    SET @a= 1;

SET @anfang = 1;
insert into population_logs2(
    country_id,
    year,
    old_population,
    new_population
)
values (
    old.country_id,
    old.year,
    old.population,
    new.population
);
END//

delimiter ;

```

```
update country_stats set population = 1352617399 where country_id = 1 and year = 2020;
```

Ref:

- <https://mariadb.com/kb/en/trigger-overview/>

Ref with walkthrough

- <https://mariadb.com/kb/en/trigger-overview/>

Architecture of MariaDB

Query Cache Usage and Performance

Performance query cache

- Always try to optimize innodb with disabled query cache first (innodb_buffer_pool)
- If you use query_cache system can only use on CPU-Core. !!

How to enable query cache

```

## have_query_cache means compiled in mysql
## query_cache_type off means not enable by config
-- query cache is disabled
mysql> show variables like '%query_cache%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| have_query_cache | YES |
| query_cache_limit | 1048576 |
| query_cache_min_res_unit | 4096 |
| query_cache_size | 1048576 |
| query_cache_type | OFF |
| query_cache_wlock_invalidate | OFF |
+-----+-----+
6 rows in set (0.01 sec)

root@trn01:/etc/mysql/mysql.conf.d# tail mysqld.cnf
[mysqld]
pid-file           = /var/run/mysqld/mysqld.pid
socket             = /var/run/mysqld/mysqld.sock
datadir            = /var/lib/mysql
log-error          = /var/log/mysql/error.log

```



```
## By default we only accept connections from localhost
bind-address      = 0.0.0.0

## Disabling symbolic-links is recommended to prevent assorted security risks
symbolic-links=0
query-cache-type=1

systemctl restart mysql

mysql> show variables like '%query_cache%';
+-----+-----+
| Variable_name          | Value |
+-----+-----+
| have_query_cache       | YES   |
| query_cache_limit      | 1048576 |
| query_cache_min_res_unit | 4096  |
| query_cache_size       | 1048576 |
| query_cache_type       | ON    |
| query_cache_wlock_invalidate | OFF  |
+-----+-----+
6 rows in set (0.01 sec)

mysql> show status like '%Qcache%';
+-----+-----+
| Variable_name          | Value |
+-----+-----+
| Qcache_free_blocks    | 1      |
| Qcache_free_memory    | 1031832 |
| Qcache_hits           | 0      |
| Qcache_inserts        | 0      |
| Qcache_lowmem_prunes  | 0      |
| Qcache_not_cached     | 0      |
| Qcache_queries_in_cache | 0      |
| Qcache_total_blocks   | 1      |
+-----+-----+
8 rows in set (0.00 sec)

## status in session zurücksetzen.
mysql> flush status;
Query OK, 0 rows affected (0.00 sec)
```

Performance bottleneck - mutex

<https://mariadb.com/de/resources/blog/flexible-mariadb-server-query-cache/>

Something planned ?

- Nope ;o(Demand is new
- You might be able to use Demand together with maxscale
- Refer to: <https://mariadb.com/de/resources/blog/flexible-mariadb-server-query-cache/>

A mutual exclusion object (mutex) is a programming object that allows multiple program threads to share a resource (such as a folder) but not simultaneously. Mutex is set to unlock when the data is no longer needed or when a routine is finished. Mutex creates a bottleneck effect. The blocking means only one query can look at the Query Cache at a time and other queries must wait. A query that must wait to look in the cache only to find it isn't in the cache will be slowed instead of being accelerated.

Optimizer-Basics

General

- All optimizer today are cost-based

Cost-Based

```
## How much costs are needed to get the information
```

Installation

Installation SLES15

- https://downloads.mariadb.org/mariadb/repositories/#distro=SLES&distro_release=sles15-amd64-sles15&mirror=timor&version=10.5

Does mariadb listen to the outside world

How to check ?

```
lsof -i | grep mariadb
## localhost means it does NOT listen to the outside now
## mariabdd 5208          mysql    19u  IPv4  56942      0t0  TCP localhost:mysql (LISTEN)

netstat -tupel
## or
netstat -an
```

How to fix (Ubuntu -> Mariadb Foundation -> 11.4)

```
nano /etc/mysql/mariadb.conf.d/99-server.cnf
```

```
## In Section [mysqld]
## Change bind-address to -> bind-address = 0.0.0.0
[mariabdd]
bind-address = 0.0.0.0
```

```
systemctl restart mariadb
systemctl status mariadb
lsof -i
```

Can I bind to multiple addresses ?

```
## from 10.11 it is possible, before not, you have to use 0.0.0.0
Starting with MariaDB 10.11, a comma-separated list of addresses to bind to can be given.
```

- https://mariadb.com/kb/en/server-system-variables/#bind_address

Backup

Use xtrabackup for MariaDB 5.5

For mariadb 5.5 you can use xtrabackup instead of mariabackup

- <https://www.percona.com/doc/percona-xtrabackup/2.4/index.html>

Ready-made-back-scripts

- <https://gist.github.com/skarllot/2576266>

Simple-Backup-Script

Backup Script

```
cat backup-test.sh
#!/bin/bash

DATABASES=$(echo "select schema_name from information_schema.schemata where schema_name != 'performance_schema'
and schema_name != 'information_schema';" | mysql)
for i in $DATABASES
do
    mysqldump $i > /usr/src/dump_$i.sql
done
```

Information Schema / Status / Processes

Show server/session status

Through mysql

```
## in mysql interface (client)
mysql
status;
```

With mysqladmin

```
mysqladmin status
## or if you want to know more
mysqladmin extended status
```

with mysql -> show status

```
## Status within session (status - counters)
mysql> show status;
## Status global (since last reboot/start of mariadb server)
mysql> show global status;
mysql> -- reset session status
mysql> flush status;
## Show session status
mysql> show session status;
```

Kill long running process

```
## Session 1
## sleep for 120 seconds
select sleep(120)

## Session 2
show processlist
## kill process you have identified for sleep(120)
MariaDB [(none)]> show processlist;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host      | db      | Command | Time | State      | Info                    | Progress |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 36 | root | localhost | NULL    | Query   | 0    | starting   | show processlist        | 0.000 |
| 37 | root | localhost | training | Query   | 4    | User sleep | select sleep(120)        | 0.000 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
## take 37
kill 37

## Session 1: query terminates
ERROR 2013 (HY000): Lost connection to MySQL server during query
```

Kill (kickout user) and stop server

```
MariaDB [mysql]> show processlist;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host      | db      | Command | Time | State      | Info                    | Progress |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 30 | root | localhost | mysql   | Sleep   | 10   |            | NULL                    | 0.000 |
| 34 | root | localhost | mysql   | Query   | 0    | starting   | show processlist        | 0.000 |
| 43 | training | localhost | training | Sleep   | 5    |            | NULL                    | 0.000 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

MariaDB [mysql]> quit
Bye
root@its-lu20s04:~# mysql -e 'kill 43' && systemctl stop mariadb
root@its-lu20s04:~#
```

User Rights

Debug and Setup External Connection

Prerequisites

```
client1: 192.168.56.104
server1: 192.168.56.103
```

Step 1: Be sure server is communicating to the outside

```
lsof -i
## should be
*:mysql
```

Step 2: Test connection from client

```
mysqladmin ping -h 192.168.56.103
## on succesful connection also without authentication
echo $?
0 # 0 was success also without proper authentication

## Bad news, if
echo $?
1
## Could not connect at all
```

Step 2a: No connection possible ? check Firewall

```
## Server 1
systemctl status firewalld
firewall-cmd --state
firewall-cmd --list-all # do we see mysql as a service

## no ?
firewall-cmd --get-services
firewall-cmd --add-service=mysql # only in runtime
firewall-cmd --runtime-to-permanent # config - works after reboot

### Recheck with Step 2
```

Step 3: Setup user without grants - Server1

```
## Server 1
mysql> create user ext@192.168.56.104 identified by 'topsecretpassword';
## Doing this twice triggers an weird error
```

Step 3a: test connection from client - Client 1

```
mysql -uext -p -h 192.168.56.103
## on success
mysql>show grants
## should only be usage
mysql>show schemas
```

Step 3b: Add privileges (testing giving all) - Server1

```
## *.* = all databases and all tables
mysql> GRANT ALL ON *.* TO ext@192.168.56.104
```

Step 3c: See, if we have grants - Client 1

```
mysql>show grants
## grants will be shown but do not work yet
## we need to reconnect
mysql>quit
```

```
mysql -uext -p -h 192.168.56.103
mysql> -- now it works
```

Get Rights of user

Root can show rights of a specific user

```
## shows the right of the logged in user (you as a user)
show grants;

## show grants for a specific user
## no need for ' (quotes) if there are not special chars withing
## e.g.
show grants for training@localhost;
## if there are special chars, use quotes
show grants for 'mariadb.sys'@localhost;

## if you want to see rights of a user that has rights from everywhere
show grants for training@'%';
```

If you cannot remember the exact user (user@host) look it up

```
## within mysql client
use mysql
select * from user \G
```

Secure with SSL server/client

Variant 1: Setup 1-way ssl encryption

Create CA and Server-Key

```
## On Server - create ca and certificates
sudo mkdir -p /etc/my.cnf.d/ssl
sudo cd /etc/my.cnf.d/ssl

## create ca.
sudo openssl genrsa 4096 > ca-key.pem

## create ca-certificate
## Common Name: MariaDB CA
sudo openssl req -new -x509 -nodes -days 365000 -key ca-key.pem -out ca-cert.pem

## create server-cert
## Common Name: server1.training.local
## Password: --- leave empty ---
sudo openssl req -newkey rsa:2048 -days 365000 -nodes -keyout server-key.pem -out server-req.pem

## Next process the rsa - key
sudo openssl rsa -in server-key.pem -out server-key.pem

## Now sign the key
sudo openssl x509 -req -in server-req.pem -days 365000 -CA ca-cert.pem -CAkey ca-key.pem -set_serial 01 -out
server-cert.pem
```

Verify certificates

```
openssl verify -CAfile ca-cert.pem server-cert.pem
```

Configure Server

```
## create file
## /etc/my.cnf.d/z_ssl.cnf
[mysqld]
ssl-ca=/etc/my.cnf.d/ssl/ca-cert.pem
```

```

ssl-cert=/etc/my.cnf.d/ssl/server-cert.pem
ssl-key=/etc/my.cnf.d/ssl/server-key.pem
### Set up TLS version here. For example TLS version 1.2 and 1.3 ##
## Starts from mariadb 10.4.6 not possible before. !!!!
tls_version = TLSv1.2,TLSv1.3

## Set ownership
chown -vR mysql:mysql /etc/my.cnf.d/ssl/

```

Restart and check for errors

```

systemctl restart mariadb
journalctl -u mariadb

```

Test connection on client

```

## only if we use option --ssl we will connect with ssl
mysql --ssl -uxyz -p -h <ip-of-server>
mysql>status
SSL:                               Cipher in use is TLS_AES_256_GCM_SHA384

```

Force to use ssl

```

## on server
## now client can only connect, when using ssl
mysql> grant USAGE on *.* to remote@10.10.9.144 require ssl;

```

Variant 2: 1-way ssl-encryption but checking server certificate

Prerequisites

```

server1: 192.168.56.103
client1: 192.168.56.104

```

Copy ca-cert to client

```

## on server1
cd /etc/my.cnf.d/ssl
scp ca-cert.pem kurs@192.168.56.104:/tmp

## on clien1
cd /etc/my.cnf.d
mkdir ssl
cd ssl
mv /tmp/ca-cert.pem .

```

Configure client1 - client -config

```

sudo vi /etc/my.cnf.d/mysql-clients.cnf

Append/edit in [mysql] section:

### MySQL Client Configuration ##
ssl-ca=/etc/my.cnf.d/ssl/ca-cert.pem

### Force TLS version for client too
##tls_version = TLSv1.2,TLSv1.3
#### This option is disabled by default ###
#### ssl-verify-server-cert ###

## only works if you have no self-signed certificate
ssl-verify-server-cert
ssl

## domain-name in hosts setzen
## because in dns
vi /etc/hosts

```

```
192.168.56.103 server1.training.local

## now you to connect with hostname
## otherwise no check against certificate can be done
mysql -uext -p -h server1.training.local

## if it does not work, you get
ERROR 2026 (HY000): SSL connection error: Validation of SSL server certificate failed
```

Variant 3: 2-way - Security (Encryption) - validated on server and client

Client - Create certificate on server

- we are using the same ca as on the server

```
## on server1
cd /etc/my.cnf.d/ssl
## Bitte Common-Name: MariaDB Client
openssl req -newkey rsa:2048 -days 365 -nodes -keyout client-key.pem -out client-req.pem

## process RSA - Key
## Eventually also works without - what does it do ?
## openssl rsa -in client-key.pem -out client-key.pem

## sign certificate with CA
openssl x509 -req -in client-req.pem -days 365 -CA ca-cert.pem -CAkey ca-key.pem -set_serial 01 -out client-cert.pem
```

Client - Zertifikate validieren

```
openssl verify -CAfile ca-cert.pem client-cert.pem
```

Zertifikate für Client zusammenpacken

```
mkdir cl-certs; cp -a client* cl-certs; cp -a ca-cert.pem cl-certs ; tar cvfz cl-certs.tar.gz cl-certs
```

Zertifikate auf Client transferieren

```
scp cl-certs.tar.gz kurs@192.168.56.104:/tmp
```

Zertifikate einrichten

```
## on client1
## cleanup old config
rm /etc/my.cnf.d/ssl/ca-cert.pem

mv /tmp/cl-certs.tar.gz /etc/my.cnf.d/ssl
cd /etc/my.cnf.d; tar xzvf cl-certs.tar.gz

vi mysql-clients.cnf
[mysql]
ssl-ca=/etc/my.cnf.d/cl-certs/ca-cert.pem
ssl-cert=/etc/my.cnf.d/cl-certs/client-cert.pem
ssl-key=/etc/my.cnf.d/cl-certs/client-key.pem
```

Test the certificate

```
## on server1 verify: X509 for user
select user,ssl_type from mysql.user where user='ext'

## connect from client1
## Sollte die Verbindung nicht klappen stimmt auf dem
## Client etwas mit der Einrichtung nicht
mysql -uext -p -h192.168.56.103
mysql> status
```

Ref

- <https://mariadb.com/kb/en/securing-connections-for-client-and-server/>
- <https://www.cyberciti.biz/faq/how-to-setup-mariadb-ssl-and-secure-connections-from-clients/>

Auth with unix_socket

```
mysql>create user training@localhost identified via unix_socket
useradd training
passwd training

## testing
su - training
## mysql
## should not work without password
## Be sure, that use has access to socket
cd /var/lib/mysql
ls -la mysql.socket
```

User- and Permission-concepts (best-practice)

```
## user should have as little permissions as possible
## so many as needed ;o)
MariaDB [mysql]> create database eventplanner;
Query OK, 1 row affected (0.000 sec)

MariaDB [mysql]> create user eventplanner@localhost identified by 'eventplanner';
Query OK, 0 rows affected (0.001 sec)

MariaDB [mysql]> grant all on eventplanner.* to eventplanner@localhost;
Query OK, 0 rows affected (0.003 sec)
```

Setup external access

Testing

```
## Where .104 is the server you want to connect to
## Variante 1
mysqladmin ping -h 192.168.56.104
echo $?
-> 0 // it is possible to reach mysql - server

## Variante 2
mysqladmin ping -h 192.168.56.104
echo $?
-> 1 // i cannot reach mysql-server -> port might close / firewall ?

## or use telnet
telnet 192.168.56.104 3306
```

Checks on MariaDB (Theory)

- Is MariaDB - Server running ?
- Is 3306 port open (exposed to the outside)
- Is firewall open for port 3306
- Is there a valid user, who connect)

Checks on MariaDB (Practical)

```
## Step 1: Running
systemctl status mariadb
## Step 2: Port open ?
lsof -i # does it listen to all interfaces. -> *
        # or an external interface
## Step 3: Firewall open -> see next block
## Step 4: User who can connect ?
```

Checks on Firewall.


```
## Is firwall running and enabled
systemctl status firewalld
firewall-cmd --state

## Is interface setup for usage of firewalld
firewall-cmd --get-active-zones

## Is service "mysql" in zones
firewall-cmd --list-all-zones | less # is it within public - zone -> mysql

## To enable it, if not set
firewall-cmd --add-service=mysql --zone=public --permanent # writes to filesystem config
firewall-cmd --reload # rereads settings from filesystem
```

Setup valid user

```
## on server you want to connect to
mysql> create user extern@'192.168.56.%' identified by 'mysecretpass'
mysql> grant all on sakila.* to extern@'192.168.56.%'
```

```
## alternative with subnet mask
CREATE USER 'maria'@'247.150.130.0/255.255.255.0';
```

Now test from external with mysql

```
mysql -uextern -p -h 192.168.56.104
mysql>show databases;
```

SELinux

Welche Ports sind freigegeben? (MariaDb startet damit)

Welche Ports

```
semanage port -l | grep mysql
```

Neues Datenverzeichnis SELinux bekanntmachen - semanage fcontext

```
mkdir /data
chown mysql:mysql /data
semanage fcontext -a -t mysqldb_t "/data(/.*)?"
restorecon -vr /data
## type _t should mysqldb_t
ls -laZ
```

Probleme mit SELinux erkennen und debuggen

```
## Wenn mariadb nicht startet, dann zunächst Loganalyse
systemctl status mariadb
## Gibt es ERROR - Einträge ?
## Gibt es Permission / Access Denied - Einträge
## Wenn ansonsten alle Rechte stimmen, weisst das Probleme mit SELinux
journalctl -u mariadb | less

## Logs von selinux laufen über den Audit-Daemon
/var/log/audit/audit.log
## Dies können mit sealert analysiert werden
## Wichtig: Geduld haben, die Analyse dauert einen Moment
## auch nach 100% noch abwarten
sealert -a /var/log/audit/audit.log

## Allheilmittel ist meistens
## Setzt den richtigen Context, den SELinux braucht,
```

```
## damit mariadb starten kann
restorecon -rv /var/lib/mysql
```

Database - Objects

Triggers

Step 1: Create the structure

```
use sakila;
create table countries (
    country_id int auto_increment,
    name varchar(50) not null,
    primary key(country_id)
);
```

```
INSERT INTO countries (name) values ('Germany'), ('Austria');
```

```
create table country_stats(
    country_id int,
    year int,
    population int,
    primary key (country_id, year),
    foreign key(country_id)
    references countries(country_id)
);
```

```
INSERT INTO country_stats (country_id, year, population) values (1,2020,100000);
```

```
create table population_logs(
    log_id int auto_increment,
    country_id int not null,
    year int not null,
    old_population int not null,
    new_population int not null,
    updated_at timestamp default current_timestamp,
    primary key(log_id)
);
```

Create the trigger (Optional)

```
create trigger before_country_stats_update
before update on country_stats
for each row
insert into population_logs(
    country_id,
    year,
    old_population,
    new_population
)
values(
    old.country_id,
    old.year,
    old.population,
    new.population
);
```

Step 2: Create trigger (the same) but with BEGIN/END - Block

```
delimiter //
create trigger before_country_stats_update
before update on country_stats
for each row

BEGIN
```

```

SET @anfang = 1;
insert into population_logs(
    country_id,
    year,
    old_population,
    new_population
)
values(
    old.country_id,
    old.year,
    old.population,
    new.population
);
END //

delimiter ;

```

Step 3: Run a test

```

update
    country_stats
set
    population = 1352617399
where
    country_id = 1 and
    year = 2020;

-- what's the new result

select * from population_logs;

```

Continue although we have an error (Optional)

```

delimiter //
create or replace trigger before_country_stats_update
    before update on country_stats
    for each row

BEGIN
    DECLARE CONTINUE HANDLER FOR 1146
        SET @a= 1;

    SET @anfang = 1;
    insert into population_logs2(
        country_id,
        year,
        old_population,
        new_population
    )
    values(
        old.country_id,
        old.year,
        old.population,
        new.population
    );
END//

delimiter ;

```

```

update country_stats set population = 1352617399 where country_id = 1 and year = 2020;

```

Ref:

- <https://mariadb.com/kb/en/trigger-overview/>

Ref with walkthrough

- <https://mariadb.com/kb/en/trigger-overview/>

Functions

```
CREATE FUNCTION hello (s CHAR(20))
  RETURNS CHAR(50) DETERMINISTIC
  RETURN CONCAT('Hello, ',s,'!');
```

```
SELECT hello('world');
```

```
+-----+
| hello('world') |
+-----+
| Hello, world!  |
+-----+
```

Stored Procedure

Example

```
USE sakila;
DELIMITER //

CREATE PROCEDURE simpleproc (OUT param1 INT)
BEGIN
  SELECT COUNT(*) INTO param1 FROM actor;
END;
//

DELIMITER ;

CALL simpleproc(@a);

SELECT @a;
+-----+
| @a    |
+-----+
|      1 |
+-----+
```

Reference

- <https://mariadb.com/kb/en/create-procedure/>

Events

Preparation

```
-- scheduler is not there
SHOW PROCESSLIST;

-- Prüfen ob scheduler läuft
show variables like '%event%';
```

Setup in config

```
## Redhat:
##/etc/my.cnf.d/server.cnf

## Ubuntu
## /etc/mysql/mariadb.conf.d/50-server.cnf
```

```
[mysqld]
event_scheduler
```

```
systemctl restart mariad
mariadb
```

```
-- scheduler appears
SHOW PROCESSLIST;

-- Events anzeigen
show events;
```

preparation

```
create schema if not exists schulung;
USE schulung;
CREATE TABLE messages (
  id INT PRIMARY KEY AUTO_INCREMENT,
  message VARCHAR(255) NOT NULL,
  created_at DATETIME NOT NULL
);
```

One time event

```
USE schulung;
CREATE EVENT IF NOT EXISTS test_event_01
ON SCHEDULE AT CURRENT_TIMESTAMP
DO
  INSERT INTO messages(message,created_at)
  VALUES('Test MariaDB Event 1',NOW());

SELECT * FROM messages;
```

Show all events from a specific database

```
SHOW EVENTS FROM schulung;
```

Show all events in active database

```
USE schulung;
SHOW EVENTS;
```

One time event but preserved (runs 1 minute later, after created = interval)

```
-- To keep the event after it is expired, you use the  ON COMPLETION PRESERVE clause.

CREATE EVENT test_event_02
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1 MINUTE
ON COMPLETION PRESERVE
DO
  INSERT INTO messages(message,created_at)
  VALUES('Test MariaDB Event 2',NOW());
```

Same version, but with begin end block

```
DELIMITER /
CREATE EVENT test_event_03
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1 MINUTE
ON COMPLETION PRESERVE
```

```

DO
  BEGIN
    INSERT INTO messages(message,created_at)
    VALUES('Test MariaDB Event 3',NOW());
  END /

-- Delimiter wieder auf ';' setzen
DELIMITER ;

SELECT * FROM messages;

```

Recurring Example

```

CREATE EVENT test_event_04
ON SCHEDULE EVERY 1 MINUTE
STARTS CURRENT_TIMESTAMP
ENDS CURRENT_TIMESTAMP + INTERVAL 1 HOUR
DO
  INSERT INTO messages(message,created_at)
  VALUES('Test MariaDB recurring Event',NOW());

SELECT * FROM messages;

// after 1 minute
SELECT * FROM messages;

```

Drop an event

```
DROP EVENT IF EXIST test_event_03;
```

Set event-scheduler in config / my.cnf / my.ini

```

[mysqld]
event-scheduler

## after that restawrt
systemctl restart mariadb

```

Fix timezone problem Linux (when time is displayed wrong)

```

## 09:32 UTC should be 11:32 CEST
## also root ausführen
timedatectl list-timezones | grep 'Europe/Berlin';
timedatectl set-timezone Europe/Berlin
timedatectl
date
systemctl restart mariadb
mysql
mysql>select now();
mysql>--- time should ok now

```

Exercise

Step 1: Events einschalten

```

-- scheduler is not there
SHOW PROCESSLIST;

-- Prüfen ob scheduler läuft
show variables like '%event%';
set GLOBAL event_scheduler = on;

-- scheduler appears
SHOW PROCESSLIST;

```

```
-- Events anzeigen
show events;
```

Step 2: create messages for test

```
create schema if not exists schulung;
USE schulung;
CREATE TABLE messages (
  id INT PRIMARY KEY AUTO_INCREMENT,
  message VARCHAR(255) NOT NULL,
  created_at DATETIME NOT NULL
);
```

Step 3: create recurring event

```
use schulung;
CREATE EVENT test_event_03
ON SCHEDULE EVERY 1 MINUTE
STARTS CURRENT_TIMESTAMP
ENDS CURRENT_TIMESTAMP + INTERVAL 1 HOUR
DO
  INSERT INTO messages(message,created_at)
  VALUES('Test MariaDB recurring Event',NOW());

show events;
```

```
use schulung;
SELECT * FROM messages;
```

```
use schulung;
-- after 1 minute
SELECT * FROM messages;
```

Upgrade

MariaDB Upgrade 10.3 (Centos) -> 10.4 (Mariadb.org)

Prerequisites

```
Ubuntu 20.04
MariaDB-Server from Distri

Install new 10.4 from Mariadb.org
```

Prepare

- Create backup of system (with mariabackup and/or mysqldump)

Steps

```
## 1. systemctl stop mariadb
## 2. dnf remove mariadb-*
## 3. Doublecheck if components left: apt list --installed | grep mariadb
## 4. Setup repo for mariadb
## 5. dnf install MariaDB-server

## 7. systemctl enable --now mariadb # enable for next reboot and start immediately
## necessary for redhat

## 8. Perform mysql_upgrade
## On centos/redhat mysql_upgrade need to be done
mysql_upgrade

## 9. Check if it was succesfull
cat /var/lib/mysql_upgrade_info
```

Important - Check mysql - configuration structure

```
## Which directories are loaded in
/etc/mysql/my.cnf

## Eventually move files to the right directory
## As needed in migration from 10.3 (Distri) to 10.4 (mariadb.org) on Ubuntu 20.04
```

Documentation

- <https://mariadb.com/kb/en/upgrading-from-mariadb-103-to-mariadb-104/>
- https://mariadb.com/kb/en/mysql_upgrade/

MariaDB Upgrade 10.4 -> 10.5 (Centos)

```
## Step 0;
## Sicherung anlegen (mysqldump / mariabackup)

## Step 1:
## Change version in
## or where you have your repo definition
## Change 10.4 -> 10.5
/etc/yum.repos./MariaDB.repo

## Step 2:
systemctl stop mariadb

## Step 3
sudo yum remove MariaDB-server

## Step 4
sudo yum install MariaDB-server
yum list --installed | grep MariaDB # sind alle Versionen gleich ! Wichtig !
sudo yum update ## Achtung: abweichend von Doku MariaDB

## Step 4.5
## Check if old config files were saved as .rpmsave after delete of package 10.4
cd /etc/my.cnf.d
ls -la
## e.g.
mv server.cnf.rpmsave server.cnf

## Step 5:
systemctl start mariadb
systemctl enable mariadb
mysql_upgrade # After that mysql_upgrade_info will be present in /var/lib/mysql with version-info
```

Reference

- <https://mariadb.com/kb/en/upgrading-from-mariadb-104-to-mariadb-105/>

MariaDB Upgrade 5.5 -> 10.5

- <https://mariadb.com/kb/en/upgrading-between-major-mariadb-versions/>

Performance

io-Last/CPU-Last

IO-gebundene - Last (Input/Output)

```
Gegeben wenn:
- Hoher waiting wert in top (wa-wert in CPU-Liste)
- + Hohelast    1,5, 15 min    1,2    1.5    2 (Load) -> top
```

CPU-Gebundene - Last


```
Gegeben wenn:
- NUR: Hohe Last -> Wert in top -> 2 1.5 0.5 (Load)
- Waiting-wert: 0
```

Views and performance

General

```
SHOW CREATE VIEW
```

Views and Algorithms

- Views can use 3 algorithms:
 - merge
 - simple rewrites (translates the query)
- temptable
 - Creates a temptable to retrieve information
 - (In this case no indexes can be used - was the case in older versions)
- Shows up in explain with derived
- undefined
 - MySQL chooses, if to use merge or temptable
 - prefers merge over temptable if possible

Example

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table      | type | possible_keys | key  | key_len | ref  | rows | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | PRIMARY    | <derived2> | ALL  | NULL          | NULL | NULL    | NULL | 33  | NULL  |
| 2  | DERIVED    | task       | ALL  | NULL          | NULL | NULL    | NULL | 33  | NULL  |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Handling (best practice)

- You can define the algorithm when creating the view
- If you define merge and mysql cannot handle it
 - you will get a warning

Example of handling

```
mysql> CREATE ALGORITHM=MERGE VIEW priority_counts AS SELECT priority_id, COUNT(1) AS quantity FROM task GROUP BY
priority_id;
Query OK, 0 rows affected, 1 warning (0.12 sec)

mysql> SHOW WARNINGS;
+-----+-----+-----+
| Level | Code | Message                                                                                               |
+-----+-----+-----+
| Warning | 1354 | View merge algorithm can't be used here for now (assumed undefined algorithm) |
+-----+-----+-----+
1 row in set (0.08 sec)
```

Reference

- Ref: <https://dba.stackexchange.com/questions/54481/determining-what-algorithm-mysql-view-is-using>

Partitions and Explain

Walkthrough (Version 1) - RANGE

```
-- EXPLAIN PARTITIONS

DROP TABLE IF EXISTS audit_log;
CREATE TABLE audit_log (
  yr      YEAR NOT NULL,
  msg     VARCHAR(100) NOT NULL)
ENGINE=InnoDB
```

```

PARTITION BY RANGE (yr) (
    PARTITION p0 VALUES LESS THAN (2010),
    PARTITION p1 VALUES LESS THAN (2011),
    PARTITION p2 VALUES LESS THAN (2012),
    PARTITION pmax VALUES LESS THAN MAXVALUE);
INSERT INTO audit_log(yr,msg) VALUES (2005,'2005'), (2006,'2006'), (2011,'2011'), (2020,'2020');
EXPLAIN PARTITIONS SELECT * from audit_log WHERE yr in (2011,2012)\G

```

Walkthrough (Version 1) - RANGE - testing DATA DIR

```
ALTER TABLE audit_log REORGANIZE PARTITION p0,p1,p2,p3 INTO (
PARTITION p0 VALUES LESS THAN (2010) DATA DIRECTORY = '/home/kurs/mysql/',
PARTITION p1 VALUES LESS THAN (2011) DATA DIRECTORY = '/home/kurs/mysql/',
PARTITION p2 VALUES LESS THAN (2012) DATA DIRECTORY = '/home/kurs/mysql/',
PARTITION p3 VALUES LESS THAN MAXVALUE DATA DIRECTORY = '/home/kurs/mysql/')
```

) ;

```
Query OK, 4 rows affected, 4 warnings (0,021 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
MariaDB [sakila]> show warnings;
```

```

+-----+-----+-----+
| Level | Code | Message                                     |
+-----+-----+-----+
| Warning | 1982 | <DATA DIRECTORY> option ignored for InnoDB partition |
| Warning | 1982 | <DATA DIRECTORY> option ignored for InnoDB partition |
| Warning | 1982 | <DATA DIRECTORY> option ignored for InnoDB partition |
| Warning | 1982 | <DATA DIRECTORY> option ignored for InnoDB partition |
+-----+-----+-----+
4 rows in set (0,000 sec)

```

<https://jira.mariadb.org/browse/MDEV-16594>

<https://github.com/MariaDB/server/commit/031c695b8c865e5eb6c4c09ced404ae08f98430f>

Adding new partition with other DATA DIRECTORY

```
## Step 1: Create table with partitions
```

```
DROP TABLE IF EXISTS audit_log;
```

```
CREATE TABLE audit_log (
```

```
yr      YEAR NOT NULL,
msg     VARCHAR(100) NOT NULL)
```

ENGINE=InnoDB

PARTITION BY RANGE (yr) (

```
PARTITION p0 VALUES LESS THAN (2010),
PARTITION p1 VALUES LESS THAN (2011),
PARTITION p2 VALUES LESS THAN (2012),
PARTITION pmax VALUES LESS THAN MAXVALUE);
```

```
## Step 2: Delete pmax, add new year, and add pmax again
```

```
ALTER TABLE audit_log DROP PARTITION pmax;
```

```
ALTER TABLE audit_log ADD PARTITION (PARTITION p2026 VALUES LESS than (2027) DATA DIRECTORY='/tmp');
```

```
ALTER TABLE audit_log ADD PARTITION (PARTITION pmax VALUES LESS than maxvalue DATA DIRECTORY='/tmp');
```

```
## In filesystem. these are symbolic links in datadir.
```

```
ls -la /var/lib/mysql/sakila/audit_log*
```

```
## files with .isl suffix
```

```
## Reorganize for new diretories does not work but you might want to
change it with vi
```

```
systemctl stop mariadb
```

```
cd /var/lib/mysql/sakila
```

```
vi audit_log#P#pmax.isl
```

```
/tmp/foo/sakila/audit_log#P#pmax.ibd
```

```
systemctl start mariadb
```

Partitions sliced by hash of field

```
CREATE TABLE employees (
  id INT NOT NULL,
  fname VARCHAR(30),
  lname VARCHAR(30),
  hired DATE NOT NULL DEFAULT '1970-01-01',
  separated DATE NOT NULL DEFAULT '9999-12-31',
  job_code INT,
  store_id INT
)
PARTITION BY HASH(store_id)
PARTITIONS 4;
```

Partitioning by datetime

```
CREATE TABLE tbl (
  dt DATETIME NOT NULL, -- or DATE
  ...
  PRIMARY KEY (... , dt),
  UNIQUE KEY (... , dt),
  ...
)
PARTITION BY RANGE (TO_DAYS(dt)) (
  PARTITION start VALUES LESS THAN (0),
  PARTITION from20120315 VALUES LESS THAN (TO_DAYS('2012-03-16')),
  PARTITION from20120316 VALUES LESS THAN (TO_DAYS('2012-03-17')),
  ...
  PARTITION from20120414 VALUES LESS THAN (TO_DAYS('2012-04-15')),
  PARTITION from20120415 VALUES LESS THAN (TO_DAYS('2012-04-16')),
  PARTITION future VALUES LESS THAN MAXVALUE
);
```

3 Phases of DataSize

Phase 1: Table content is small (only some rows)

```
## table scan is quicker than index search
## e.g. 10 entries

## so eventually index is not needed
```

Phase 2: Index is good !!

```
## performance gain by using index
## Step 1: Obtaining id's from index (primary key id)
## Step 2: Retrieving data
```

Phase 3: Index is not improve performance / or would makes performance worse

```
Step 1: lookup in index:
1
70
1040
2100
35000
-> there is a lot of space (other rows) in between.

Step 2: Lookup data, but a lot lookups needed

-> random reads
-> So mysql might be better off to do a table scan.
```

Slow Query Log

Walkthrough (MariaDB from 10.11)

```
## Step 1
## /etc/my.cnf.d/server.cnf
## or: debian /etc/mysql/mariadb.conf.d/50-server.cnf
[mysqld]
slow-query-log
```

```
systemctl restart mariadb
```

```
mariadb
```

```
-- kleinst mögliche Zeit 0.000001 Sekunden
SET GLOBAL log_slow_query_time=0.000001;
-- in der session setzen, damit das sofort funktioniert
SET log_slow_query_time=0.000001;
-- Achtung, steht nach nächstem Neustart wieder auf 10 Sekunden (Default)
```

```
-- prüfen
select @@GLOBAL.log_slow_query_time;
select @@log_slow_query_time;
```

Walkthrough (before 10.11)

```
## Step 1
## /etc/my.cnf.d/mariadb-server.cnf
## or: debian /etc/mysql/mariadb.conf.d/50-server.cnf
[mysqld]
slow-query-log
```

```
systemctl restart mariadb
```

```
## ODER
## Alternative: Step 1a
mysql>SET GLOBAL slow_query_log = 1
mysql>SET slow_query_log = 1
```

```
## Step 2:
## Zeit festlegen, ab wann eine query langsam ist
## global für den Server (greift erst bei der nächsten Session)
mysql>SET GLOBAL long_query_time = 0.000001;
## und für die aktuelle Session
mysql>SET long_query_time = 0.000001
```

```
## Step 3
## run some time / data
## and look into your slow-query-log
/var/lib/mysql/hostname-slow.log
```

Exercise (mariadb 10.6 from mariadb.org)

```
## Step 1
## /etc/my.cnf.d/server.cnf
[mysqld]
slow-query-log
```

```
## Step 2: restart server
systemctl restart mariadb
mysql
```

```
-- Step 3: set long_query_time (global and in session)
select @@slow_query_log;

-- set and show global
```

```

set global long_query_time = 0.000001;
select @@global.long_query_time;
show global variables like '%long%';

-- (Optional) set and show session (for this session)
set long_query_time = 0.000001;
select @@long_query_time;
show variables like '%long%';

```

```

## Step 4: Import data
cd /usr/src/sakila-db
mysql < sakila-schema.sql
mysql < sakila-data.sql

```

```

## Step 5: what did we log
cd /var/lib/mysql
ls -la server1-slow.log
less server1-slow.log

```

Geschwätzigkeit (Verbosity) erhöhen

Before 10.6.16, 10.11.6

```
SET GLOBAL log_slow_verbosity='query_plan,explain'
```

Best option from 10.6.16 and 10.11.6

```

-- Enable all possible options
SET GLOBAL log_slow_verbosity='full'

```

Queries die keine Indizes verwenden

```
SET GLOBAL log_queries_not_using_indexes=ON;
```

Reference

- <https://mariadb.com/kb/en/slow-query-log-overview/>

profiling-get-time-for-execution-of.query

- Get better values, how long queries take

Example

```

set profiling = 1
## Step 2 - Execute query
select last_name as gross from donors where last_name like lower('WILLI%')

## Step 3 - Show profiles
show profiles;
+-----+-----+-----+
| Query_ID | Duration | Query |
+-----+-----+-----+
| 1 | 0.01993525 | select last_name as gross from donors where last_name like lower('WILLI%') |
4 rows in set, 1 warning (0.00 sec)

## Step 4 - Show profile for a specific query
mysql> show profile for query 1;
+-----+-----+
| Status | Duration |
+-----+-----+
| starting | 0.000062 |
| checking permissions | 0.000006 |
| Opening tables | 0.000021 |
| init | 0.000017 |
| System lock | 0.000007 |
| optimizing | 0.000007 |
| statistics | 0.000083 |

```

```

| preparing          | 0.000012 |
| executing          | 0.000004 |
| Sending data       | 0.022251 |
| end                | 0.000005 |
| query end          | 0.000008 |
| closing tables     | 0.000007 |
| freeing items      | 0.001792 |
| cleaning up        | 0.000016 |
+-----+-----+
15 rows in set, 1 warning (0.00 sec)

```

Replication

Slave einrichten - Centos - old style (master_pos)

Configure master

```

## /etc/my.cnf.d/server.cnf
## /etc/my.cnf.d/mariadb-server.cnf
[mysqld]
log-bin=mariadb-bin
server_id=1
log-basename=master1
binlog-format=row

systemctl stop mariadb
systemctl start mariadb

```

Setup replication user on master

```

CREATE USER 'replication_user'@'%' IDENTIFIED BY 'big3cret';
GRANT REPLICATION SLAVE ON *.* TO 'replication_user'@'%';

```

Slave aufsetzen

```

### Wichtig: möglichst gleiche Version

## repo einrichten von mariadb

## /etc/yum.repos.d/mariadb.repo
## MariaDB 10.4 CentOS repository list - created 2022-01-14 08:34 UTC
## https://mariadb.org/download/
[mariadb]
name = MariaDB
baseurl = https://mirror.kumi.systems/mariadb/yum/10.4/centos8-amd64
module_hotfixes=1
gpgkey=https://mirror.kumi.systems/mariadb/yum/RPM-GPG-KEY-MariaDB
gpgcheck=1
## MariaDB 10.4 CentOS repository list - created 2022-01-14 08:34 UTC
## https://mariadb.org/download/
[mariadb]
name = MariaDB
baseurl = https://mirror.kumi.systems/mariadb/yum/10.4/centos8-amd64
module_hotfixes=1
gpgkey=https://mirror.kumi.systems/mariadb/yum/RPM-GPG-KEY-MariaDB
gpgcheck=1

## MariaDB - Server installieren
dnf install -y MariaDB-server

## server - config von master rüberspielen
## auf server
cd /etc
tar cvfz my.cnf.d.tar.gz my.cnf.d

```

```

scp my.cnf.d.tar.gz kurs@192.168.56.104:/tmp

## auf slave ausrollen
cd /etc
mv my.cnf.d my.cnf.d.bkup
mv /tmp/my.cnf.d.tar.gz .
tar cvfz my.cnf.d.tar.gz

## config anpassen
## /etc/my.cnf.d/server.cnf
## pr: /etc/my.cnf.d/mariadb-server.cnf
[mysqld]

innodb-buffer-pool-size=3G
innodb-flush-method=O_DIRECT

## Enable slow-query-log
slow-query-log

server_id=2

## only necessary, if you want the slave to
## become master later on
log-bin=mariadb-bin
binlog-format=row
log-slave-updates=1
log-basename=slave1

## server restarten
systemctl stop mariadb
systemctl start mariadb

```

backup auf master ausspielen und auf slave kopieren

```

mysqldump --all-databases --single-transaction --events --routines --master-data=2 --flush-logs --delete-master-logs > /usr/src/master-dump.sql

scp /usr/src/master-dump.sql kurs@192.168.56.104:/tmp

```

auf slave backup einspielen einspielen und konfigurieren

```

## vi /root/.my.cnf
[client]
password=mysupersecret

mv /tmp/master-dump.sql /usr/src
mysql < master-dump.sql

## vi /root/master.sql
CHANGE MASTER TO
  MASTER_HOST='192.168.56.103',
  MASTER_USER='replication_user',
  MASTER_PASSWORD='bigs3cret',
  MASTER_PORT=3306,
  MASTER_LOG_FILE='master1-bin.000002',
  MASTER_LOG_POS=389,
  MASTER_CONNECT_RETRY=10;

mysql < /root/master.sql

```

slave starten

```

mysql>slave start
mysql>show slave status \G

```

Ref:

- <https://mariadb.com/kb/en/setting-up-replication/>

Skip-Counter

```
SET GLOBAL SQL_SLAVE_SKIP_COUNTER = 1;  
START SLAVE;
```

MaxScale installieren**Why do Loadbalancing with MaxScale ?**

- Cluster node transparent to application
 - Application does not see single nodes
- If one node fails you will have no downtime
 - In opposite: To talking to this node directly

License Implications since 2.x

- MariaDB MaxScale >= 2.0 is licensed under MariaDB BSL.
- maximum of three servers in a commercial context.
 - Any more, and you'll need to buy their commercial license.
- MariaDB MaxScale 2.1.0 will be released under BSL 1.1 from the start
- Each release transitions in about max 4 years to GPL

Current version

- Current Version is maxscale 6 (05/2023)

The MaxScale load-balancer and its components

- Routers
- Listeners
- Filters
- Servers (backend database server)

Filters

- Logging Filters
- Statement rewriting filters
- Result set manipulation filters
- Pipeline control filters
 - e.g. tee and send to a second server
- Ref: <https://mariadb.com/kb/en/mariadb-maxscale-6-regex-filter/>

Documentation - maxctrl

- <https://mariadb.com/kb/en/mariadb-maxscale-6-maxctrl/>

Installation and Setup**Installation**

```
apt update  
apt install apt-transport-https  
  
## Setting up the repos  
curl -sS https://downloads.mariadb.com/MariaDB/mariadb_repo_setup | sudo bash  
## Installing maxscale  
apt install maxscale
```

Setup (Part 1: MaxScale db-user)

- Do this on one of the galera nodes

- Adjust IP !! // best-> private ip from maxscale -> e.g. 10.135.0.30

```
## IP FROM MAXSCALE
## Setup privileges on cluster nodes
## It is sufficient to set it on one node, because
## it will be synced to all the other nodes
## on node 1
CREATE USER 'maxscale'@'10.135.0.x' IDENTIFIED BY 'P@ssw0rd';
##
GRANT SELECT ON mysql.db TO 'maxscale'@'10.135.0.x';
GRANT SELECT ON mysql.user TO 'maxscale'@'10.135.0.x';
GRANT SELECT ON mysql.tables_priv TO 'maxscale'@'10.135.0.x';
##
GRANT SELECT ON mysql.columns_priv TO 'maxscale'@'10.135.0.x';
GRANT SELECT ON mysql.proxies_priv TO 'maxscale'@'10.135.0.x';
##
GRANT SHOW DATABASES ON *.* TO 'maxscale'@'10.135.0.x';
## Needed for maxscale
GRANT SELECT ON mysql.procs_priv TO 'maxscale'@'10.135.0.x';
GRANT SELECT ON mysql.roles_mapping TO 'maxscale'@'10.135.0.x';
```

```
#### Testing if user works
## On maxscale - server
apt update
apt install mariadb-client
## Test the connection
## Verbindung sollte aufgebaut werden
mysql -u maxscale -p -h <ip-eines-der-nodes>
mysql>show databases
```

SETUP (PART 2: CONFIGURATION)

```
## /etc/maxscale.cnf

[maxscale]

threads=auto
syslog=0
maxlog=1
log_warning=1
log_notice=1
log_info=0
log_debug=0

[Galera-Monitor]

type=monitor
module=galeramon
servers=server1,server2,server3
user=maxscale
password=P@ssw0rd
monitor_interval=2000ms
disable_master_failback=1
## Needs to be false, when block sst-method like rsync is used
available_when_donor=false

[RW-Split-Router]

type=service
router=readwritesplit
servers=server1,server2,server3
user=maxscale
password=P@ssw0rd

[RW-Split-Listener]

type=listener
service=RW-Split-Router
```

```
protocol=MariaDBClient
port=3306
```

```
[server1]
type=server
address=142.93.98.60
port=3306
protocol=MariaDBBackend
```

```
[server2]
type=server
address=142.93.103.153
port=3306
protocol=MariaDBBackend
```

```
[server3]
type=server
address=142.93.103.246
port=3306
protocol=MariaDBBackend
```

```
## Start
```

```
systemctl start maxscale
```

```
## What does the log say ?
```

```
## /var/log/maxscale/maxscale.log
```

maxctrl

```
maxctrl list servers
maxctrl set server server1 maintenance
maxctrl clear server server1 maintenance
maxctrl show server server1
maxctrl list services
maxctrl show service ReadWrite-Split-Router
```

Exercise

```
## Create user for client and maxscale on one of the galera nodes
## IP-Adresse from MaxScale
create database if not exists training;
create user joe@'10.135.0.45' identified by 'password';
grant all on training.* to joe@'10.135.0.45';
```

```
## Create same user for client
## adjust x in ip by ip - part of client
create database if not exists training;
create user joe@'10.135.0.x' identified by 'password';
grant all on training.* to joe@'10.135.0.x';
```

```
## Fire Up client-server (used only for mysql-client
## there
## simply take the one from the repo
apt install mariadb-client
## then connect
mysql -ujoe -p -h<ip-of-maxscale>
mysql>create database training; use training; create table trainee (id int,name varchar(50), primary key(id));
insert into trainee (id, name) values (1,'Jochen'); select @@hostname;
mysql>use training; select * from trainee; select @@hostname;
## These should be different servers
```

Reference: MaxScale-Proxy mit Monitoring

[MaxScale MariaDB-Monitor](#)

Walkthrough:Automatic Failover Master Slave

<https://mariadb.com/kb/en/mariadb-maxscale-25-automatic-failover-with-mariadb-monitor/>

Tools & Tricks

Percona-toolkit-Installation - Ubuntu

Walkthrough

```
## Howto
## https://www.percona.com/doc/percona-toolkit/LATEST/installation.html

## Step 1: repo installieren mit deb -paket
wget https://repo.percona.com/apt/percona-release_latest.focal_all.deb;
apt update;
apt install -y curl;
dpkg -i percona-release_latest.focal_all.deb;
apt update;
apt install -y percona-toolkit;
```

pt-query-digest - analyze slow logs

Requires

- Install percona-toolkit

Usage

```
## first enable slow_query_log
set global slow_query_log = on
set global long_query_time = 0.2
## to avoid, that i have to reconnect with new session
set session long_query_time = 0.2

## produce slow query - for testing
select * from contributions where vendor_last_name like 'W%';
mysql > quit

##
cd /var/lib/mysql
## look for awhile wih -slow.log - suffix
pt-query-digest mysql-slow.log > /usr/src/report-slow.txt
less report-slow.txt
```

pt-online-schema-change howto

Requirements

- Install percona-toolkit

What does it do ?

```
## Altering table without blocking them
## Do a dry-run beforehand
pt-online-schema-change --alter "ADD INDEX idx_city (city)" --dry-run D=contributions,t=donors
##
pt-online-schema-change --alter "ADD INDEX idx_city (city)" --execute D=contributions,t=donors
```

Problems -> high cpu load

```
## fine - tune params
## e.g. --max-load
## refer to docs
https://www.percona.com/doc/percona-toolkit/3.0/pt-online-schema-change.html#:~:text=pt%2Donline%2Dschema%2Dchange%20works%20by%20creating%20an%20empty,it%20with%20the%20new%20one.
```

Ubuntu-with-Vagrant

Walkthrough

```
## Step 1: Download git for windows
https://git-scm.com/downloads
## Step 2: Install Virtualbox
https://download.virtualbox.org/virtualbox/6.1.18/VirtualBox-6.1.18-142142-Win.exe
## Step 3: Auf dem Desktop, rechte Maustaste -> git bash here
## in the bash
mkdir myvirtualmachine
vagrant init ubuntu/focal64
vagrant up
## and the you are in the machine (shell)
vagrant ssh
## within machine switch from vagrant user to root without password
sudo su -
## there you go - install whatever
```

Include provisioning in Vagrantfile

```
config.vm.provision "shell", inline: <<-SHELL
  apt-get update
  apt-get install -y mysql-server-5.7 wget
  cd /usr/src
  touch foo
  wget https://downloads.mysql.com/docs/sakila-db.tar.gz
  tar xzvf sakila-db.tar.gz
  cd sakila-db
  mysql < sakila-schema.sql
  mysql < sakila-data.sql
SHELL
end
```

Destroy machine

```
vagrant destroy -f
```

mysql-client

\G Spezialausgabe

```
## Spalten werden als Zeilen angezeigt
## nur im mysql-client
mysql

mysql> show variables like 'bind%' \G
```

Pager

```
## pager innerhalb von mysql verwenden
mysql> pager less
mysql> -- Jetzt wird der Linux Pager less verwendet
mysql> -- so schalte ich ihn wieder ab
mysql> pager
```

Schweizer Such-Taschenmesser grep -r

```
grep -r "PermitRootLogin" /etc
```

Set timezone in Centos 7/8

```
## as root
timedatectl list-timezones | grep 'Europe/Berlin'
```

```
timedatectl set-timezone 'Europe/Berlin'
timedatectl
```

Ist die Netzwerkkarte eingerichtet - nmtui

```
## Grafische Oberfläche auf der Kommandozeile
nmtui
```

User anlegen und passwort vergeben (Centos/Redhat)

```
## als root ausführen
useradd training
passwd training
```

Scripts for deploying galera-cluster to Ubuntu 20.04

- <https://github.com/jmetzger/ansible-galera-cluster-maxscale>

Extras

User Variables

```
## only valid within one session
set @host='localhost';

## You can use it in select
select @host;

## You can use it in the where clause
select mysql.user where host=@host;

## not possible to use it within create user
## DOES NOT WORK !
set @mypass='password';
create user someuser@somehost identified by @mypass;
```

Installation sakila-db

```
cd /usr/src
wget https://downloads.mysql.com/docs/sakila-db.tar.gz
tar xvf sakila-db.tar.gz

cd sakila-db
mariadb < sakila-schema.sql
mariadb < sakila-data.sql
```

Diagnosis and measurement of performance

Best practices to narrow down performance problems

Pre-Requisites

- System is slow

Analyze - Checklist - Step 1

```
## Are there slow queries ?
## look for time
show full processlist

### or time - in seconds
select * from information_schema.processlist where time > 10;
```

Re-Execute SELECT or where from UPDATE / DELETE

```
## Is it still slow ?
## Eventually kill
mysql>show processlist
mysql>--kill <Thread-id>
mysql>-- example
mysql>kill 44
```

Explain what is going on

Explain Select....

Performance and optimization of SQL statements

Do not use '*' whenever possible

Why ?

- You are adding .. to the server:
 - I/O
 - memory
 - CPU
- You are preventing covering indexes

Walkthrough. (Look at the time)

Using '*'

```
## using '*'
pager grep "rows in set";
select * from donors where last_name like 'Willia%'; select * from donors where last_name like 'Willia%';
-- time between 0.02 and 0.04 secs
-- 2424 rows in set (0.02 sec)
-- reset pager
pager

## corresponding Explain (QEP)
explain select * from donors where last_name like 'Willia%';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows |
filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| 1 | SIMPLE | donors | NULL | range | donors_donor_info | donors_donor_info | 213 | NULL | 4748 |
100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

using specific fields

```
pager grep 'rows in set'; select last_name,first_name from donors where last_name like 'Willia%'; pager;
PAGER set to 'grep 'rows in set''
2424 rows in set (0.01 sec)
```

```
explain select last_name,first_name from donors where last_name like 'Willia%';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows |
filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| 1 | SIMPLE | donors | NULL | range | donors_donor_info | donors_donor_info | 213 | NULL | 4748 |
100.00 | Using where; Using index |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

- Uses cover index (indicator in Extra: using index)

Ref:

- <https://www.oreilly.com/library/view/high-performance-mysql/9780596101718/ch04.html>

Optimizer-hints (and why you should not use them)

Tell the optimizer what to do and what not to do

- <https://dev.mysql.com/doc/refman/5.7/en/optimizer-hints.html#optimizer-hints-syntax>

Replication

Replikation Read/Write

- <https://proxysql.com/blog/configure-read-write-split/>

Performance

Best Practices

Indexes

2 Indexes vs. Combined Index

- In most cases a combined index is better than 2 indexes.

Joins

Field-Type

- Do not use varchar() or char() aka string types of join field
- better: integer (unsigned) && same size
 - e.g. actor_id int unsigned

Views

General

- Only use views with merge
- NO temptable please, these CANNOT be indexed.

Where

No functions in where please

- Why ? Index cannot be used.
- example:
 - select first_name from actor where upper(first_name) like 'A%'

Alternative solution

- use a virtual field and index virtual field (possible from mysql > 5.7)
- Massive improvements in mysql 8

Example sys-schema and Reference

Examples

```
mysql> select * from sys.host_summary\G
***** 1. row *****
      host: localhost
      statements: 1347
      statement_latency: 7.55 m
      statement_avg_latency: 336.50 ms
      table_scans: 15
      file_ios: 612857
      file_io_latency: 1.66 m
      current_connections: 1
      total_connections: 7
      unique_users: 1
      current_memory: 0 bytes
      total_memory_allocated: 0 bytes
1 row in set (0.01 sec)
```

Ref:

- <https://github.com/mysql/mysql-sys/blob/master/README.md>

Change schema online (pt-online-schema-change)

- <https://www.percona.com/doc/percona-toolkit/3.0/pt-online-schema-change.html>

Optimizer-Hints

Tell the optimizer what to do and what not to do

- <https://dev.mysql.com/doc/refman/5.7/en/optimizer-hints.html#optimizer-hints-syntax>

Documentation / Literature

MySQL - Performance Blog

- <https://www.percona.com/blog/>

Killing connection

- <https://mariadb.com/kb/en/kill/>

MariaDB - One Script Installation

- <https://mariadb.com/kb/en/mariadb-package-repository-setup-and-usage/>

MariaDB - Information Schema Tables

- <https://mariadb.com/kb/en/information-schema-tables/>

MariaDB - slow query log

- <https://mariadb.com/kb/en/slow-query-log-overview/>

MariaDB - sys - vor 10.6

- <https://github.com/FromDual/mariadb-sys>

Differences Community / Enterprise Version - nearly the same

- <https://fromdual.com/mariadb-enterprise-server-vs-mariadb-community-server>

Hardware Optimization

- <https://mariadb.com/kb/en/hardware-optimization/>

Source-Code MariaDB

- <https://github.com/MariaDB/server>

Effective MySQL

- <https://www.amazon.com/Effective-MySQL-Optimizing-Statements-Oracle/dp/0071782796>

Last Training

- <https://github.com/jmetzger/training-mysql-developers-basics>

Questions and Answers

Questions and Answers

1. Do you recommend Aurora

In my current humble opinion Aurora is a double edged sword. Aurora looks promising for scalability, but a lot of stuff is modified mysql-stuff and in my opinion has a lot of restrictions.

You should be aware, that moving to Aurora might be a task and reverting back even more.

- Refer to: <https://ahmedahamid.com/aurora-mysql/>

I would like to point you to a performance measurement report here:

- <https://galeracluster.com/2019/09/everdata-reports-galera-cluster-outshines-amazon-aurora-and-rds/>

2. Get rid of unattended - upgrades problem (dirty hack)

```
ps aux | grep unatt
kill <process-id-von-unattended-upgrades>
```


3. Archive Data

```
https://www.percona.com/doc/percona-toolkit/LATEST/pt-archiver.html
```

4. Does innodb do defragmentation by itself ?

```
## Some background while doing research.  
## Nil performance benefits of defragmentation in index.  
https://stackoverflow.com/questions/48569979/mariadb-table-defragmentation-using-optimize
```

5. Defragmentation

```
## Optimize table  
ALTER TABLE contributions engine = InnoDB  
  
## mariadb has a patch for defragmentation  
https://mariadb.org/defragmenting-unused-space-on-innodb-tablespace/  
  
## alter table xyz engine=InnoDB - defragmentations  
## but is also invasive.  
## with ibdata1 innodb_file_per_table it lets the size grow
```

6. Is it possible to do select, update, deletes without using innodb_buffer in specific

```
No, this is not possible
```

7. Unit test framework in MySQL

```
No, there is no testing framework with MySQL
```

8. MariaDB - Advantages

- flashback
- Verschlüsselung von Tabellen // mariabackup
- Einige Storage Engine (Aria -> MyISAM - crash-recovery)
- JSON anders implementiert
- galera
- feature: defragmentation

```
MySQL 8 does not:  
decode  
set profiling (still available but deprecated )
```

9. Select without locking

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED ;  
BEGIN ;  
SELECT * FROM TABLE_NAME ;  
COMMIT ;
```

Best filesystem for MariaDB

- <https://mariadb.com/de/resources/blog/what-is-the-best-linux-filesystem-for-mariadb/>

MySQL Do-Nots

mysql-do-nots

1. No function in where (column_name)

```
## Never use a function for the column name in where
## e.g.
select * from donors where upper(last_name) like 'Willia%'
```

Why ?

- Not index can be used

[illegible]